

Introduction to Predictive Modeling

From Classical Regression and Classification to Neural Networks from Scratch

by Dr. Rishikesh Yadav

June 25, 2025

Assistant Professor, School of Mathematical and Statistical Sciences, IIT Mandi, India

Table of Contents – Basics of Predictive Modeling

- 1. Predictive Modeling
 - 1.1 Motivation
 - 1.2 Regression
 - 1.3 Classification
 - 1.4 Gradient Descent Algorithm
 - 1.5 Gradient Descent Algorithm From Scratch for Regression and Classification Task
 - 1.6 Model Validation (Choosing the Best Model)

Table of Contents – Introduction to Neural Networks

2. Neural Networks

2.1 Intro to Neural Networks

2.2 Components of Neural Networks

2.3 Standard Linear and Non-Linear Regressions as Neural Networks

2.4 Unfolding a Two-Layer Neural Network

2.5 Training of NNs (Learning the Weights)

2.6 Some Important Strategies/Considerations When Training a NN

2.7 Building Your Very First Neural Network From Scratch

2.8 Fitting the Neural Network using Tensorflow

2.9 Some key Takeaway

2.10 Motivation for Deep Neural Network (Deep Learning)

Course Objectives

The objectives of this lecture would be:

- Learn the basics of **regression** and **classification** techniques.

Course Objectives

The objectives of this lecture would be:

- Learn the basics of **regression** and **classification** techniques.
- Train **regression** and **classification** using **the gradient descent** algorithm.

Course Objectives

The objectives of this lecture would be:

- Learn the basics of **regression** and **classification** techniques.
- Train **regression** and **classification** using **the gradient descent** algorithm.
- Understand key ideas from scratch in **neural networks** and **deep learning**.

Course Objectives

The objectives of this lecture would be:

- Learn the basics of **regression** and **classification** techniques.
- Train **regression** and **classification** using **the gradient descent** algorithm.
- Understand key ideas from scratch in **neural networks** and **deep learning**.
- Build and train simple **neural networks** from scratch and also using **TensorFlow**.

Course Objectives

The objectives of this lecture would be:

- Learn the basics of **regression** and **classification** techniques.
- Train **regression** and **classification** using **the gradient descent** algorithm.
- Understand key ideas from scratch in **neural networks** and **deep learning**.
- Build and train simple **neural networks** from scratch and also using **TensorFlow**.
- Bridge **traditional statistical regression methods** with **machine learning algorithms**.

Course Objectives

The objectives of this lecture would be:

- Learn the basics of **regression** and **classification** techniques.
- Train **regression** and **classification** using **the gradient descent** algorithm.
- Understand key ideas from scratch in **neural networks** and **deep learning**.
- Build and train simple **neural networks** from scratch and also using **TensorFlow**.
- Bridge **traditional statistical regression methods** with **machine learning algorithms**.

Course Objectives

The objectives of this lecture would be:

- Learn the basics of **regression** and **classification** techniques.
- Train **regression** and **classification** using **the gradient descent** algorithm.
- Understand key ideas from scratch in **neural networks** and **deep learning**.
- Build and train simple **neural networks** from scratch and also using **TensorFlow**.
- Bridge **traditional statistical regression methods** with **machine learning algorithms**.

Note: Throughout the whole workshop lectures, we will be only covering the **supervised learning** task (that means we will have input/output **(labeled)** data). We will not cover any unsupervised, semi-supervised, or reinforcement algorithms.

Some Good References

- Bishop, C. M. (2007). *Pattern Recognition and Machine Learning*. Springer.
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly. [[GitHub companion code](#)]
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. [[Free online](#)]
- Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning.
- Nielsen, M. A. (2015). *Neural Networks and Deep Learning*. [[Free online book](#)]
- Online learning platforms:
 - [Coursera - Deep Learning Specialization](#)
 - [fast.ai - Practical Deep Learning for Coders](#)
 - [DeepLearning.AI](#)

Predictive Modeling

Agenda

1. Predictive Modeling

1.1 Motivation

1.2 Regression

1.3 Classification

1.4 Gradient Descent Algorithm

1.5 Gradient Descent Algorithm From Scratch for Regression and Classification Task

1.6 Model Validation (Choosing the Best Model)

Why Predictive Modeling (1)?

While exploring the data, why are just summary statistics and graphical summaries not enough?

Why Predictive Modeling (1)?

While exploring the data, why are just summary statistics and graphical summaries not enough?

Think About:

- What about prediction?

Why Predictive Modeling (1)?

While exploring the data, why are just summary statistics and graphical summaries not enough?

Think About:

- What about prediction?
- What about uncertainty quantification?

Why Predictive Modeling (1)?

While exploring the data, why are just summary statistics and graphical summaries not enough?

Think About:

- What about prediction?
- What about uncertainty quantification?
- What can we say when we have another set of data from the same population?

Why Predictive Modeling (1)?

While exploring the data, why are just summary statistics and graphical summaries not enough?

Think About:

- What about prediction?
- What about uncertainty quantification?
- What can we say when we have another set of data from the same population?

Why Predictive Modeling (1)?

While exploring the data, why are just summary statistics and graphical summaries not enough?

Think About:

- What about prediction?
- What about uncertainty quantification?
- What can we say when we have another set of data from the same population?

All of them can be solved by performing statistical/machine learning modeling.

Why Predictive Modeling (2)?

- In data science and applied statistics, we often aim to understand, quantify, and **predict** outcomes based on observed variables.

Why Predictive Modeling (2)?

- In data science and applied statistics, we often aim to understand, quantify, and **predict** outcomes based on observed variables.
- Two fundamental types of predictive tasks:

Why Predictive Modeling (2)?

- In data science and applied statistics, we often aim to understand, quantify, and **predict** outcomes based on observed variables.
- Two fundamental types of predictive tasks:
 - **Regression**: Predict a **continuous** outcome (e.g., income, temperature, recovery time)

Why Predictive Modeling (2)?

- In data science and applied statistics, we often aim to understand, quantify, and **predict** outcomes based on observed variables.
- Two fundamental types of predictive tasks:
 - **Regression**: Predict a **continuous** outcome (e.g., income, temperature, recovery time)
 - **Classification**: Predict a **categorical** outcome (e.g., disease status, pass/fail)

Why Predictive Modeling (2)?

- In data science and applied statistics, we often aim to understand, quantify, and **predict** outcomes based on observed variables.
- Two fundamental types of predictive tasks:
 - **Regression**: Predict a **continuous** outcome (e.g., income, temperature, recovery time)
 - **Classification**: Predict a **categorical** outcome (e.g., disease status, pass/fail)
- These models allow us to:

Why Predictive Modeling (2)?

- In data science and applied statistics, we often aim to understand, quantify, and **predict** outcomes based on observed variables.
- Two fundamental types of predictive tasks:
 - **Regression**: Predict a **continuous** outcome (e.g., income, temperature, recovery time)
 - **Classification**: Predict a **categorical** outcome (e.g., disease status, pass/fail)
- These models allow us to:
 - Forecast outcomes

Why Predictive Modeling (2)?

- In data science and applied statistics, we often aim to understand, quantify, and **predict** outcomes based on observed variables.
- Two fundamental types of predictive tasks:
 - **Regression**: Predict a **continuous** outcome (e.g., income, temperature, recovery time)
 - **Classification**: Predict a **categorical** outcome (e.g., disease status, pass/fail)
- These models allow us to:
 - Forecast outcomes
 - Evaluate importance of features

Why Predictive Modeling (2)?

- In data science and applied statistics, we often aim to understand, quantify, and **predict** outcomes based on observed variables.
- Two fundamental types of predictive tasks:
 - **Regression**: Predict a **continuous** outcome (e.g., income, temperature, recovery time)
 - **Classification**: Predict a **categorical** outcome (e.g., disease status, pass/fail)
- These models allow us to:
 - Forecast outcomes
 - Evaluate importance of features
 - Make data-driven decisions

Why Predictive Modeling (2)?

- In data science and applied statistics, we often aim to understand, quantify, and **predict** outcomes based on observed variables.
- Two fundamental types of predictive tasks:
 - **Regression**: Predict a **continuous** outcome (e.g., income, temperature, recovery time)
 - **Classification**: Predict a **categorical** outcome (e.g., disease status, pass/fail)
- These models allow us to:
 - Forecast outcomes
 - Evaluate importance of features
 - Make data-driven decisions
- Commonly applied in healthcare, business, environmental science, social sciences, and more.

Common Terminology in Predictive Modeling

- Target (label, output, response):
 - The variable we aim to predict.
 - Also called: **dependent variable** (in statistics), **output variable**, **ground truth**.
 - Can be:
 - **Continuous** → regression (e.g., price, temperature)
 - **Categorical** → classification (e.g., spam vs. not, disease yes/no)

Common Terminology in Predictive Modeling

- Target (label, output, response):
 - The variable we aim to predict.
 - Also called: **dependent variable** (in statistics), **output variable**, **ground truth**.
 - Can be:
 - **Continuous** → regression (e.g., price, temperature)
 - **Categorical** → classification (e.g., spam vs. not, disease yes/no)
- Features (inputs, attributes, predictors):
 - The variables used to predict the target.
 - Also called: **independent variables**, **explanatory variables**, **covariates** (in statistics).
 - Represent measurable characteristics or properties of the data.

Common Terminology in Predictive Modeling

- **Target (label, output, response):**
 - The variable we aim to predict.
 - Also called: **dependent variable** (in statistics), **output variable**, **ground truth**.
 - Can be:
 - **Continuous** → regression (e.g., price, temperature)
 - **Categorical** → classification (e.g., spam vs. not, disease yes/no)
- **Features (inputs, attributes, predictors):**
 - The variables used to predict the target.
 - Also called: **independent variables**, **explanatory variables**, **covariates** (in statistics).
 - Represent measurable characteristics or properties of the data.
- **Example - Predicting Diabetes Diagnosis (Classification):**
 - **Target:** Diabetic (Yes = 1, No = 0)
 - **Features:** Age, BMI, glucose level, blood pressure

Common Terminology in Predictive Modeling

- **Target (label, output, response):**
 - The variable we aim to predict.
 - Also called: **dependent variable** (in statistics), **output variable**, **ground truth**.
 - Can be:
 - **Continuous** → regression (e.g., price, temperature)
 - **Categorical** → classification (e.g., spam vs. not, disease yes/no)
- **Features (inputs, attributes, predictors):**
 - The variables used to predict the target.
 - Also called: **independent variables**, **explanatory variables**, **covariates** (in statistics).
 - Represent measurable characteristics or properties of the data.
- **Example - Predicting Diabetes Diagnosis (Classification):**
 - **Target:** Diabetic (Yes = 1, No = 0)
 - **Features:** Age, BMI, glucose level, blood pressure
- **Notation:**

Common Terminology in Predictive Modeling

- **Target (label, output, response):**
 - The variable we aim to predict.
 - Also called: **dependent variable** (in statistics), **output variable**, **ground truth**.
 - Can be:
 - **Continuous** → regression (e.g., price, temperature)
 - **Categorical** → classification (e.g., spam vs. not, disease yes/no)
- **Features (inputs, attributes, predictors):**
 - The variables used to predict the target.
 - Also called: **independent variables**, **explanatory variables**, **covariates** (in statistics).
 - Represent measurable characteristics or properties of the data.
- **Example - Predicting Diabetes Diagnosis (Classification):**
 - **Target:** Diabetic (Yes = 1, No = 0)
 - **Features:** Age, BMI, glucose level, blood pressure
- **Notation:**
 - **X:** Feature matrix and **Y:** Target vector

Regression vs. Classification

- **Regression:**
 - Predicts a **continuous numeric** response variable.
 - **Example:** Predicting a person's blood pressure based on age and weight.

Regression vs. Classification

- **Regression:**
 - Predicts a **continuous numeric** response variable.
 - **Example:** Predicting a person's blood pressure based on age and weight.
- **Classification:**
 - Predicts a **categorical or binary** class label.
 - **Example:** Predicting whether a tumor is benign or malignant based on imaging features.

Regression vs. Classification

- **Regression:**

- Predicts a **continuous numeric** response variable.
- **Example:** Predicting a person's blood pressure based on age and weight.

- **Classification:**

- Predicts a **categorical or binary** class label.
- **Example:** Predicting whether a tumor is benign or malignant based on imaging features.

- **Shared foundation:**

- Both use observed variables (predictors) to model outcomes.
- Both aim to learn a function $f(\mathbf{X})$ to map inputs to outputs.

General Formulation of Predictive Modeling

- We aim to learn the relationship between predictors $\mathbf{X}$ and a response Y :

$$Y = f(\mathbf{X}) + \varepsilon$$

General Formulation of Predictive Modeling

- We aim to learn the relationship between predictors $\mathbf{X}$ and a response Y :

$$Y = f(\mathbf{X}) + \varepsilon$$

- For **regression**:
 - $Y \in \mathbb{R}$: continuous output
 - E.g., house price, blood pressure

General Formulation of Predictive Modeling

- We aim to learn the relationship between predictors $\mathbf{X}$ and a response Y :

$$Y = f(\mathbf{X}) + \varepsilon$$

- For **regression**:
 - $Y \in \mathbb{R}$: continuous output
 - E.g., house price, blood pressure
- For **classification**:
 - $Y \in \{0, 1\}$ or more general classes
 - E.g., fraud detection, disease status

General Formulation of Predictive Modeling

- We aim to learn the relationship between predictors $\mathbf{X}$ and a response Y :

$$Y = f(\mathbf{X}) + \varepsilon$$

- For **regression**:
 - $Y \in \mathbb{R}$: continuous output
 - E.g., house price, blood pressure
- For **classification**:
 - $Y \in \{0, 1\}$ or more general classes
 - E.g., fraud detection, disease status
- $f(\mathbf{X})$ may be:
 - Linear (e.g., linear regression, logistic regression)
 - Non-linear (e.g., trees, neural networks)

General Formulation of Predictive Modeling

- We aim to learn the relationship between predictors $\mathbf{X}$ and a response Y :

$$Y = f(\mathbf{X}) + \varepsilon$$

- For **regression**:
 - $Y \in \mathbb{R}$: continuous output
 - E.g., house price, blood pressure
- For **classification**:
 - $Y \in \{0, 1\}$ or more general classes
 - E.g., fraud detection, disease status
- $f(\mathbf{X})$ may be:
 - Linear (e.g., linear regression, logistic regression)
 - Non-linear (e.g., trees, neural networks)
- Error ε is the one that kind of make things and models accounting for uncertainty.

Example 1: Regression Task — Housing Prices

- **Goal:** Predict house prices using features of the property.

Example 1: Regression Task — Housing Prices

- **Goal:** Predict house prices using features of the property.
- **Target (Y):** Sale price (continuous)

Example 1: Regression Task — Housing Prices

- **Goal:** Predict house prices using features of the property.
- **Target (Y):** Sale price (continuous)
- **Predictors (X):**
 - Square footage
 - Number of bedrooms
 - Location
 - House age

Example 1: Regression Task — Housing Prices

- **Goal:** Predict house prices using features of the property.
- **Target (Y):** Sale price (continuous)
- **Predictors (X):**
 - Square footage
 - Number of bedrooms
 - Location
 - House age
- **Key Questions:**
 - How does location affect price?
 - What is the price increase per added bedroom?

Example 2: Classification Task — Medical Study

- **Goal:** Predict patient outcome (e.g., disease presence).

Example 2: Classification Task — Medical Study

- **Goal:** Predict patient outcome (e.g., disease presence).
- **Target (Y):** Treatment success (Yes/No)

Example 2: Classification Task — Medical Study

- **Goal:** Predict patient outcome (e.g., disease presence).
- **Target (Y):** Treatment success (Yes/No)
- **Predictors (X):**
 - Age
 - Sex
 - Treatment type
 - Comorbidities

Example 2: Classification Task — Medical Study

- **Goal:** Predict patient outcome (e.g., disease presence).
- **Target (Y):** Treatment success (Yes/No)
- **Predictors (X):**
 - Age
 - Sex
 - Treatment type
 - Comorbidities
- **Key Questions:**

Example 2: Classification Task — Medical Study

- **Goal:** Predict patient outcome (e.g., disease presence).
- **Target (Y):** Treatment success (Yes/No)
- **Predictors (X):**
 - Age
 - Sex
 - Treatment type
 - Comorbidities
- **Key Questions:**
 - Which features are most predictive of outcome?

Example 2: Classification Task — Medical Study

- **Goal:** Predict patient outcome (e.g., disease presence).
- **Target (Y):** Treatment success (Yes/No)
- **Predictors (X):**
 - Age
 - Sex
 - Treatment type
 - Comorbidities
- **Key Questions:**
 - Which features are most predictive of outcome?
 - Can we flag high-risk patients before treatment?

Agenda

1. Predictive Modeling

1.1 Motivation

1.2 Regression

1.3 Classification

1.4 Gradient Descent Algorithm

1.5 Gradient Descent Algorithm From Scratch for Regression and Classification Task

1.6 Model Validation (Choosing the Best Model)

Linear Regression (1)

- **Linear Regression** is a statistical method for modeling the relationship between a dependent variable and one or more independent variables.

Linear Regression (1)

- **Linear Regression** is a statistical method for modeling the relationship between a dependent variable and one or more independent variables.
- The relationship is modeled using a linear predictor function whose unknown parameters are estimated from the data.

Linear Regression (1)

- **Linear Regression** is a statistical method for modeling the relationship between a dependent variable and one or more independent variables.
- The relationship is modeled using a linear predictor function whose unknown parameters are estimated from the data.
- In its simplest form, with one dependent variable y and one independent variable x , the linear regression model, so-called **Simple Linear Regression (SLR)** is:

$$y = w_0 + w_1x + \varepsilon = \mathbf{w}^\top \mathbf{x} + \varepsilon, \quad \mathbf{x} = (1, x)^\top, \quad \mathbf{w} = (w_0, w_1)^\top,$$

where:

Linear Regression (1)

- **Linear Regression** is a statistical method for modeling the relationship between a dependent variable and one or more independent variables.
- The relationship is modeled using a linear predictor function whose unknown parameters are estimated from the data.
- In its simplest form, with one dependent variable y and one independent variable x , the linear regression model, so-called **Simple Linear Regression (SLR)** is:

$$y = w_0 + w_1x + \varepsilon = \mathbf{w}^\top \mathbf{x} + \varepsilon, \quad \mathbf{x} = (1, x)^\top, \quad \mathbf{w} = (w_0, w_1)^\top,$$

where:

- y is the output variable.

Linear Regression (1)

- **Linear Regression** is a statistical method for modeling the relationship between a dependent variable and one or more independent variables.
- The relationship is modeled using a linear predictor function whose unknown parameters are estimated from the data.
- In its simplest form, with one dependent variable y and one independent variable x , the linear regression model, so-called **Simple Linear Regression (SLR)** is:

$$y = w_0 + w_1x + \varepsilon = \mathbf{w}^\top \mathbf{x} + \varepsilon, \quad \mathbf{x} = (1, x)^\top, \quad \mathbf{w} = (w_0, w_1)^\top,$$

where:

- y is the output variable.
- x is the input variable.

Linear Regression (1)

- **Linear Regression** is a statistical method for modeling the relationship between a dependent variable and one or more independent variables.
- The relationship is modeled using a linear predictor function whose unknown parameters are estimated from the data.
- In its simplest form, with one dependent variable y and one independent variable x , the linear regression model, so-called **Simple Linear Regression (SLR)** is:

$$y = w_0 + w_1x + \varepsilon = \mathbf{w}^\top \mathbf{x} + \varepsilon, \quad \mathbf{x} = (1, x)^\top, \quad \mathbf{w} = (w_0, w_1)^\top,$$

where:

- y is the output variable.
- x is the input variable.
- w_0 is the intercept term, which represents the expected value of y when $x = 0$.

Linear Regression (1)

- **Linear Regression** is a statistical method for modeling the relationship between a dependent variable and one or more independent variables.
- The relationship is modeled using a linear predictor function whose unknown parameters are estimated from the data.
- In its simplest form, with one dependent variable y and one independent variable x , the linear regression model, so-called **Simple Linear Regression (SLR)** is:

$$y = w_0 + w_1x + \varepsilon = \mathbf{w}^\top \mathbf{x} + \varepsilon, \quad \mathbf{x} = (1, x)^\top, \quad \mathbf{w} = (w_0, w_1)^\top,$$

where:

- y is the output variable.
- x is the input variable.
- w_0 is the intercept term, which represents the expected value of y when $x = 0$.
- w_1 is the slope term, which represents the change in y for a one-unit change in x .

Linear Regression (1)

- **Linear Regression** is a statistical method for modeling the relationship between a dependent variable and one or more independent variables.
- The relationship is modeled using a linear predictor function whose unknown parameters are estimated from the data.
- In its simplest form, with one dependent variable y and one independent variable x , the linear regression model, so-called **Simple Linear Regression (SLR)** is:

$$y = w_0 + w_1x + \varepsilon = \mathbf{w}^\top \mathbf{x} + \varepsilon, \quad \mathbf{x} = (1, x)^\top, \quad \mathbf{w} = (w_0, w_1)^\top,$$

where:

- y is the output variable.
- x is the input variable.
- w_0 is the intercept term, which represents the expected value of y when $x = 0$.
- w_1 is the slope term, which represents the change in y for a one-unit change in x .
- ε is the error term, which accounts for the variability in y that

Linear Regression (2)

- In a classical regression setting, the parameters w_0 and w_1 are estimated from the data using methods such as **Ordinary Least Squares (OLS)**, which minimizes the sum of the squared differences between the observed values and the predicted values.

Linear Regression (2)

- In a classical regression setting, the parameters w_0 and w_1 are estimated from the data using methods such as **Ordinary Least Squares (OLS)**, which minimizes the sum of the squared differences between the observed values and the predicted values.
- The **OLS** estimate of the parameters can be obtained by solving the following normal equations:

$$\begin{pmatrix} \hat{w}_0 \\ \hat{w}_1 \end{pmatrix} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

where:

Linear Regression (2)

- In a classical regression setting, the parameters w_0 and w_1 are estimated from the data using methods such as **Ordinary Least Squares (OLS)**, which minimizes the sum of the squared differences between the observed values and the predicted values.
- The **OLS** estimate of the parameters can be obtained by solving the following normal equations:

$$\begin{pmatrix} \hat{w}_0 \\ \hat{w}_1 \end{pmatrix} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

where:

- $\mathbf{X}$ is the design matrix, including a column of ones for the intercept and a column of the predictor variable values.

Linear Regression (2)

- In a classical regression setting, the parameters w_0 and w_1 are estimated from the data using methods such as **Ordinary Least Squares (OLS)**, which minimizes the sum of the squared differences between the observed values and the predicted values.
- The **OLS** estimate of the parameters can be obtained by solving the following normal equations:

$$\begin{pmatrix} \hat{w}_0 \\ \hat{w}_1 \end{pmatrix} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

where:

- $\mathbf{X}$ is the design matrix, including a column of ones for the intercept and a column of the predictor variable values.
- $\mathbf{y}$ is the vector of observed values of the dependent variable.

Linear Regression (2)

- In a classical regression setting, the parameters w_0 and w_1 are estimated from the data using methods such as **Ordinary Least Squares (OLS)**, which minimizes the sum of the squared differences between the observed values and the predicted values.
- The **OLS** estimate of the parameters can be obtained by solving the following normal equations:

$$\begin{pmatrix} \hat{w}_0 \\ \hat{w}_1 \end{pmatrix} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

where:

- $\mathbf{X}$ is the design matrix, including a column of ones for the intercept and a column of the predictor variable values.
- $\mathbf{y}$ is the vector of observed values of the dependent variable.
- SLR can be extended to include multiple independent variables, resulting in **Multiple Linear Regression (MLR)**:

$$y = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_p x_p + \epsilon.$$

where $x_1, x_2, \dots, x_p$ are the predictor variables.

Interpretation of Regression Model Parameters

- **Estimated (Predicted) Regression Line:**

$$\hat{y} = \hat{w}_0 + \hat{w}_1 x$$

Interpretation of Regression Model Parameters

- **Estimated (Predicted) Regression Line:**

$$\hat{y} = \hat{w}_0 + \hat{w}_1 x$$

- **Interpretation of $\hat{w}_1$ (Slope):**
 - Represents the estimated change in y for a one-unit increase in X .
 - **Example:** If $\hat{w}_1 = 2$, then for each additional unit of X , Y is expected to increase by 2 units.

Interpretation of Regression Model Parameters

- **Estimated (Predicted) Regression Line:**

$$\hat{y} = \hat{w}_0 + \hat{w}_1 x$$

- **Interpretation of $\hat{w}_1$ (Slope):**

- Represents the estimated change in y for a one-unit increase in X .
- **Example:** If $\hat{w}_1 = 2$, then for each additional unit of X , Y is expected to increase by 2 units.

- **Interpretation of $\hat{w}_0$ (Intercept):**

- Represents the estimated value of Y when $X = 0$.
- **Note:** The intercept may not always have a meaningful interpretation, especially if $X = 0$ is outside the range of observed data.

Agenda

1. Predictive Modeling

1.1 Motivation

1.2 Regression

1.3 Classification

1.4 Gradient Descent Algorithm

1.5 Gradient Descent Algorithm From Scratch for Regression and Classification Task

1.6 Model Validation (Choosing the Best Model)

What is Classification?

- **Classification** is a supervised learning task where the goal is to predict a **categorical target variable** (label).

What is Classification?

- **Classification** is a supervised learning task where the goal is to predict a **categorical target variable** (label).
- The output is a class label rather than a continuous number.

What is Classification?

- **Classification** is a supervised learning task where the goal is to predict a **categorical target variable** (label).
- The output is a class label rather than a continuous number.
- **Binary classification**: Two classes (e.g., spam vs. not spam, disease vs. no disease)

What is Classification?

- **Classification** is a supervised learning task where the goal is to predict a **categorical target variable** (label).
- The output is a class label rather than a continuous number.
- **Binary classification**: Two classes (e.g., spam vs. not spam, disease vs. no disease)
- **Multiclass classification**: More than two classes (e.g., handwritten digit recognition: 0 to 9)

What is Classification?

- **Classification** is a supervised learning task where the goal is to predict a **categorical target variable** (label).
- The output is a class label rather than a continuous number.
- **Binary classification**: Two classes (e.g., spam vs. not spam, disease vs. no disease)
- **Multiclass classification**: More than two classes (e.g., handwritten digit recognition: 0 to 9)
- Given input features $\mathbf{x}$, the goal is to learn a function that predicts class membership:

$$f(\mathbf{x}) \in \{0, 1\} \quad (\text{or more generally, } \{1, 2, \dots, K\})$$

What is Classification?

- **Classification** is a supervised learning task where the goal is to predict a **categorical target variable** (label).
- The output is a class label rather than a continuous number.
- **Binary classification**: Two classes (e.g., spam vs. not spam, disease vs. no disease)
- **Multiclass classification**: More than two classes (e.g., handwritten digit recognition: 0 to 9)
- Given input features $\mathbf{x}$, the goal is to learn a function that predicts class membership:

$$f(\mathbf{x}) \in \{0, 1\} \quad (\text{or more generally, } \{1, 2, \dots, K\})$$

- **Example**: Predicting if a patient has diabetes based on age, BMI, and glucose levels.

Logistic Regression (1)

- **Logistic Regression** is a classical statistical model used for **binary classification**.

Logistic Regression (1)

- **Logistic Regression** is a classical statistical model used for **binary classification**.
- The model estimates the **probability** that a given input belongs to the positive class (e.g., $Y = 1$).

Logistic Regression (1)

- **Logistic Regression** is a classical statistical model used for **binary classification**.
- The model estimates the **probability** that a given input belongs to the positive class (e.g., $Y = 1$).
- The relationship between the predictors $\mathbf{x}$ and the probability $P(Y = 1 \mid \mathbf{x})$ is modeled as:

$$P(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

Logistic Regression (1)

- **Logistic Regression** is a classical statistical model used for **binary classification**.
- The model estimates the **probability** that a given input belongs to the positive class (e.g., $Y = 1$).
- The relationship between the predictors $\mathbf{x}$ and the probability $P(Y = 1 \mid \mathbf{x})$ is modeled as:

$$P(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

- Here, $\sigma(\cdot)$ is the logistic (sigmoid) function.

Logistic Regression (1)

- **Logistic Regression** is a classical statistical model used for **binary classification**.
- The model estimates the **probability** that a given input belongs to the positive class (e.g., $Y = 1$).
- The relationship between the predictors $\mathbf{x}$ and the probability $P(Y = 1 \mid \mathbf{x})$ is modeled as:

$$P(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

- Here, $\sigma(\cdot)$ is the logistic (sigmoid) function.
- Outputs are interpreted as class probabilities, and a threshold (e.g., 0.5) is used to make class predictions.

No Closed-Form Solution for Logistic Regression

- Unlike linear regression, logistic regression does **not** have a closed-form solution for estimating parameters $\mathbf{w}$.

No Closed-Form Solution for Logistic Regression

- Unlike linear regression, logistic regression does **not** have a closed-form solution for estimating parameters $\mathbf{w}$.
- Instead, we use an **optimization approach** to maximize the likelihood of the observed data.

No Closed-Form Solution for Logistic Regression

- Unlike linear regression, logistic regression does **not** have a closed-form solution for estimating parameters $\mathbf{w}$.
- Instead, we use an **optimization approach** to maximize the likelihood of the observed data.
- This involves minimizing the **log-loss** (or cross-entropy loss), given by:

$$\mathcal{L}(\mathbf{w}) = - \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

where $\hat{p}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$.

No Closed-Form Solution for Logistic Regression

- Unlike linear regression, logistic regression does **not** have a closed-form solution for estimating parameters $\mathbf{w}$.
- Instead, we use an **optimization approach** to maximize the likelihood of the observed data.
- This involves minimizing the **log-loss** (or cross-entropy loss), given by:

$$\mathcal{L}(\mathbf{w}) = - \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

where $\hat{p}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$.

- Since this is a convex optimization problem, we use **iterative algorithms** such as **gradient descent** to find the optimal $\mathbf{w}$.

Agenda

1. Predictive Modeling

1.1 Motivation

1.2 Regression

1.3 Classification

1.4 Gradient Descent Algorithm

1.5 Gradient Descent Algorithm From Scratch for Regression and Classification Task

1.6 Model Validation (Choosing the Best Model)

Gradient Descent Algorithm (1)

Gradient Descent Algorithm is an optimization technique used to minimize a loss function by iteratively moving towards the minimum of the function.

The main components of the gradient descent algorithm are:

- **Loss Function:**

Gradient Descent Algorithm (1)

Gradient Descent Algorithm is an optimization technique used to minimize a loss function by iteratively moving towards the minimum of the function.

The main components of the gradient descent algorithm are:

- **Loss Function:**
 - The loss function measures the error or difference between the predicted values and the actual values. That means it measures how well the model's predictions match the actual data.

Gradient Descent Algorithm (1)

Gradient Descent Algorithm is an optimization technique used to minimize a loss function by iteratively moving towards the minimum of the function.

The main components of the gradient descent algorithm are:

- **Loss Function:**

- The loss function measures the error or difference between the predicted values and the actual values. That means it measures how well the model's predictions match the actual data.
- The goal of gradient descent is to minimize this loss function to improve the accuracy of the model.

Gradient Descent Algorithm (1)

Gradient Descent Algorithm is an optimization technique used to minimize a loss function by iteratively moving towards the minimum of the function.

The main components of the gradient descent algorithm are:

- **Loss Function:**

- The loss function measures the error or difference between the predicted values and the actual values. That means it measures how well the model's predictions match the actual data.
- The goal of gradient descent is to minimize this loss function to improve the accuracy of the model.
- Examples of loss functions include **Mean Squared Error (MSE)** for regression problems and **Cross-Entropy Loss** for classification problems.

Gradient Descent Algorithm (1)

Gradient Descent Algorithm is an optimization technique used to minimize a loss function by iteratively moving towards the minimum of the function.

The main components of the gradient descent algorithm are:

- **Loss Function:**

- The loss function measures the error or difference between the predicted values and the actual values. That means it measures how well the model's predictions match the actual data.
- The goal of gradient descent is to minimize this loss function to improve the accuracy of the model.
- Examples of loss functions include **Mean Squared Error (MSE)** for regression problems and **Cross-Entropy Loss** for classification problems.
- Mathematically, for a set of predictions $\hat{y}$ and actual values y , the Mean Squared Error is given by:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Gradient Descent Algorithm (2)

- **Gradient:**

- The gradient is a vector of partial derivatives of the loss function with respect to all the parameters of the model.
- It points in the direction of the steepest ascent of the function.
- In gradient descent, we move in the opposite direction of the gradient to find the minimum of the loss function.
- For a parameter $\mathbf{w} = (w_0, w_1, \dots, w_p)^\top$, the gradient of the loss function $\mathcal{L}$ with respect to $\mathbf{w}$ is given by:

$$\nabla_{\mathbf{w}} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial w_0}, \frac{\partial \mathcal{L}}{\partial w_1}, \dots, \frac{\partial \mathcal{L}}{\partial w_p} \right).$$

Gradient Descent Algorithm (2)

- **Gradient:**

- The gradient is a vector of partial derivatives of the loss function with respect to all the parameters of the model.
- It points in the direction of the steepest ascent of the function.
- In gradient descent, we move in the opposite direction of the gradient to find the minimum of the loss function.
- For a parameter $\mathbf{w} = (w_0, w_1, \dots, w_p)^\top$, the gradient of the loss function $\mathcal{L}$ with respect to $\mathbf{w}$ is given by:

$$\nabla_{\mathbf{w}} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial w_0}, \frac{\partial \mathcal{L}}{\partial w_1}, \dots, \frac{\partial \mathcal{L}}{\partial w_p} \right).$$

- **Update Rule:**

Gradient Descent Algorithm (2)

- **Gradient:**

- The gradient is a vector of partial derivatives of the loss function with respect to all the parameters of the model.
- It points in the direction of the steepest ascent of the function.
- In gradient descent, we move in the opposite direction of the gradient to find the minimum of the loss function.
- For a parameter $\mathbf{w} = (w_0, w_1, \dots, w_p)^\top$, the gradient of the loss function $\mathcal{L}$ with respect to $\mathbf{w}$ is given by:

$$\nabla_{\mathbf{w}} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial w_0}, \frac{\partial \mathcal{L}}{\partial w_1}, \dots, \frac{\partial \mathcal{L}}{\partial w_p} \right).$$

- **Update Rule:**

- In each iteration of gradient descent, the model parameters are updated using the gradient and the learning rate η .

Gradient Descent Algorithm (2)

- **Gradient:**

- The gradient is a vector of partial derivatives of the loss function with respect to all the parameters of the model.
- It points in the direction of the steepest ascent of the function.
- In gradient descent, we move in the opposite direction of the gradient to find the minimum of the loss function.
- For a parameter $\mathbf{w} = (w_0, w_1, \dots, w_p)^\top$, the gradient of the loss function $\mathcal{L}$ with respect to $\mathbf{w}$ is given by:

$$\nabla_{\mathbf{w}} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial w_0}, \frac{\partial \mathcal{L}}{\partial w_1}, \dots, \frac{\partial \mathcal{L}}{\partial w_p} \right).$$

- **Update Rule:**

- In each iteration of gradient descent, the model parameters are updated using the gradient and the learning rate η .
- The update rule for a parameter $\mathbf{w}$ is given by:

$$\mathbf{w} := \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L},$$

where η is the learning rate and $\nabla_{\mathbf{w}} \mathcal{L}$ is the gradient of the loss function with respect to $\mathbf{w}$.

Gradient Descent Algorithm (3)

- **Learning Rate η :**

- The learning rate is a hyperparameter that determines the size of the steps taken towards the minimum of the loss function.
- A smaller learning rate means smaller steps, leading to more accurate convergence but slower progress.
- A larger learning rate can speed up the process but risks overshooting the minimum and potentially diverging.
- It is important to choose an appropriate learning rate to ensure efficient and effective training of the model.

Gradient Descent Algorithm (3)

- **Learning Rate η :**

- The learning rate is a hyperparameter that determines the size of the steps taken towards the minimum of the loss function.
- A smaller learning rate means smaller steps, leading to more accurate convergence but slower progress.
- A larger learning rate can speed up the process but risks overshooting the minimum and potentially diverging.
- It is important to choose an appropriate learning rate to ensure efficient and effective training of the model.

- **Convergence:**

- The algorithm iteratively updates the parameters until the loss function converges to a minimum value.
- Convergence can be determined by setting a threshold for the change in the loss function or by specifying a maximum number of iterations.

Gradient Descent Algorithm: Algorithm Steps

- **Initialize:** Start with an initial guess for the parameters.

Gradient Descent Algorithm: Algorithm Steps

- **Initialize:** Start with an initial guess for the parameters.
- **Compute Gradient:** Calculate the gradient of the loss function with respect to each parameter.

Gradient Descent Algorithm: Algorithm Steps

- **Initialize:** Start with an initial guess for the parameters.
- **Compute Gradient:** Calculate the gradient of the loss function with respect to each parameter.
- **Update Parameters:** Adjust the parameters by moving them in the direction opposite to the gradient, scaled by the learning rate.

Gradient Descent Algorithm: Algorithm Steps

- **Initialize:** Start with an initial guess for the parameters.
- **Compute Gradient:** Calculate the gradient of the loss function with respect to each parameter.
- **Update Parameters:** Adjust the parameters by moving them in the direction opposite to the gradient, scaled by the learning rate.
- **Iterate:** Repeat the process until the parameters converge to the minimum or a stopping criterion is met.

Agenda

1. Predictive Modeling

1.1 Motivation

1.2 Regression

1.3 Classification

1.4 Gradient Descent Algorithm

1.5 Gradient Descent Algorithm From Scratch for Regression and Classification Task

1.6 Model Validation (Choosing the Best Model)

Recall:

$$X = \begin{bmatrix} 1 & x_{11} & \dots & x_{p1} \\ 1 & x_{12} & & x_{p2} \\ \vdots & \vdots & & \vdots \\ 1 & x_{1n} & & x_{pn} \end{bmatrix}, Y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

↓
output

$\Rightarrow n$:

p :

$$Y = f(x) + \varepsilon$$

lin. Reg: $f(x) = \omega_0 + \omega_1 x_1 + \dots + \omega_p x_p$

log. Reg: $f(x) = \frac{1}{1 + e^{-(\omega_0 + \omega_1 x_1 + \dots + \omega_p x_p)}}$

↓
class

$$\begin{pmatrix} \hat{\omega}_0 \\ \hat{\omega}_1 \\ \vdots \\ \hat{\omega}_p \end{pmatrix} = \underbrace{(X^T X)^{-1} X^T Y}_{\text{}} :$$

$$\hat{f}(x) = \hat{\omega}_0 + \hat{\omega}_1 x_1 + \dots + \hat{\omega}_p x_p$$

$$\hat{y} = \hat{f}(x)$$

$$\Rightarrow \hat{y}_i = \hat{\omega}_0 + \hat{\omega}_1 \underline{x_{1i}} + \dots + \hat{\omega}_p \underline{x_{pi}}$$

$i = 1 \dots n$

→ Reg: closed form exp

$$\hat{\omega} = (\hat{\omega}_0 \dots \hat{\omega}_p)$$

log. Reg: class → Not closed

$\omega \rightarrow$ gradient:

(i) $\hookrightarrow L(\omega)$: Reg: MSE
class: cross.

(ii) $\nabla_{\omega} L(\omega) = \begin{pmatrix} \end{pmatrix}$
 $p+1 \times 1$

$$\omega = \omega - \eta \nabla L(\omega)$$

$\hookrightarrow \omega_0 \rightarrow \omega_1 = \omega_0 - \eta \nabla L(\omega_0)$
 $1000 - 999$
 $\|\omega_N - \omega_{N-1}\| < 1e^{-6}$ $\swarrow$ $\omega_2 = \omega_1 - \eta \nabla L(\omega_1)$
 $\omega_3 = \omega_2 - \eta \nabla L(\omega_2) \rightarrow \omega_N$

Lab 1: Gradient Descent for Logistic Regression

- **Recall:** **Gradient descent** is an iterative method to find parameters that minimize a loss function.

Lab 1: Gradient Descent for Logistic Regression

- **Recall:** **Gradient descent** is an iterative method to find parameters that minimize a loss function.
- **Loss function** (Binary Cross-Entropy / Log-Loss):

$$\mathcal{L}(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)],$$

where $\hat{y}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$ and $\sigma(z) = \frac{1}{1+e^{-z}}$.

Lab 1: Gradient Descent for Logistic Regression

- **Recall:** **Gradient descent** is an iterative method to find parameters that minimize a loss function.
- **Loss function** (Binary Cross-Entropy / Log-Loss):

$$\mathcal{L}(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)],$$

where $\hat{y}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$ and $\sigma(z) = \frac{1}{1+e^{-z}}$. $\hookrightarrow \mathbf{w} = (w_0, \dots, w_p)$

- **Gradient:**

$$\boxed{\nabla \mathcal{L}(\mathbf{w})} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \mathbf{x}_i$$

$\rightarrow$ $\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \mathbf{x}_i$
 $\mathbf{x}_i = (x_{i1}, \dots, x_{ip})$
 $(\hat{y}_i - y_i) \mathbf{x}_i$

Lab 1: Gradient Descent for Logistic Regression

- **Recall:** **Gradient descent** is an iterative method to find parameters that minimize a loss function.
- **Loss function** (Binary Cross-Entropy / Log-Loss):

$$\mathcal{L}(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)],$$

where $\hat{y}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$ and $\sigma(z) = \frac{1}{1+e^{-z}}$.

- **Gradient:**

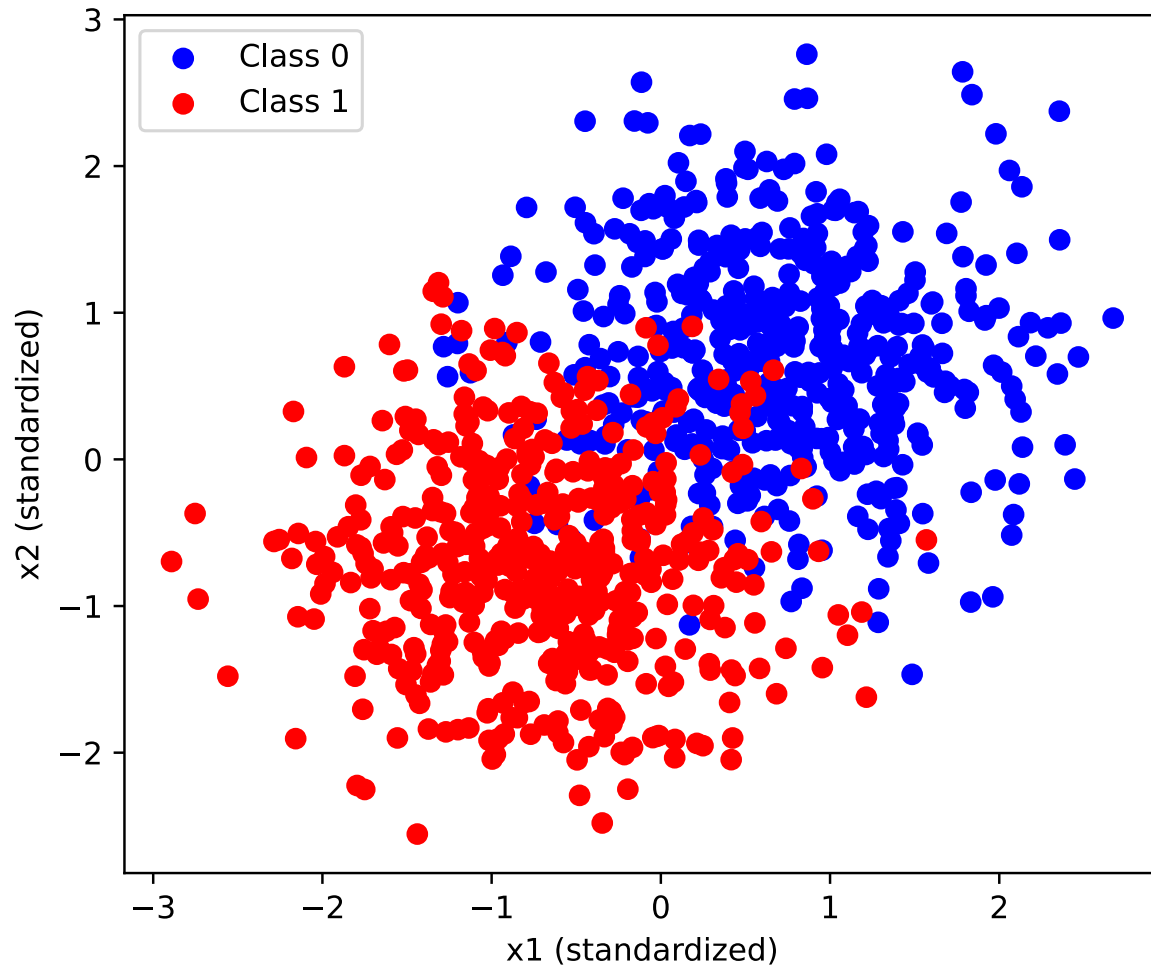
$$\nabla \mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \mathbf{x}_i$$

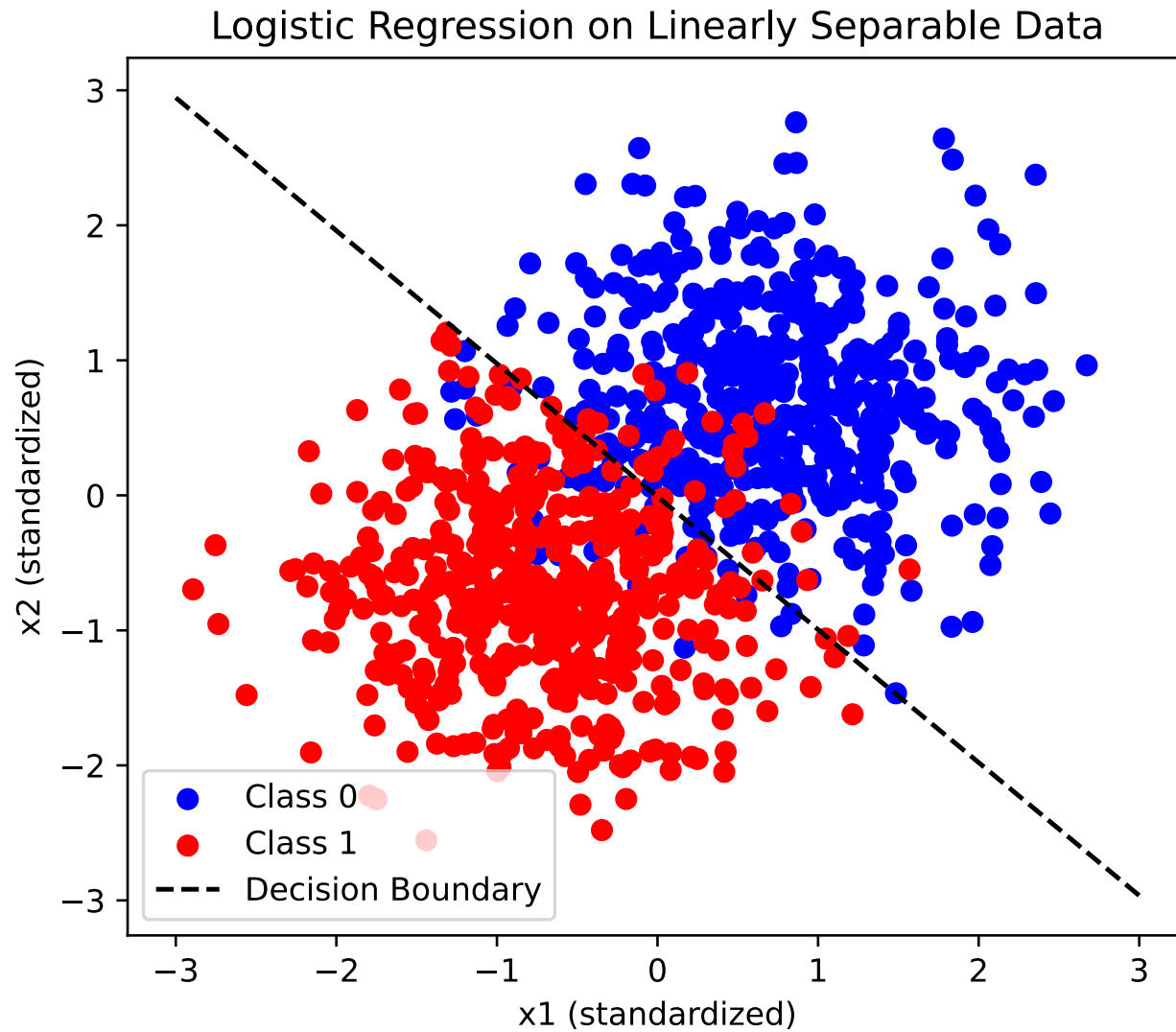
- **Parameter update rule:**

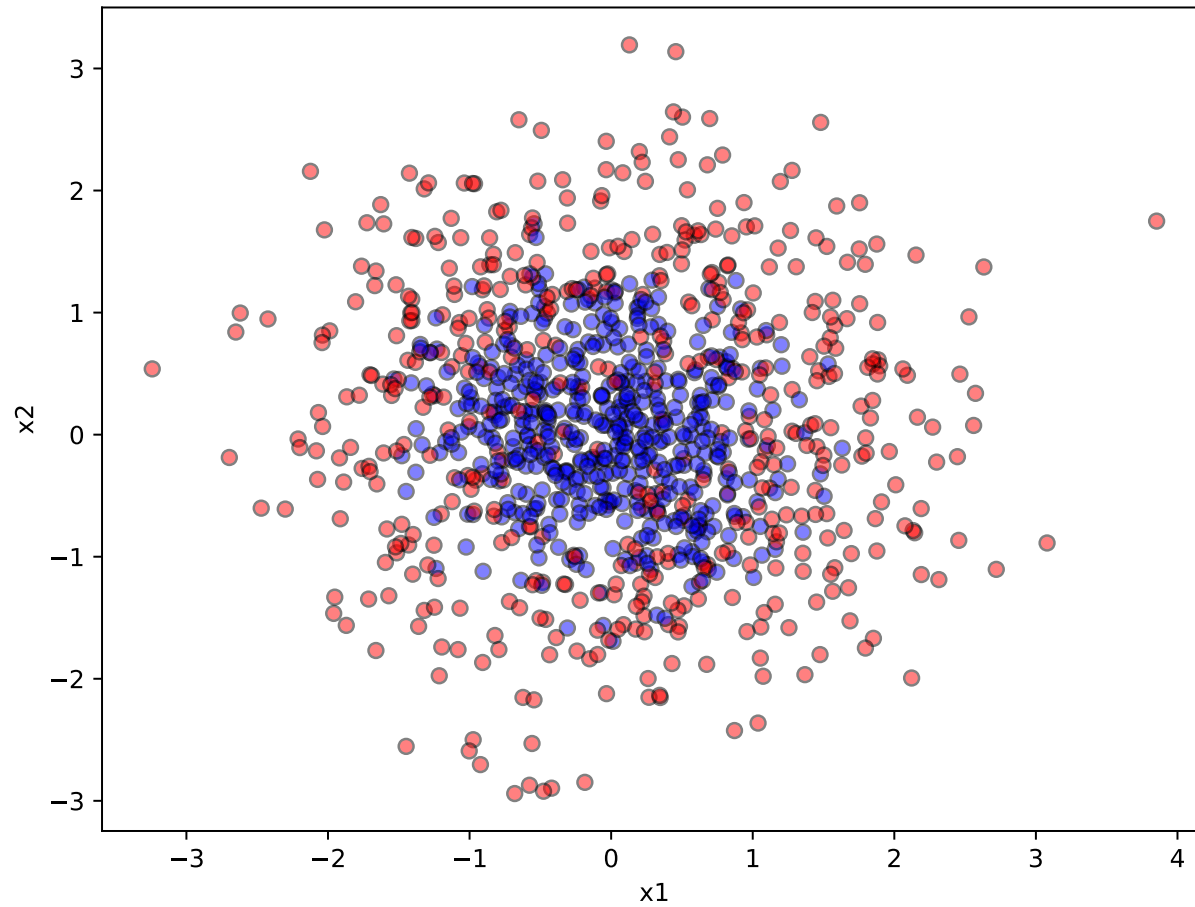
$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \cdot \nabla \mathcal{L}(\mathbf{w}^{(t)})$$

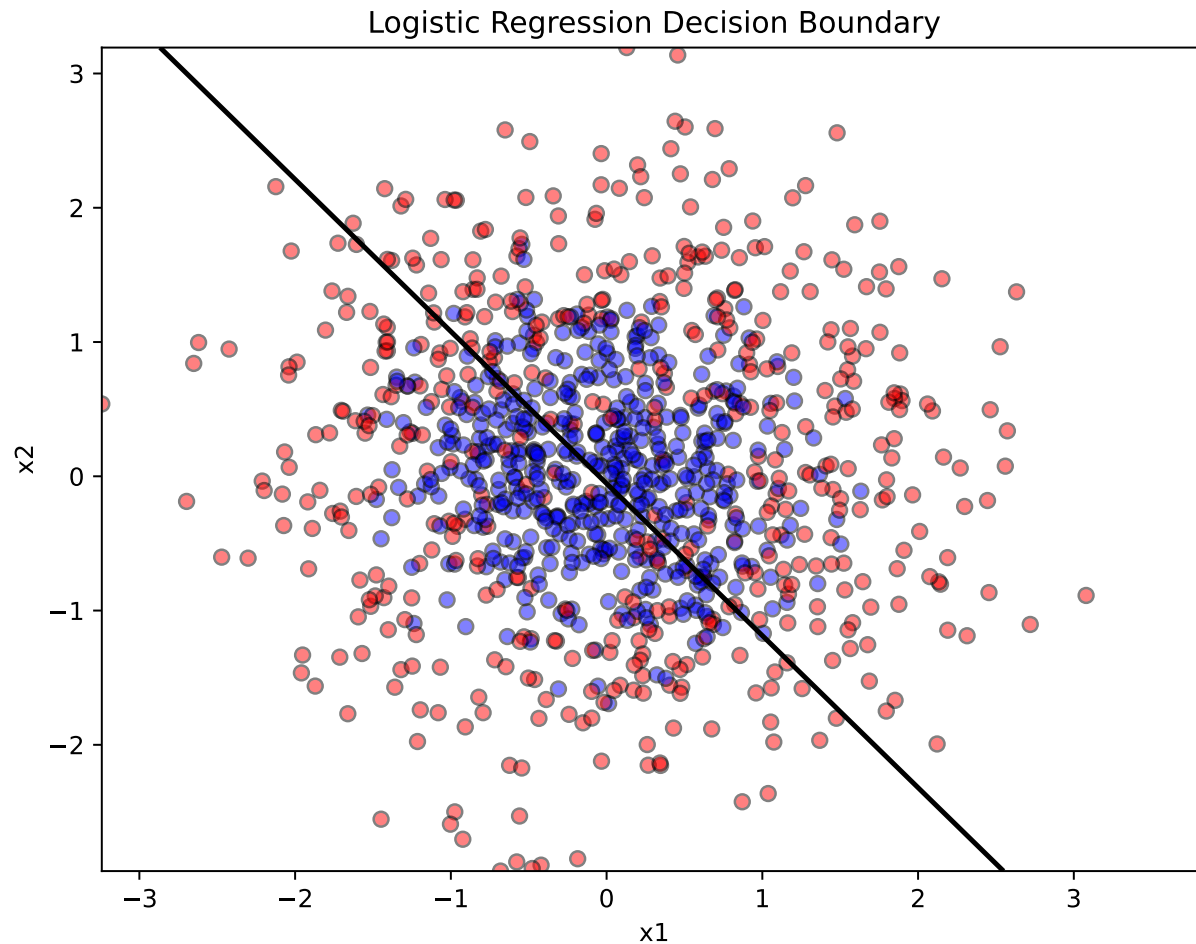
where η is the learning rate.

$\rightarrow t=0 \quad \mathbf{w}^{(0)}$
 $\rightarrow t=0 \dots N$
 $\rightarrow \mathbf{w}^{(N)} = \hat{\mathbf{w}}$









1

¹Open the python code from the GitHub Link:
https://github.com/yadavrishikesh/Masterclass-IITMandi/blob/main/Module3/code/logistic_regression_from_scratch.ipynb

Lab 2: Gradient Descent for Linear Regression

- **Loss function** (Mean Squared Error, MSE):

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \hat{y}_i = w_0 + w_1 x_i$$

Lab 2: Gradient Descent for Linear Regression

- **Loss function** (Mean Squared Error, MSE):

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \hat{y}_i = w_0 + w_1 x_i$$

- **Gradients:**

$$\frac{\partial \mathcal{L}}{\partial w_1} = -\frac{2}{n} \sum_{i=1}^n x_i (y_i - \hat{y}_i), \quad \frac{\partial \mathcal{L}}{\partial w_0} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

Lab 2: Gradient Descent for Linear Regression

- **Loss function** (Mean Squared Error, MSE):

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \hat{y}_i = w_0 + w_1 x_i$$

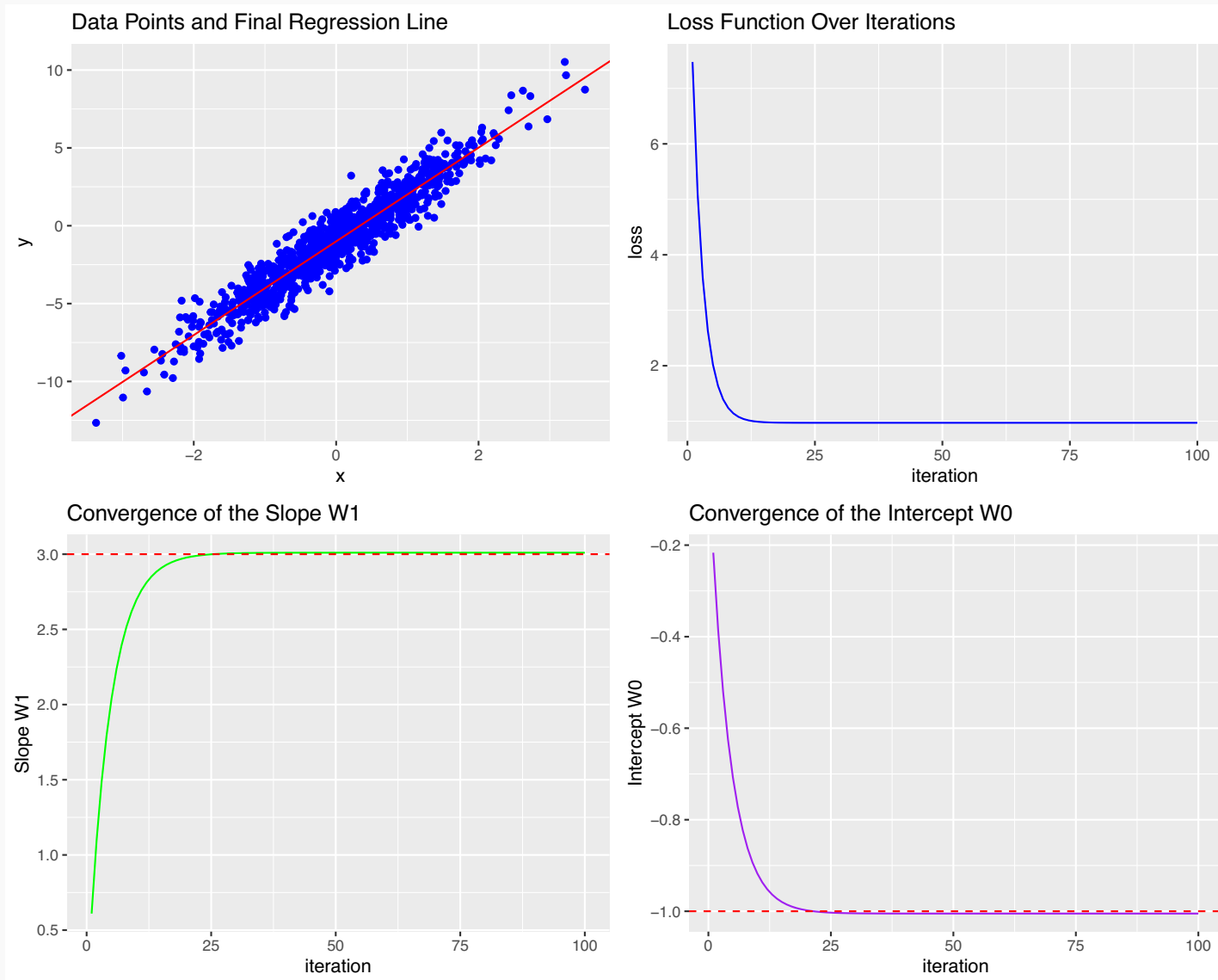
- **Gradients:**

$$\frac{\partial \mathcal{L}}{\partial w_1} = -\frac{2}{n} \sum_{i=1}^n x_i (y_i - \hat{y}_i), \quad \frac{\partial \mathcal{L}}{\partial w_0} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

- **Parameter updates:**

$$w_1 \leftarrow w_1 - \eta \frac{\partial \mathcal{L}}{\partial w_1}, \quad w_0 \leftarrow w_0 - \eta \frac{\partial \mathcal{L}}{\partial w_0}$$

Results Lab 2



2

²Open the python code from the GitHub Link:

https://github.com/yadavrishikesh/Masterclass-IITMandi/blob/main/Module3/code/gradient_descent_LM.ipynb

Exercise: Implement the Gradient Descent for Polynomial Regression

- **Model:** $y = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M + \epsilon$

Exercise: Implement the Gradient Descent for Polynomial Regression

- **Model:** $y = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M + \epsilon$
- **Predicted output:**

$$\hat{y}_i = \sum_{j=0}^M w_j x_i^j$$

Exercise: Implement the Gradient Descent for Polynomial Regression

- **Model:** $y = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M + \epsilon$
- **Predicted output:**

$$\hat{y}_i = \sum_{j=0}^M w_j x_i^j$$

- **Loss function** (Mean Squared Error):

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Exercise: Implement the Gradient Descent for Polynomial Regression

- **Model:** $y = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M + \epsilon$
- **Predicted output:**

$$\hat{y}_i = \sum_{j=0}^M w_j x_i^j$$

- **Loss function** (Mean Squared Error):

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Gradient for each weight** w_j :

$$\frac{\partial \mathcal{L}}{\partial w_j} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) x_i^j$$

Exercise: Implement the Gradient Descent for Polynomial Regression

- **Model:** $y = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M + \epsilon$
- **Predicted output:**

$$\hat{y}_i = \sum_{j=0}^M w_j x_i^j$$

- **Loss function** (Mean Squared Error):

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Gradient for each weight** w_j :

$$\frac{\partial \mathcal{L}}{\partial w_j} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) x_i^j$$

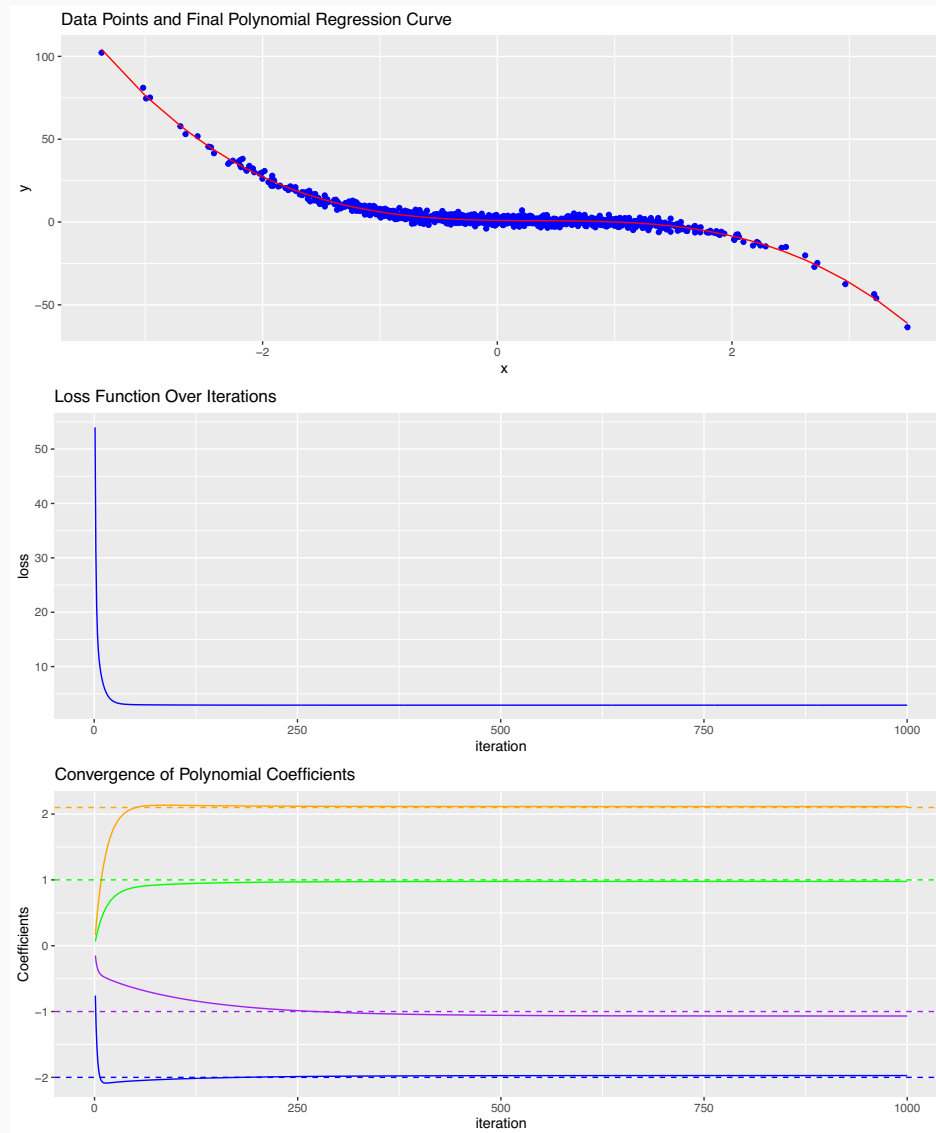
- **Update rule:**

$$w_j \leftarrow w_j - \eta \cdot \frac{\partial \mathcal{L}}{\partial w_j}, \quad \text{for } j = 0, \dots, M$$

Exercise: Implement the Gradient Descent for Polynomial Regression

Look at the exercise from https://github.com/yadavrishikesh/Masterclass-IITMandi/blob/main/Module3/assignments/gradient_descent_assignment.ipynb

Results Exercise



3

³Open the python code from the GitHub Link:

https://github.com/yadavrishikesh/Masterclass-IITMandi/blob/main/Module3/code/gradient_descent_polynomial.ipynb

Regression vs. Classification Summary

Aspect	Regression	Classification
Output type	Continuous	Categorical
Model example	Linear / Polynomial Regression	Logistic Regression
Loss function	$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$	$\mathcal{L}_{\text{log}} = -\frac{1}{n} \sum [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$
Closed-form solution	Yes (e.g., OLS)	No
Estimation method	Matrix inversion or GD	Gradient descent
Goal	Predict a number	Predict a class or probability

Agenda

1. Predictive Modeling

1.1 Motivation

1.2 Regression

1.3 Classification

1.4 Gradient Descent Algorithm

1.5 Gradient Descent Algorithm From Scratch for Regression and Classification Task

1.6 Model Validation (Choosing the Best Model)

Model Validation: Why Do We Need It?

How do we know if our model is actually any good?

- Is a **linear regression model** good enough for a given data, OR

Model Validation: Why Do We Need It?

How do we know if our model is actually any good?

- Is a **linear regression model** good enough for a given data, OR
- Do we need a **polynomial model**? If so, what **degree** is enough?

Model Validation: Why Do We Need It?

How do we know if our model is actually any good?

- Is a **linear regression model** good enough for a given data, OR
- Do we need a **polynomial model**? If so, what **degree** is enough?
- Are any models other than **polynomial** needed to be looked at?

Model Validation: Why Do We Need It?

How do we know if our model is actually any good?

- Is a **linear regression model** good enough for a given data, OR
- Do we need a **polynomial model**? If so, what **degree** is enough?
- Are any models other than **polynomial** needed to be looked at?
- A model that performs well on **training data** may fail on **new data**.

Model Validation: Why Do We Need It?

How do we know if our model is actually any good?

- Is a **linear regression model** good enough for a given data, OR
- Do we need a **polynomial model**? If so, what **degree** is enough?
- Are any models other than **polynomial** needed to be looked at?
- A model that performs well on **training data** may fail on **new data**.
- We need to assess the model's **generalization performance**.

Model Validation: Why Do We Need It?

How do we know if our model is actually any good?

- Is a **linear regression model** good enough for a given data, OR
- Do we need a **polynomial model**? If so, what **degree** is enough?
- Are any models other than **polynomial** needed to be looked at?
- A model that performs well on **training data** may fail on **new data**.
- We need to assess the model's **generalization performance**.
- Model validation helps avoid **overfitting** and supports better **model selection**.

Training, Validation, and Test Splits

- **Training Set** ($\approx 60\text{--}70\%$ of data): Used to **fit** the model - the model “**learns**” from this data.

Training, Validation, and Test Splits

- **Training Set** ($\approx 60\text{--}70\%$ of data): Used to **fit** the model - the model “**learns**” from this data.
- **Validation Set** ($\approx 15\text{--}20\%$):
 - Used to **tune hyperparameters** and monitor performance.
 - Allows techniques like **early stopping** or **model selection**.

Training, Validation, and Test Splits

- **Training Set** ($\approx 60\text{--}70\%$ of data): Used to **fit** the model - the model “**learns**” from this data.
- **Validation Set** ($\approx 15\text{--}20\%$):
 - Used to **tune hyperparameters** and monitor performance.
 - Allows techniques like **early stopping** or **model selection**.
- **Test Set** ($\approx 15\text{--}20\%$):
 - Held out until **final evaluation**.
 - Measures true **generalization** to **unseen data**.

Training, Validation, and Test Splits

- **Training Set** ($\approx 60\text{--}70\%$ of data): Used to **fit** the model - the model “**learns**” from this data.
- **Validation Set** ($\approx 15\text{--}20\%$):
 - Used to **tune hyperparameters** and monitor performance.
 - Allows techniques like **early stopping** or **model selection**.
- **Test Set** ($\approx 15\text{--}20\%$):
 - Held out until **final evaluation**.
 - Measures true **generalization** to **unseen data**.

Training, Validation, and Test Splits

- **Training Set** ($\approx 60\text{--}70\%$ of data): Used to **fit** the model - the model “**learns**” from this data.
- **Validation Set** ($\approx 15\text{--}20\%$):
 - Used to **tune hyperparameters** and monitor performance.
 - Allows techniques like **early stopping** or **model selection**.
- **Test Set** ($\approx 15\text{--}20\%$):
 - Held out until **final evaluation**.
 - Measures true **generalization** to **unseen data**.

Important Note: In most cases, we only divide the whole data into training and testing datasets.

Most Common Data Splitting Strategies

- **Random Split:**
 - Common and simple.
 - Risky if the dataset is small or imbalanced.

Most Common Data Splitting Strategies

- **Random Split:**
 - Common and simple.
 - Risky if the dataset is small or imbalanced.
- **Stratified Split:**
 - Preserves class proportions in classification.

Most Common Data Splitting Strategies

- **Random Split:**
 - Common and simple.
 - Risky if the dataset is small or imbalanced.
- **Stratified Split:**
 - Preserves class proportions in classification.
- **Time-Based Split:**
 - Used for time-series data where order matters.

Cross-Validation (CV)

- Splits data into multiple folds and rotates the validation set.

Cross-Validation (CV)

- Splits data into multiple folds and rotates the validation set.
- Common types:

Cross-Validation (CV)

- Splits data into multiple folds and rotates the validation set.
- Common types:
 - **k-Fold CV**: Divides data into k equal parts

Cross-Validation (CV)

- Splits data into multiple folds and rotates the validation set.
- Common types:
 - **k-Fold CV**: Divides data into k equal parts
 - **Leave-One-Out CV**: Each data point is its own fold

Cross-Validation (CV)

- Splits data into multiple folds and rotates the validation set.
- Common types:
 - **k-Fold CV**: Divides data into k equal parts
 - **Leave-One-Out CV**: Each data point is its own fold
- Helps in more reliable model evaluation when data is limited.

Model Evaluation Metrics: Regression Metrics

- **Mean Squared Error (MSE):**

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where y_i is the observed value, $\hat{y}_i$ is the predicted value, and n is the total number of observations.

Model Evaluation Metrics: Regression Metrics

- **Mean Squared Error (MSE):**

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where y_i is the observed value, $\hat{y}_i$ is the predicted value, and n is the total number of observations.

- **Root Mean Squared Error (RMSE):**

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Gives error in the same unit as the response variable.

Model Evaluation Metrics: Regression Metrics

- **Mean Squared Error (MSE):**

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where y_i is the observed value, $\hat{y}_i$ is the predicted value, and n is the total number of observations.

- **Root Mean Squared Error (RMSE):**

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Gives error in the same unit as the response variable.

- **Coefficient of Determination R^2 :**

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

where $\bar{y}$ is the mean of the observed values. It represents the proportion of variance in y explained by the model.

Model Evaluation Metrics: Classification Metrics

- **Accuracy:**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where:

- TP = True Positives
- TN = True Negatives
- FP = False Positives
- FN = False Negatives

Model Evaluation Metrics: Classification Metrics

- **Accuracy:**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where:

- TP = True Positives
- TN = True Negatives
- FP = False Positives
- FN = False Negatives

- **Precision and Recall:**

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$

Precision measures correctness of positive predictions; recall measures ability to find all actual positives.

Model Evaluation Metrics: Classification Metrics

- **Accuracy:**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where:

- TP = True Positives
- TN = True Negatives
- FP = False Positives
- FN = False Negatives

- **Precision and Recall:**

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$

Precision measures correctness of positive predictions; recall measures ability to find all actual positives.

- **F1 Score** (harmonic mean of precision and recall):

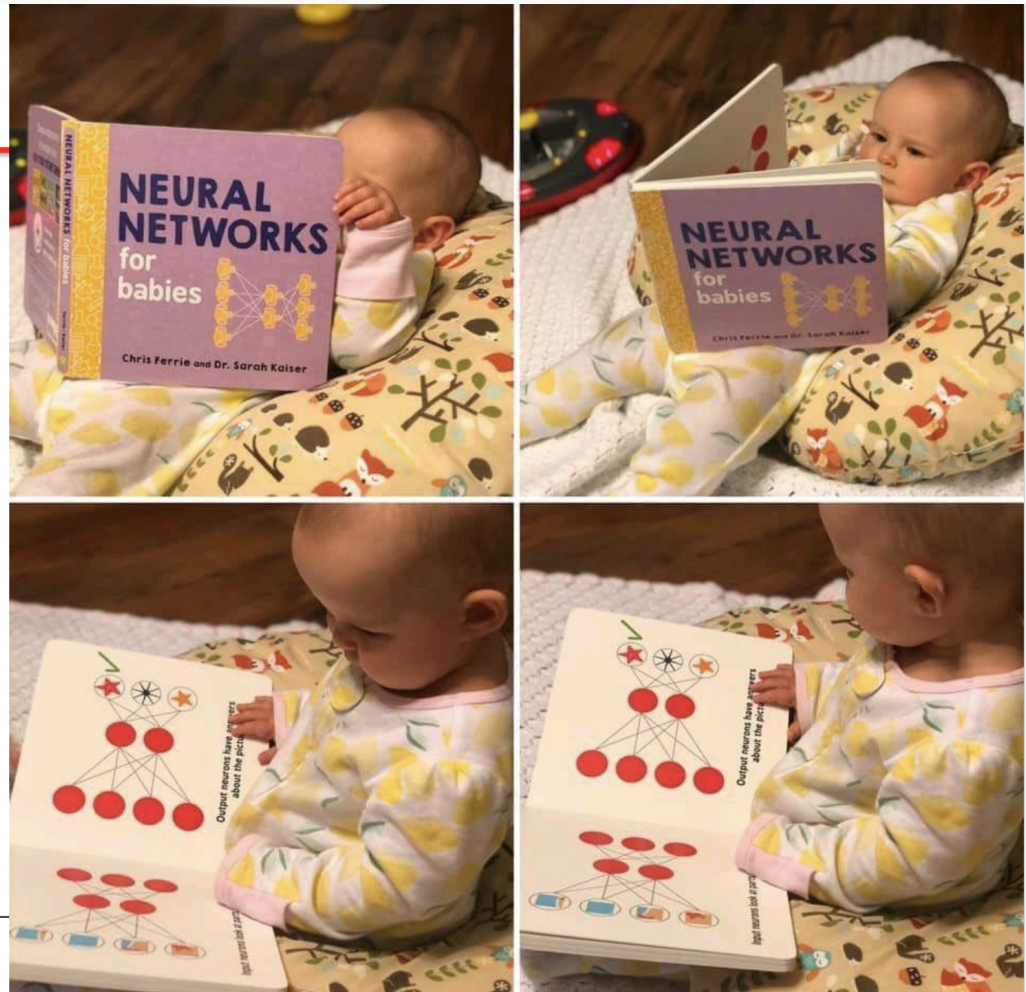
$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Useful when classes are imbalanced or when both precision and recall are important.

Neural Networks

Everybody is learning

Never too late



4

Agenda

2. Neural Networks

2.1 Intro to Neural Networks

2.2 Components of Neural Networks

2.3 Standard Linear and Non-Linear Regressions as Neural Networks

2.4 Unfolding a Two-Layer Neural Network

2.5 Training of NNs (Learning the Weights)

2.6 Some Important Strategies/Considerations When Training a NN

2.7 Building Your Very First Neural Network From Scratch

2.8 Fitting the Neural Network using Tensorflow

2.9 Some key Takeaway

2.10 Motivation for Deep Neural Network (Deep Learning)

What is a Neural Network?

- A **Neural Network** is a computational model inspired by the way biological neural networks in the human brain process information.

What is a Neural Network?

- A **Neural Network** is a computational model inspired by the way biological neural networks in the human brain process information.
- It consists of **layers** of interconnected **nodes**, or **neurons**, which work together to solve complex problems.

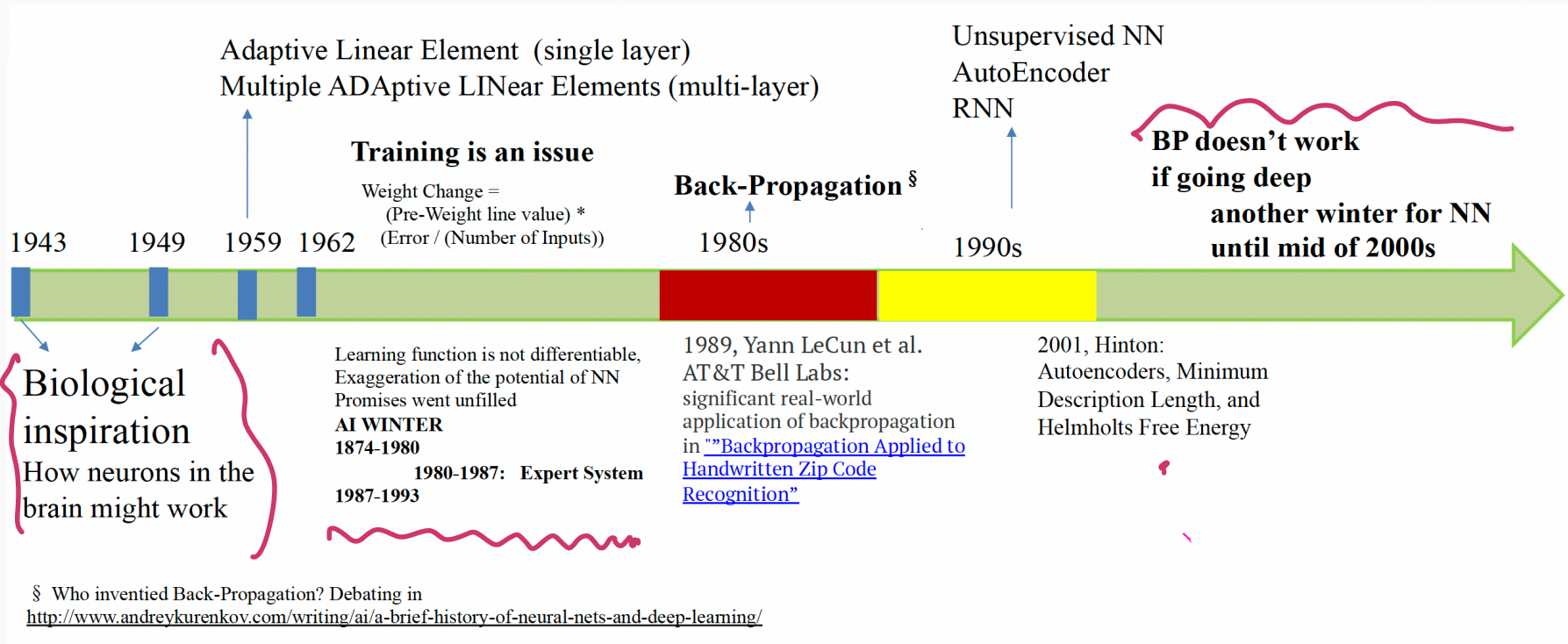
What is a Neural Network?

- A **Neural Network** is a computational model inspired by the way biological neural networks in the human brain process information.
- It consists of **layers** of interconnected **nodes**, or **neurons**, which work together to solve complex problems.
- Similarly, brain activity occurs when a **stimulus** enters the system; information is processed through the network via neurons that extract relevant information, and this information is passed to another area.

What is a Neural Network?

- A **Neural Network** is a computational model inspired by the way biological neural networks in the human brain process information.
- It consists of **layers** of interconnected **nodes**, or **neurons**, which work together to solve complex problems.
- Similarly, brain activity occurs when a **stimulus** enters the system; information is processed through the network via neurons that extract relevant information, and this information is passed to another area.
- Neural networks can **learn** from data through **training**, making them powerful tools for **pattern recognition** and **predictive modeling**.

History of NNs



5

Components of Neural Networks

- Layers
- Neurons
- Weights
- Bias
- Activation Functions
- Forward Propagation
- Loss Function
- Backpropagation
- Learning Rate
- Training Data
- Validation and Test Data

A Basic Architectures of Neural Networks (1)

- A neural network consists of several key components:

A Basic Architectures of Neural Networks (1)

- A neural network consists of several key components:
- **Input Layer:** Each input represents a single predictor variable, which can be a scalar value, a sequence, an image, or even a sequence of images.

A Basic Architectures of Neural Networks (1)

- A neural network consists of several key components:
- **Input Layer:** Each input represents a single predictor variable, which can be a scalar value, a sequence, an image, or even a sequence of images.
- **Output Layer:** For prediction or classification tasks, this layer typically has a single node. For Conditional Density Estimation (CDE), it contains multiple nodes.

A Basic Architectures of Neural Networks (1)

- A neural network consists of several key components:
- **Input Layer:** Each input represents a single predictor variable, which can be a scalar value, a sequence, an image, or even a sequence of images.
- **Output Layer:** For prediction or classification tasks, this layer typically has a single node. For Conditional Density Estimation (CDE), it contains multiple nodes.
- **Hidden Layers:** These layers, often more than one, perform calculations at each node. Each hidden layer's computations are parameterized by weights and biases.

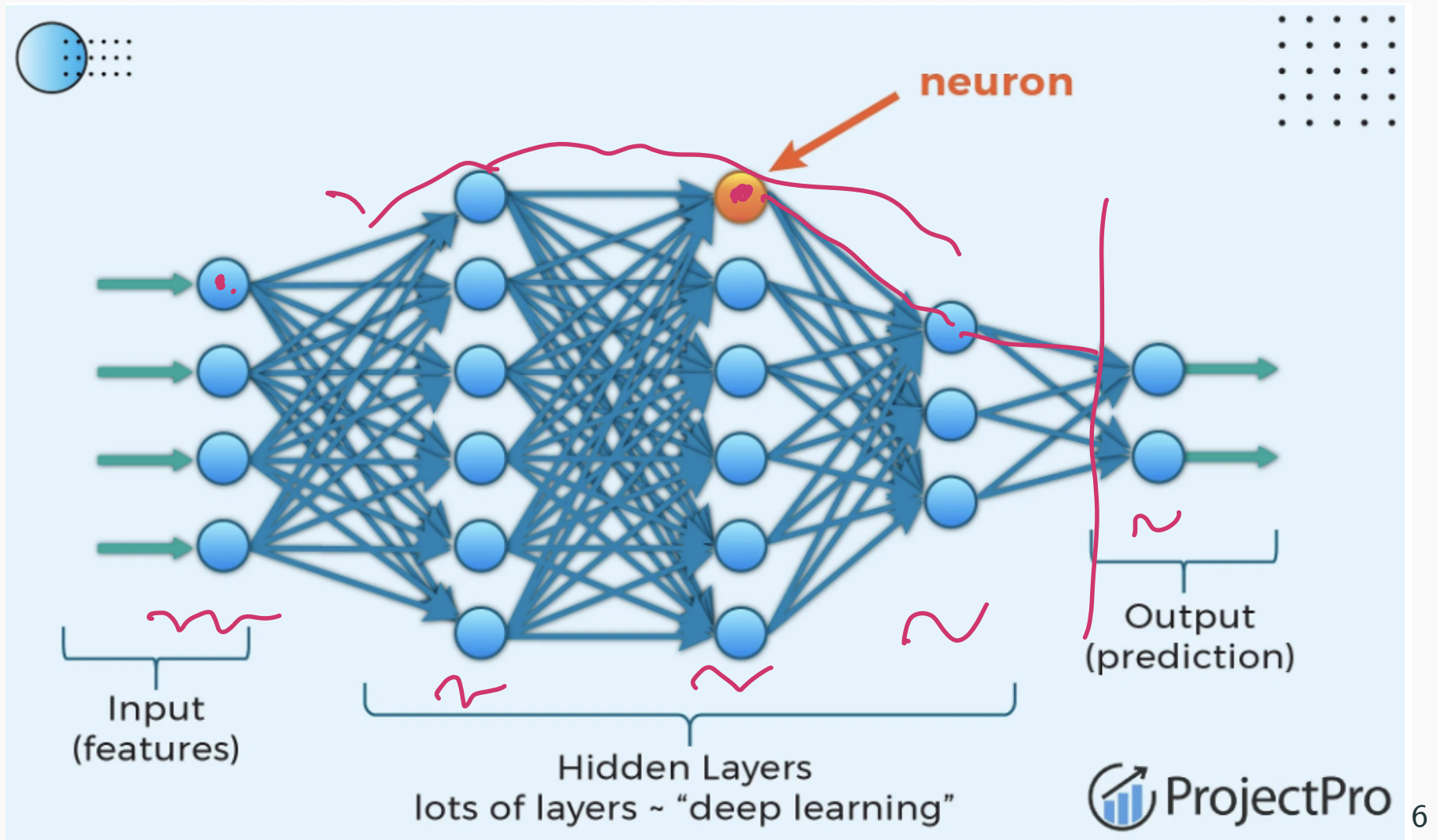
A Basic Architectures of Neural Networks (1)

- A neural network consists of several key components:
- **Input Layer:** Each input represents a single predictor variable, which can be a scalar value, a sequence, an image, or even a sequence of images.
- **Output Layer:** For prediction or classification tasks, this layer typically has a single node. For Conditional Density Estimation (CDE), it contains multiple nodes.
- **Hidden Layers:** These layers, often more than one, perform calculations at each node. Each hidden layer's computations are parameterized by weights and biases.
- The calculations within a layer can vary in type, including standard, convolutional, and recurrent layers.

A Basic Architectures of Neural Networks (1)

- A neural network consists of several key components:
- **Input Layer:** Each input represents a single predictor variable, which can be a scalar value, a sequence, an image, or even a sequence of images.
- **Output Layer:** For prediction or classification tasks, this layer typically has a single node. For Conditional Density Estimation (CDE), it contains multiple nodes.
- **Hidden Layers:** These layers, often more than one, perform calculations at each node. Each hidden layer's computations are parameterized by weights and biases.
- The calculations within a layer can vary in type, including standard, convolutional, and recurrent layers.
- The structure of a neural network, referred to as its **architecture**, defines the arrangement and connectivity of these layers.

A Basic Architectures of Neural Networks (2)



Agenda

2. Neural Networks

2.1 Intro to Neural Networks

2.2 Components of Neural Networks

2.3 Standard Linear and Non-Linear Regressions as Neural Networks

2.4 Unfolding a Two-Layer Neural Network

2.5 Training of NNs (Learning the Weights)

2.6 Some Important Strategies/Considerations When Training a NN

2.7 Building Your Very First Neural Network From Scratch

2.8 Fitting the Neural Network using Tensorflow

2.9 Some key Takeaway

2.10 Motivation for Deep Neural Network (Deep Learning)

Layers

Neural computing requires a number of neurons, to be connected together into a neural network. Neurons are arranged in layers.

- **Input Layer**

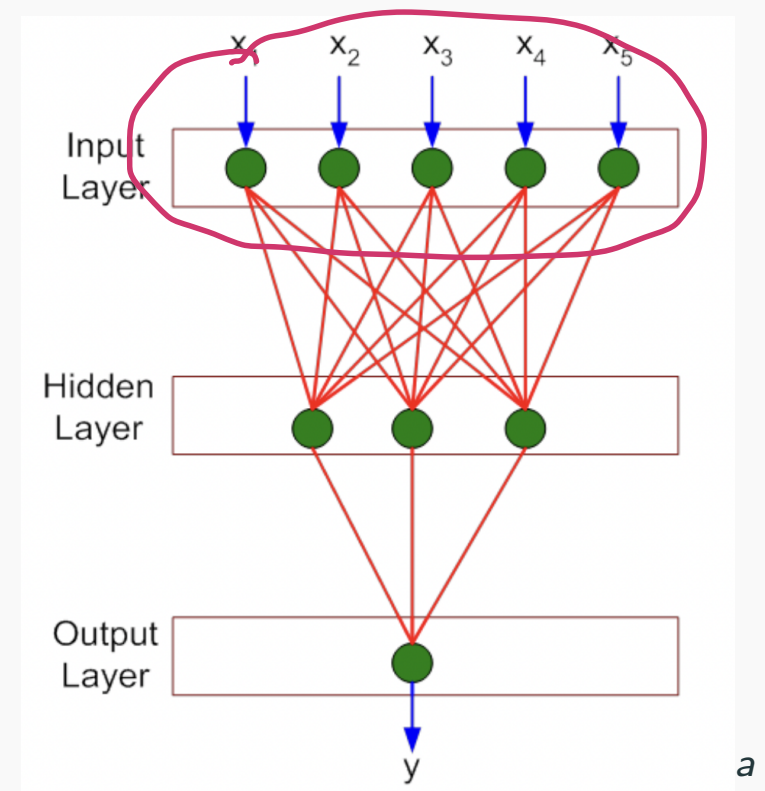
- First layer of neurons.
- Receives raw data directly from the input features.

- **Hidden Layers**

- Layers between the input and output layers.
- Perform computations.
- Can have one or many hidden layers.

- **Output Layer**

- Final layer.
- Produces the network's output.
- Structure depends on the specific task (e.g., classification, regression).



^aCS 229: Prof. Xiangliang Zhang (KAUST)

Layers

Neural computing requires a number of neurons, to be connected together into a neural network. Neurons are arranged in layers.

- **Input Layer**

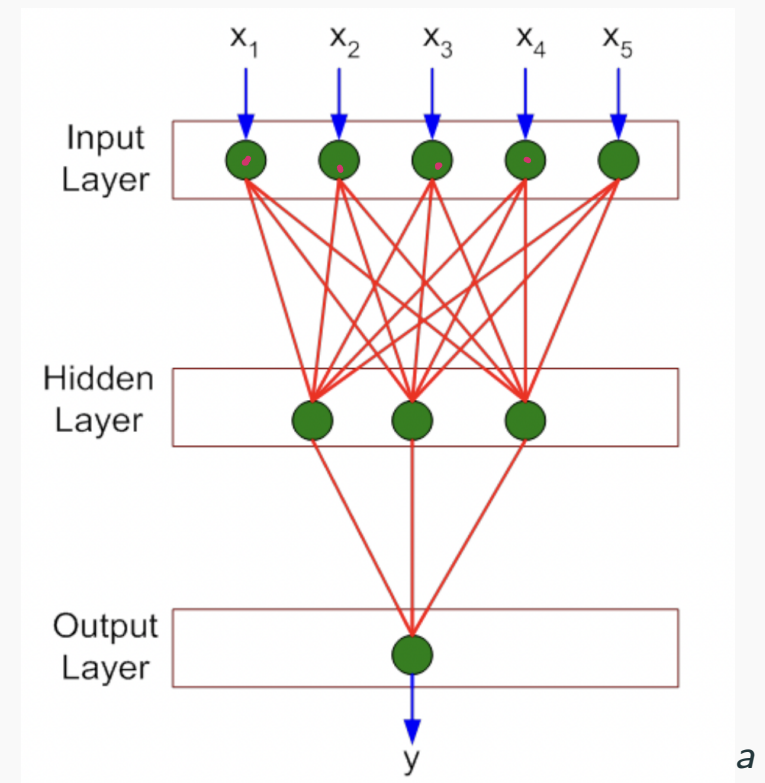
- First layer of neurons.
- Receives raw data directly from the input features.

- **Hidden Layers**

- Layers between the input and output layers.
- Perform computations.
- Can have one or many hidden layers.

- **Output Layer**

- Final layer.
- Produces the network's output.
- Structure depends on the specific task (e.g., classification, regression).



^aCS 229: Prof. Xiangliang Zhang (KAUST)

Layers

Neural computing requires a number of neurons, to be connected together into a neural network. Neurons are arranged in layers.

- **Input Layer**

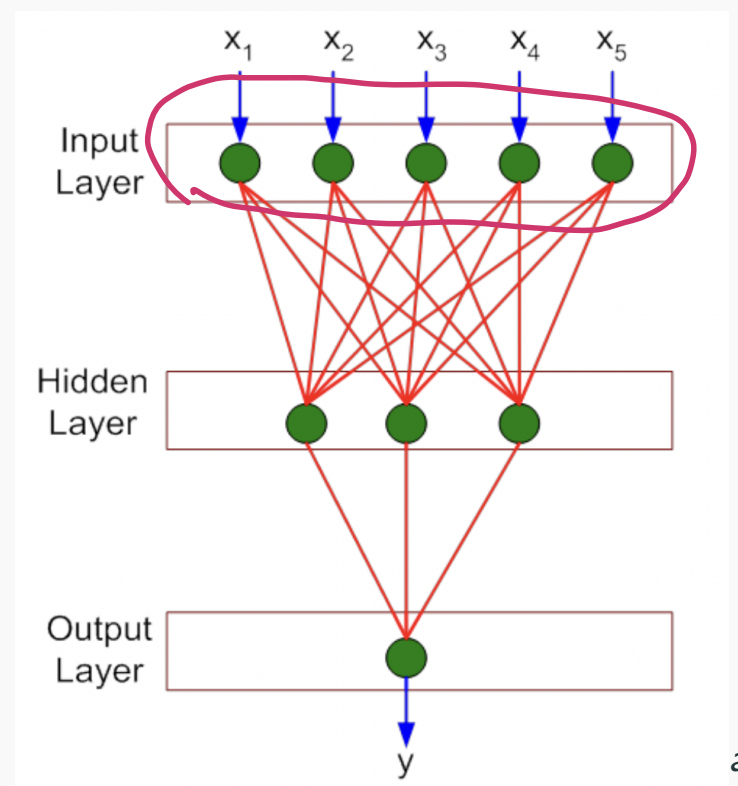
- First layer of neurons.
- Receives raw data directly from the input features.

- **Hidden Layers**

- Layers between the input and output layers.
- Perform computations.
- Can have one or many hidden layers.

- **Output Layer**

- Final layer.
- Produces the network's output.
- Structure depends on the specific task (e.g., classification, regression).



^aCS 229: Prof. Xiangliang Zhang (KAUST)

Layers

Neural computing requires a number of neurons, to be connected together into a neural network. Neurons are arranged in layers.

- **Input Layer**

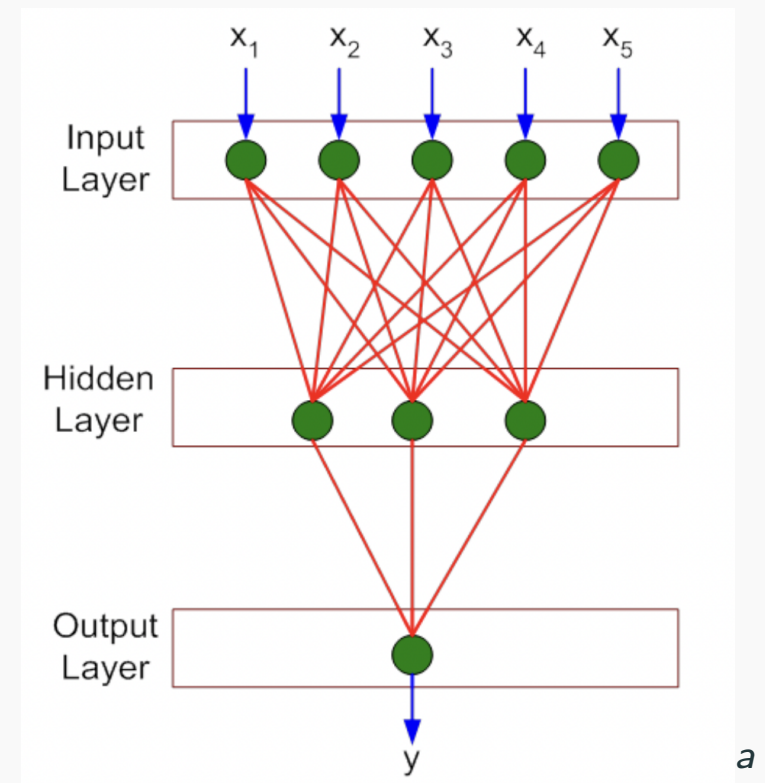
- First layer of neurons.
- Receives raw data directly from the input features.

- **Hidden Layers**

- Layers between the input and output layers.
- Perform computations.
- Can have one or many hidden layers.

- **Output Layer**

- Final layer.
- Produces the network's output.
- Structure depends on the specific task (e.g., classification, regression).



^aCS 229: Prof. Xiangliang Zhang (KAUST)

Layers

Neural computing requires a number of neurons, to be connected together into a neural network. Neurons are arranged in layers.

- **Input Layer**

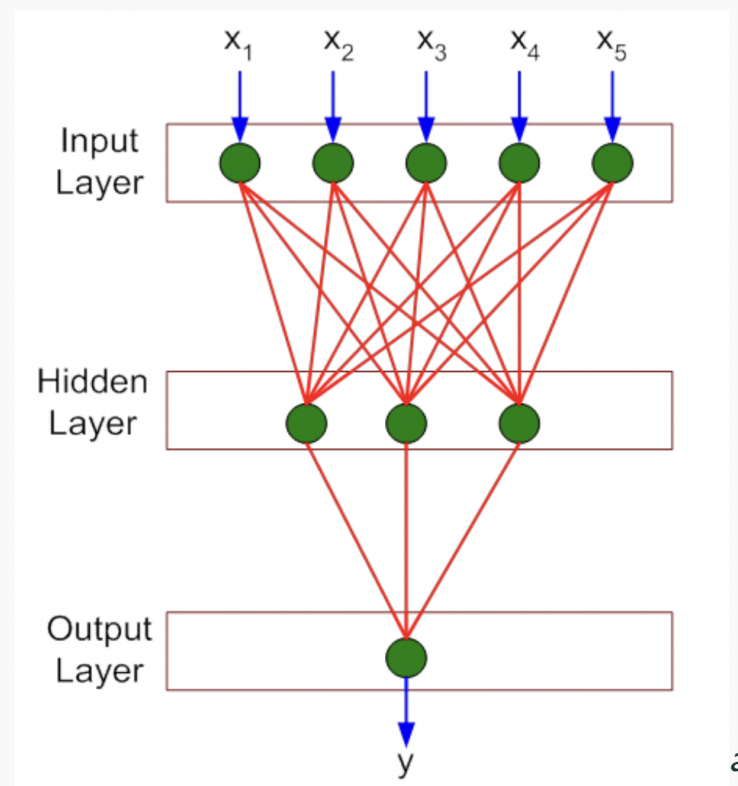
- First layer of neurons.
- Receives raw data directly from the input features.

- **Hidden Layers**

- Layers between the input and output layers.
- Perform computations.
- Can have one or many hidden layers.

- **Output Layer**

- Final layer.
- Produces the network's output.
- Structure depends on the specific task (e.g., classification, regression).



^aCS 229: Prof. Xiangliang Zhang (KAUST)

Layers

Neural computing requires a number of neurons, to be connected together into a neural network. Neurons are arranged in layers.

- **Input Layer**

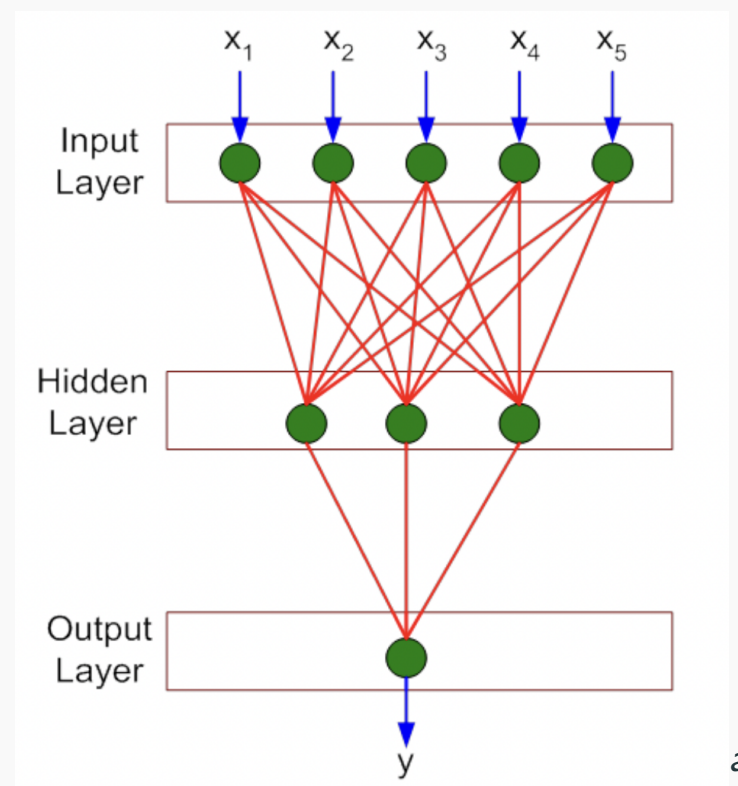
- First layer of neurons.
- Receives raw data directly from the input features.

- **Hidden Layers**

- Layers between the input and output layers.
- Perform computations.
- Can have one or many hidden layers.

- **Output Layer**

- Final layer.
- Produces the network's output.
- Structure depends on the specific task (e.g., classification, regression).



^aCS 229: Prof. Xiangliang Zhang (KAUST)

Layers

Neural computing requires a number of neurons, to be connected together into a neural network. Neurons are arranged in layers.

- **Input Layer**

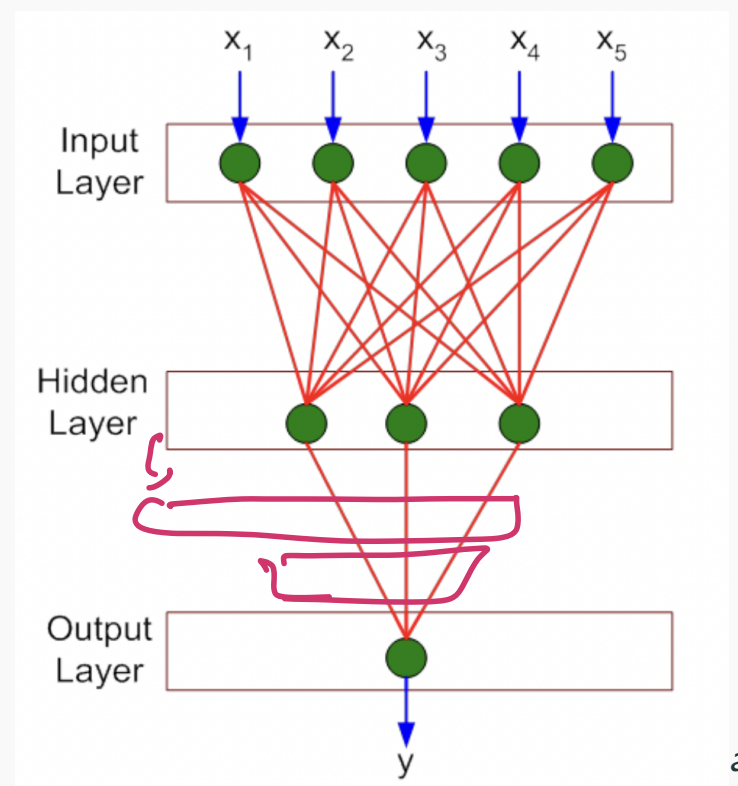
- First layer of neurons.
- Receives raw data directly from the input features.

- **Hidden Layers**

- Layers between the input and output layers.
- Perform computations.
- Can have one or many hidden layers.

- **Output Layer**

- Final layer.
- Produces the network's output.
- Structure depends on the specific task (e.g., classification, regression).



^aCS 229: Prof. Xiangliang Zhang (KAUST)

Layers

Neural computing requires a number of neurons, to be connected together into a neural network. Neurons are arranged in layers.

- **Input Layer**

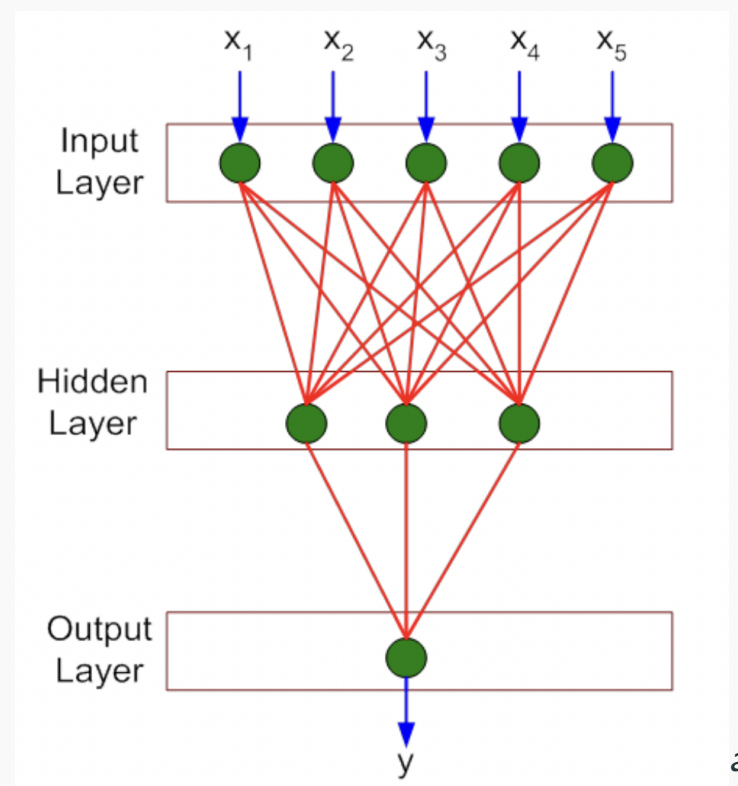
- First layer of neurons.
- Receives raw data directly from the input features.

- **Hidden Layers**

- Layers between the input and output layers.
- Perform computations.
- Can have one or many hidden layers.

- **Output Layer**

- Final layer.
- Produces the network's output.
- Structure depends on the specific task (e.g., classification, regression).



^aCS 229: Prof. Xiangliang Zhang (KAUST)

Layers

Neural computing requires a number of neurons, to be connected together into a neural network. Neurons are arranged in layers.

- **Input Layer**

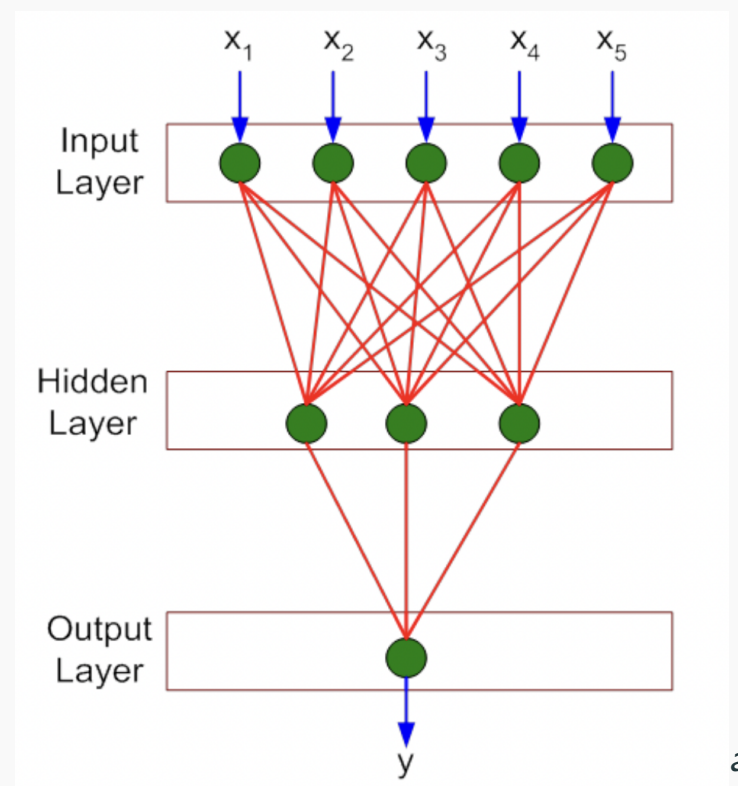
- First layer of neurons.
- Receives raw data directly from the input features.

- **Hidden Layers**

- Layers between the input and output layers.
- Perform computations.
- Can have one or many hidden layers.

- **Output Layer**

- Final layer.
- Produces the network's output.
- Structure depends on the specific task (e.g., classification, regression).



^aCS 229: Prof. Xiangliang Zhang (KAUST)

Layers

Neural computing requires a number of neurons, to be connected together into a neural network. Neurons are arranged in layers.

- **Input Layer**

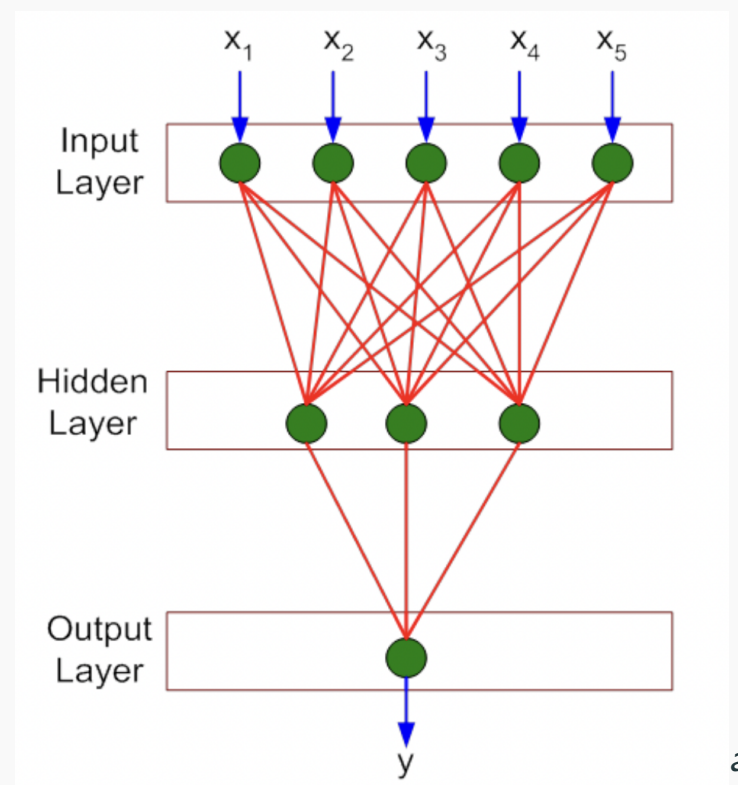
- First layer of neurons.
- Receives raw data directly from the input features.

- **Hidden Layers**

- Layers between the input and output layers.
- Perform computations.
- Can have one or many hidden layers.

- **Output Layer**

- Final layer.
- Produces the network's output.
- Structure depends on the specific task (e.g., classification, regression).



^aCS 229: Prof. Xiangliang Zhang (KAUST)

Layers

Neural computing requires a number of neurons, to be connected together into a neural network. Neurons are arranged in layers.

- **Input Layer**

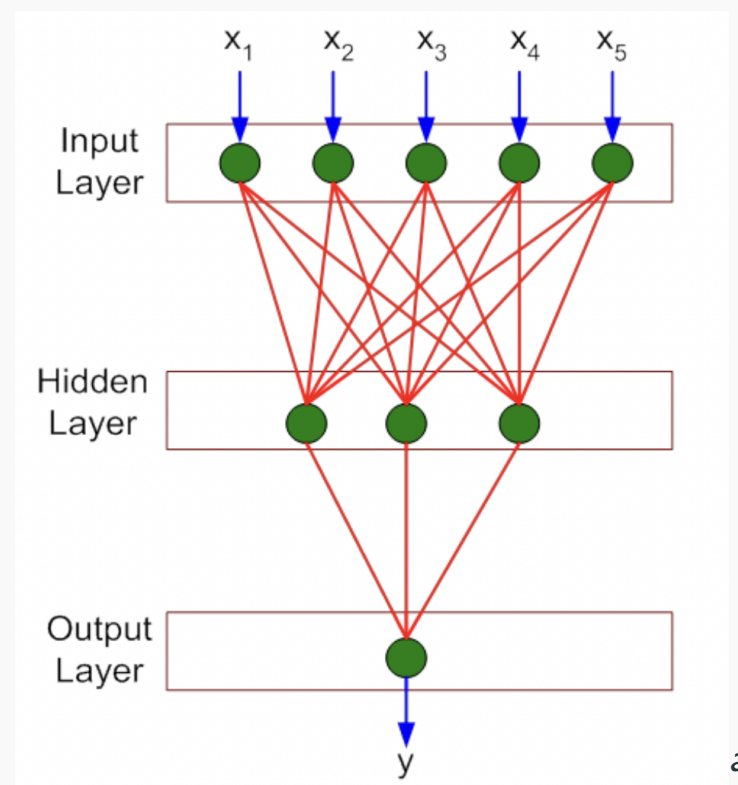
- First layer of neurons.
- Receives raw data directly from the input features.

- **Hidden Layers**

- Layers between the input and output layers.
- Perform computations.
- Can have one or many hidden layers.

- **Output Layer**

- Final layer.
- Produces the network's output.
- Structure depends on the specific task (e.g., classification, regression).



^aCS 229: Prof. Xiangliang Zhang (KAUST)

Neurons, Weights, and Biases

- **Neurons:**

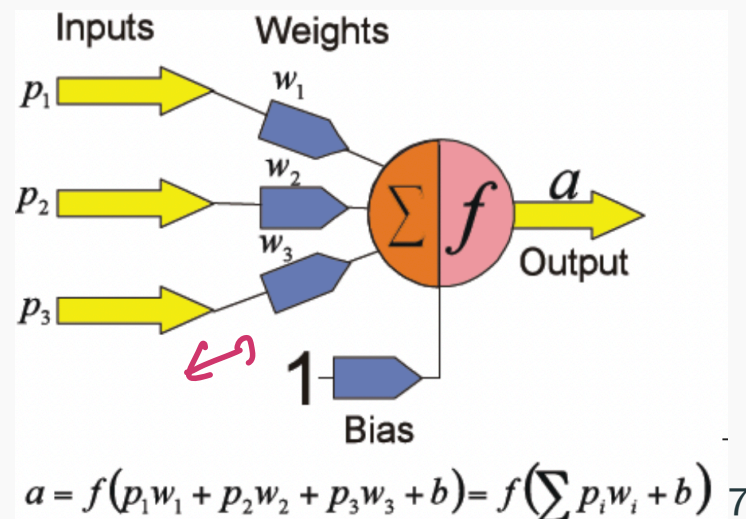
- Basic units of a neural network.
- Receive input, process it, and pass it to other neurons.
- Apply a mathematical function to generate an output.

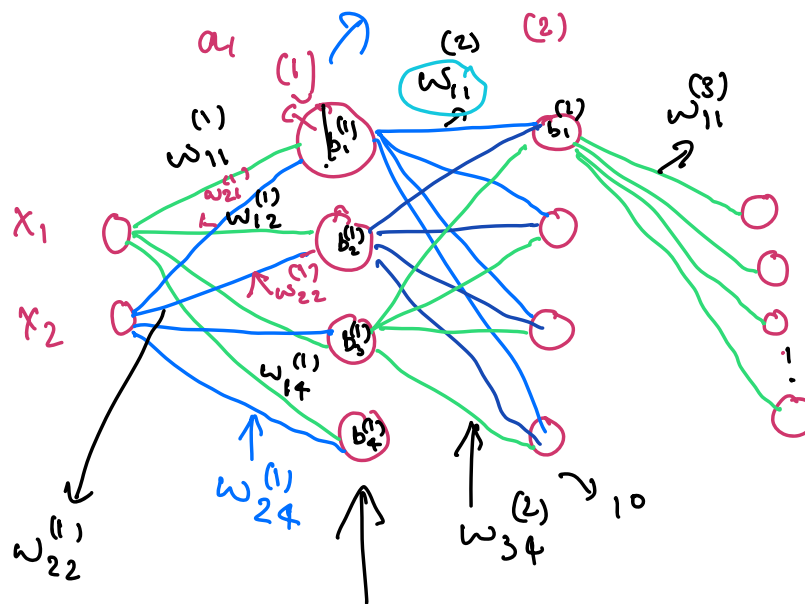
- **Weights:**

- Parameters that transform input data within the network.
- Each connection between neurons has an associated weight.

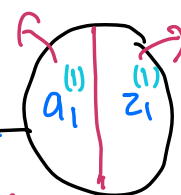
- **Biases:**

- Additional parameters in a neuron that allow to fit the data better.
- Provide ability to each neuron have output not 0 when inputs are 0.

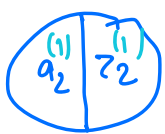


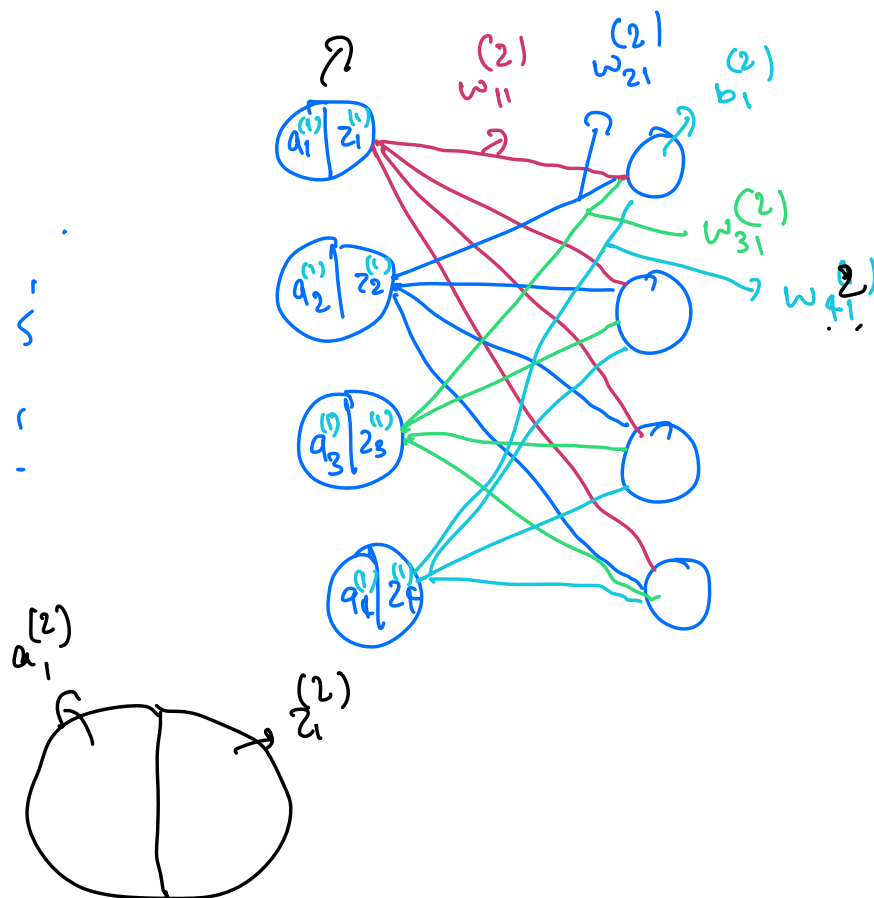


←————— activation —————→



$$\begin{aligned}
 a_1 &= w_{11}^{(1)} x_1 + w_{21}^{(1)} x_2 + b_1^{(1)} \rightarrow h(a_1) = z_1 \\
 a_2 &= w_{22}^{(1)} x_2 + w_{12}^{(1)} x_1 + b_2^{(1)} \rightarrow h(a_2) = z_2 \\
 a_4 &= \dots \rightarrow h(a_4) = z_4 \\
 a_m &= \dots
 \end{aligned}$$





$$a_1^{(2)} = w_{11}^{(2)} z_1^{(1)} + w_{21}^{(2)} z_2^{(1)} + w_{31}^{(2)} z_3^{(1)} + w_{41}^{(2)} z_4^{(1)} + b_1^{(2)}$$

$$z_1^{(2)} = \tanh(a_1^{(2)})$$

Neurons, Weights, and Biases

- **Neurons:**

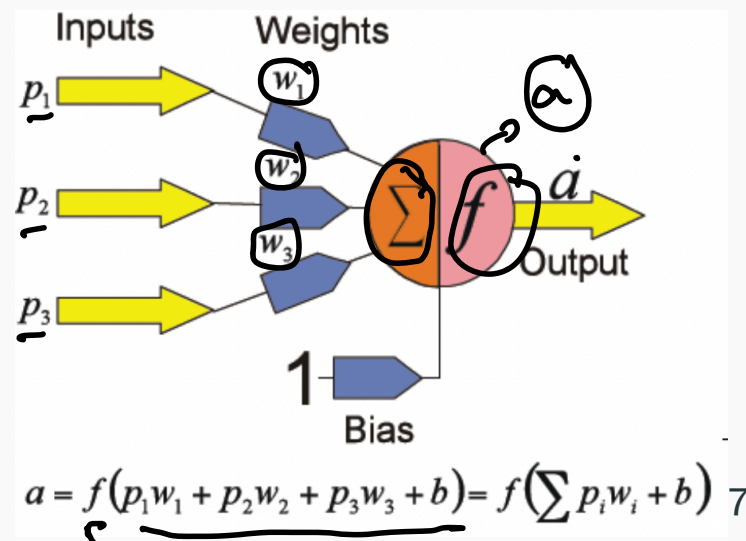
- Basic units of a neural network.
- Receive input, process it, and pass it to other neurons.
- Apply a mathematical function to generate an output.

- **Weights:**

- Parameters that transform input data within the network.
- Each connection between neurons has an associated weight.

- **Biases:**

- Additional parameters in a neuron that allow to fit the data better.
- Provide ability to each neuron have output not 0 when inputs are 0.



Neurons, Weights, and Biases

- **Neurons:**

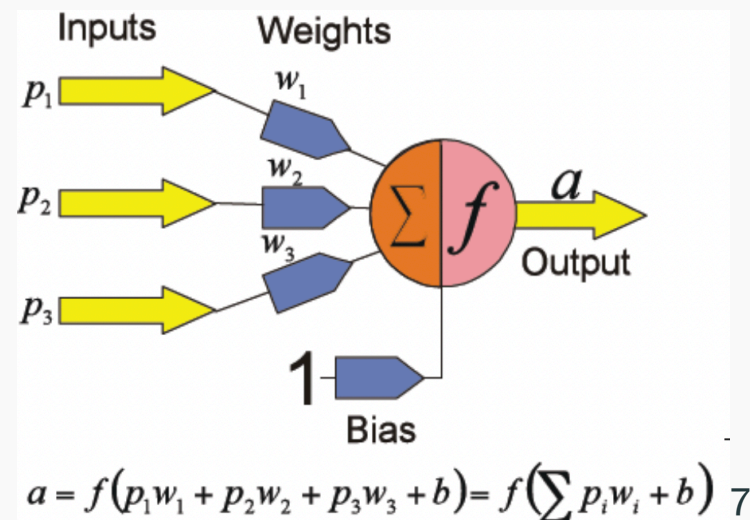
- Basic units of a neural network.
- Receive input, process it, and pass it to other neurons.
- Apply a mathematical function to generate an output.

- **Weights:**

- Parameters that transform input data within the network.
- Each connection between neurons has an associated weight.

- **Biases:**

- Additional parameters in a neuron that allow to fit the data better.
- Provide ability to each neuron have output not 0 when inputs are 0.



Neurons, Weights, and Biases

- **Neurons:**

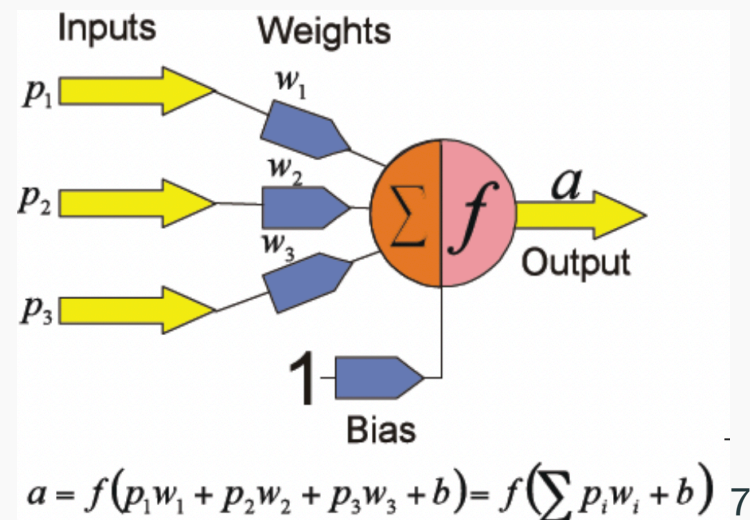
- Basic units of a neural network.
- Receive input, process it, and pass it to other neurons.
- Apply a mathematical function to generate an output.

- **Weights:**

- Parameters that transform input data within the network.
- Each connection between neurons has an associated weight.

- **Biases:**

- Additional parameters in a neuron that allow to fit the data better.
- Provide ability to each neuron have output not 0 when inputs are 0.



Neurons, Weights, and Biases

- **Neurons:**

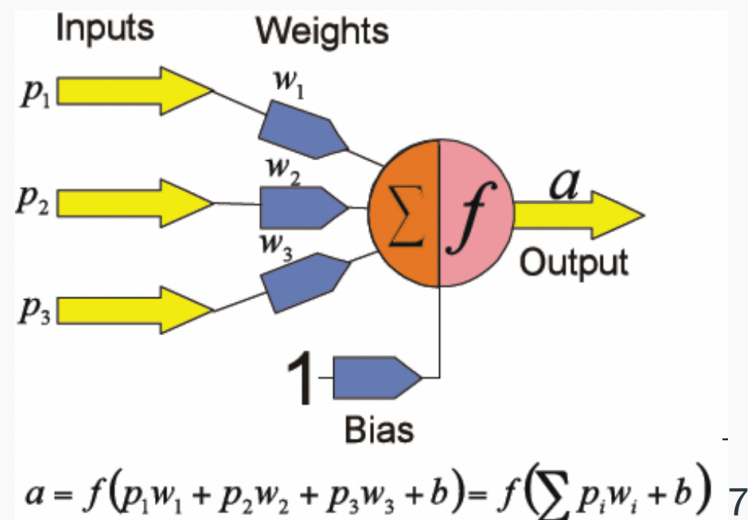
- Basic units of a neural network.
- Receive input, process it, and pass it to other neurons.
- Apply a mathematical function to generate an output.

- **Weights:**

- Parameters that transform input data within the network.
- Each connection between neurons has an associated weight.

- **Biases:**

- Additional parameters in a neuron that allow to fit the data better.
- Provide ability to each neuron have output not 0 when inputs are 0.



Neurons, Weights, and Biases

- **Neurons:**

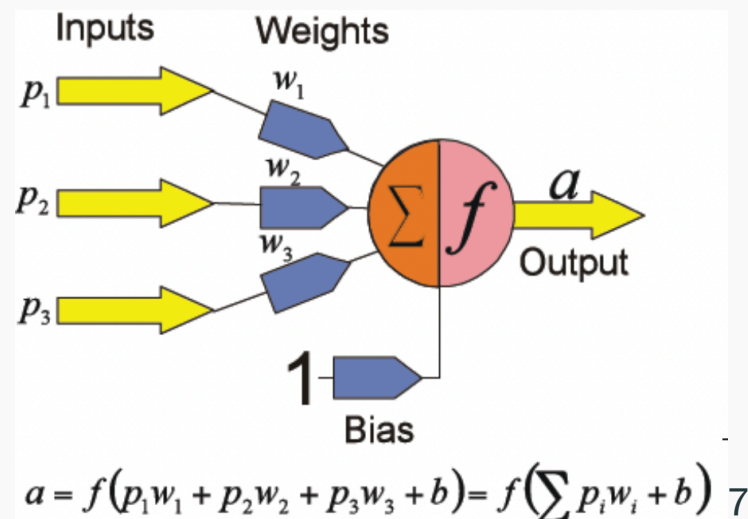
- Basic units of a neural network.
- Receive input, process it, and pass it to other neurons.
- Apply a mathematical function to generate an output.

- **Weights:**

- Parameters that transform input data within the network.
- Each connection between neurons has an associated weight.

- **Biases:**

- Additional parameters in a neuron that allow to fit the data better.
- Provide ability to each neuron have output not 0 when inputs are 0.



Neurons, Weights, and Biases

- **Neurons:**

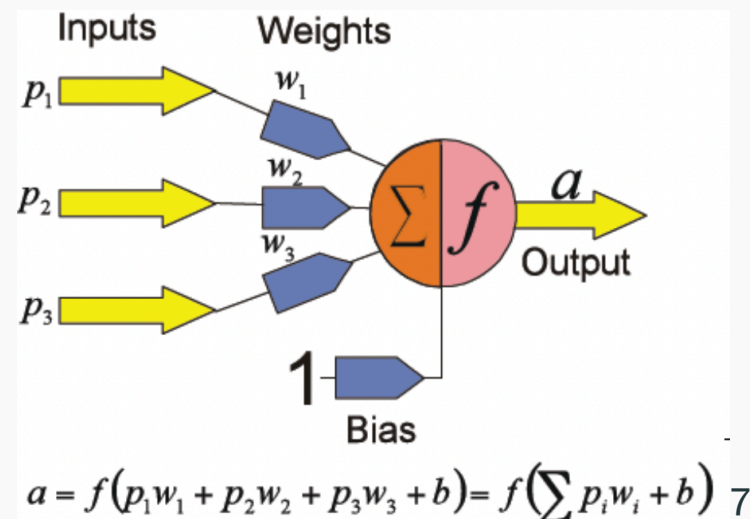
- Basic units of a neural network.
- Receive input, process it, and pass it to other neurons.
- Apply a mathematical function to generate an output.

- **Weights:**

- Parameters that transform input data within the network.
- Each connection between neurons has an associated weight.

- **Biases:**

- Additional parameters in a neuron that allow to fit the data better.
- Provide ability to each neuron have output not 0 when inputs are 0.



Neurons, Weights, and Biases

- **Neurons:**

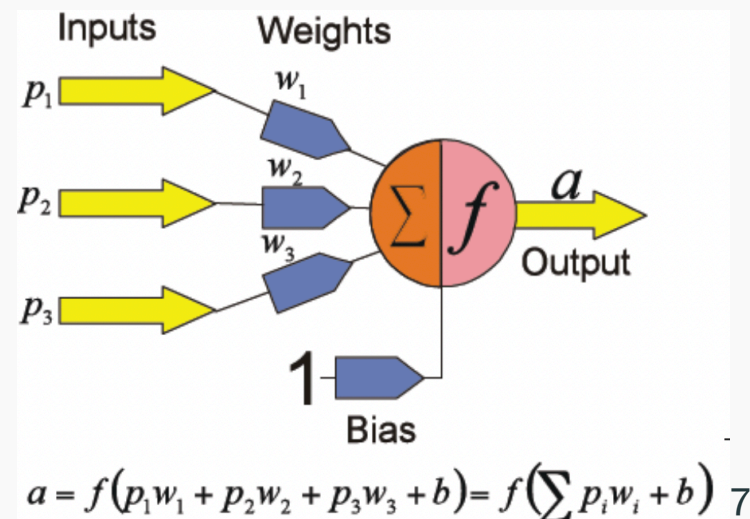
- Basic units of a neural network.
- Receive input, process it, and pass it to other neurons.
- Apply a mathematical function to generate an output.

- **Weights:**

- Parameters that transform input data within the network.
- Each connection between neurons has an associated weight.

- **Biases:**

- Additional parameters in a neuron that allow to fit the data better.
- Provide ability to each neuron have output not 0 when inputs are 0.



Neurons, Weights, and Biases

- **Neurons:**

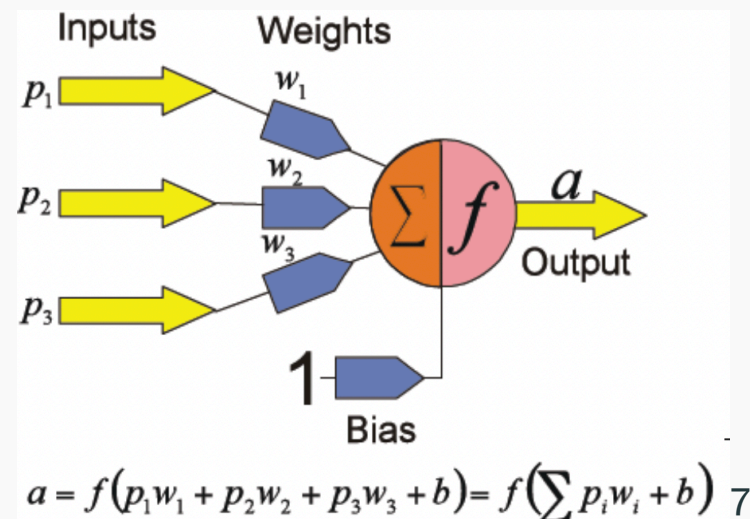
- Basic units of a neural network.
- Receive input, process it, and pass it to other neurons.
- Apply a mathematical function to generate an output.

- **Weights:**

- Parameters that transform input data within the network.
- Each connection between neurons has an associated weight.

- **Biases:**

- Additional parameters in a neuron that allow to fit the data better.
- Provide ability to each neuron have output not 0 when inputs are 0.



Neurons, Weights, and Biases

- **Neurons:** ✓

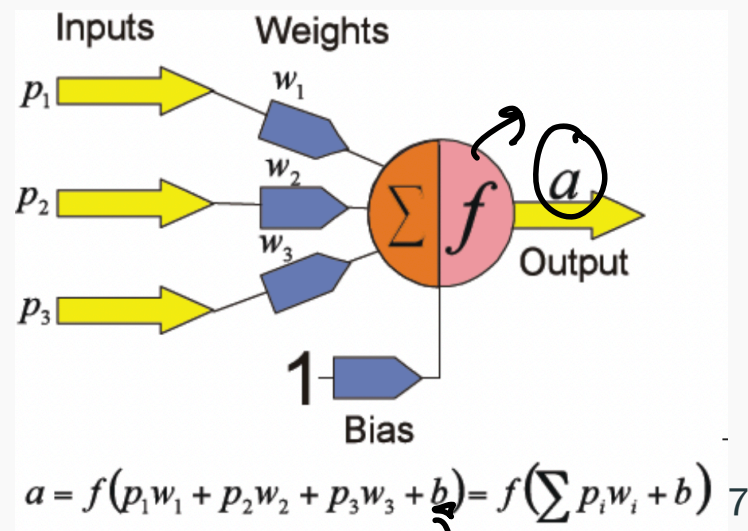
- Basic units of a neural network.
- Receive input, process it, and pass it to other neurons.
- Apply a mathematical function to generate an output.

- **Weights:** ✓

- Parameters that transform input data within the network.
- Each connection between neurons has an associated weight.

- **Biases:** ✓

- Additional parameters in a neuron that allow to fit the data better.
- Provide ability to each neuron have output not 0 when inputs are 0.



Activation function

- The activation function defines the **output of the node**. There is two different functions that we can use, but it must satisfy below two properties

Activation function

- The activation function defines the **output of the node**. There is two different functions that we can use, but it must satisfy below two properties
 - **Nonlinear**: If all node has linear activation functions, this is just a linear regression model.

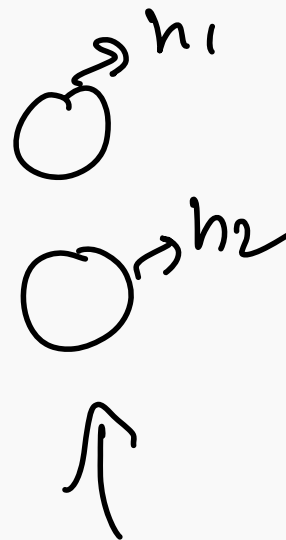
Activation function

- The activation function defines the **output of the node**. There is two different functions that we can use, but it must satisfy below two properties
 - **Nonlinear**: If all node has linear activation functions, this is just a linear regression model.
 - **Continuously differentiable**: Needed for gradient based optimization

11)

Activation function

- The activation function defines the **output of the node**. There is two different functions that we can use, but it must satisfy below two properties
 - **Nonlinear**: If all node has linear activation functions, this is just a linear regression model.
 - **Continuously differentiable**: Needed for gradient based optimization



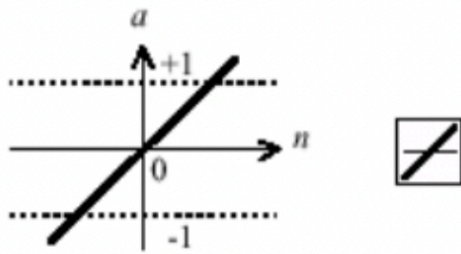
Activation function

- The activation function defines the **output of the node**. There is two different functions that we can use, but it must satisfy below two properties
 - **Nonlinear**: If all node has linear activation functions, this is just a linear regression model.
 - **Continuously differentiable**: Needed for gradient based optimization

For example, we can think of this activation functions as link function in generalized linear model. In fact a single layered NN is simply a generalized linear model.

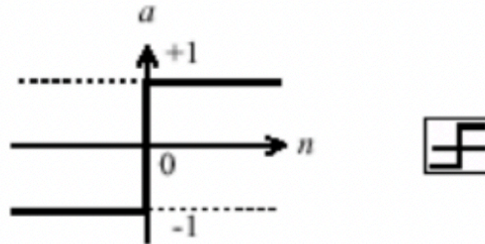
Some Examples of Activation Function

$$f(a_1w_1 + a_2w_2 + b_1) = a_1w_1 + a_2w_2 + b_1$$



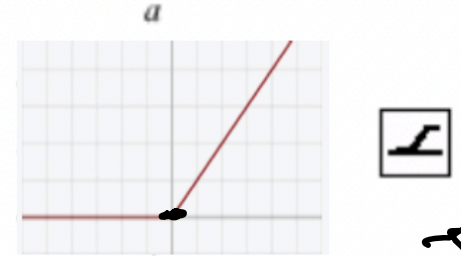
$$a = \text{purelin}(n)$$

Linear Transfer Function



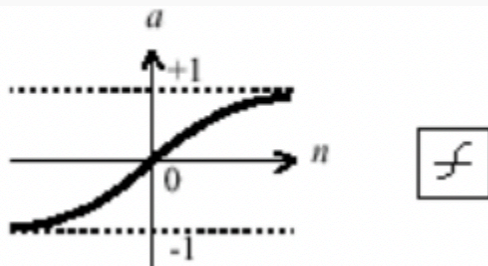
$$a = \text{hardlims}(n)$$

Symmetric Hard Limit Trans. Funct.



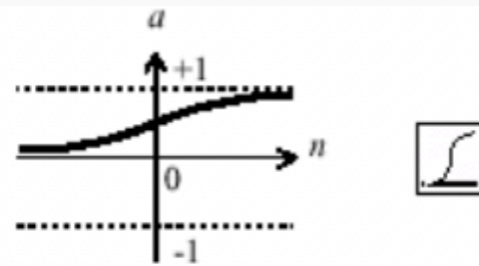
ReLU function $+ e^x$

8



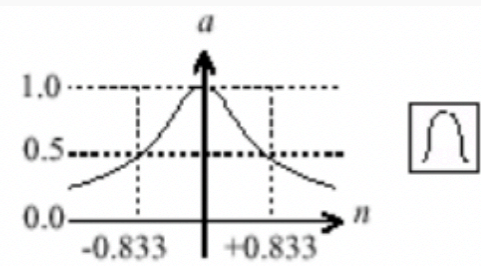
$$a = \text{tansig}(n)$$

Tan-Sigmoid Transfer Function



$$a = \text{logsig}(n)$$

Log-Sigmoid Transfer Function



$$a = \text{radbas}(n)$$

Radial Basis Function

Rectified Linear Unit (ReLU) Activation Function

- Older neural networks relied on sigmoid or tanh activation functions that suffered from vanishing gradient problems which restrict going to deep

$$\frac{\partial L}{\partial \omega^{(1)}} \rightarrow \frac{\partial L}{\partial \omega^{(2)}} - \frac{\partial L}{\partial \omega^{(k)}}$$

$$= \left(\begin{array}{c} \vdots \\ \vdots \end{array} \right) \cdot \left(\begin{array}{c} \vdots \\ \vdots \end{array} \right) \cdot \frac{\partial f(\omega)}{\partial \omega}$$

$$f(\omega) = f_1(f_2(f_3(\omega)))$$

⁹<https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/>

Rectified Linear Unit (ReLU) Activation Function

- Older neural networks relied on **sigmoid** or **tanh** activation functions that suffered from vanishing gradient problems which restrict going to deep
- **ReLU**⁹ activation function somewhat solve this issue and turn out to be a huge help

Handwritten diagram illustrating the vanishing gradient problem. It shows the weight update equation $w^{(t+1)} = w^{(t)} - \eta \nabla L(w^{(t)})$. A bracket under the term $\nabla L(w^{(t)})$ points to a scribble and the number 0, indicating that the gradient vanishes.

⁹<https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/>

Rectified Linear Unit (ReLU) Activation Function

- Older neural networks relied on **sigmoid** or **tanh** activation functions that suffered from vanishing gradient problems which restrict going to deep
- **ReLU**⁹ activation function somewhat solve this issue and turn out to be a huge help
- **ReLU** has the following forms

$$a(x) = \max\{0, x\}.$$

That means

$$\Rightarrow a(\omega_1 x_1 + \dots + \omega_p x_p + b) = \max\left(0, \boxed{\omega_1 x_1 + \dots + b}\right)$$

⁹<https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/>

Rectified Linear Unit (ReLU) Activation Function

- Older neural networks relied on **sigmoid** or **tanh** activation functions that suffered from vanishing gradient problems which restrict going to deep
- **ReLU**⁹ activation function somewhat solve this issue and turn out to be a huge help
- **ReLU** has the following forms

$$a(x) = \max\{0, x\}.$$

Handwritten note: $\approx 0 \wedge x$

That means

$$\underline{x > 0} \rightarrow x$$

- The neuron **activates** only when enough information passes through

⁹<https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/>

Rectified Linear Unit (ReLU) Activation Function

- Older neural networks relied on **sigmoid** or **tanh** activation functions that suffered from vanishing gradient problems which restrict going to deep
- **ReLU**⁹ activation function somewhat solve this issue and turn out to be a huge help
- **ReLU** has the following forms

$$a(x) = \max\{0, x\}.$$

That means

- The neuron **activates** only when enough information passes through
- **One drawbacks**: Not differentiable at zero.

⁹<https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/>

Agenda

2. Neural Networks

2.1 Intro to Neural Networks ✓

2.2 Components of Neural Networks ✓

2.3 Standard Linear and Non-Linear Regressions as Neural Networks

2.4 Unfolding a Two-Layer Neural Network

2.5 Training of NNs (Learning the Weights)

2.6 Some Important Strategies/Considerations When Training a NN

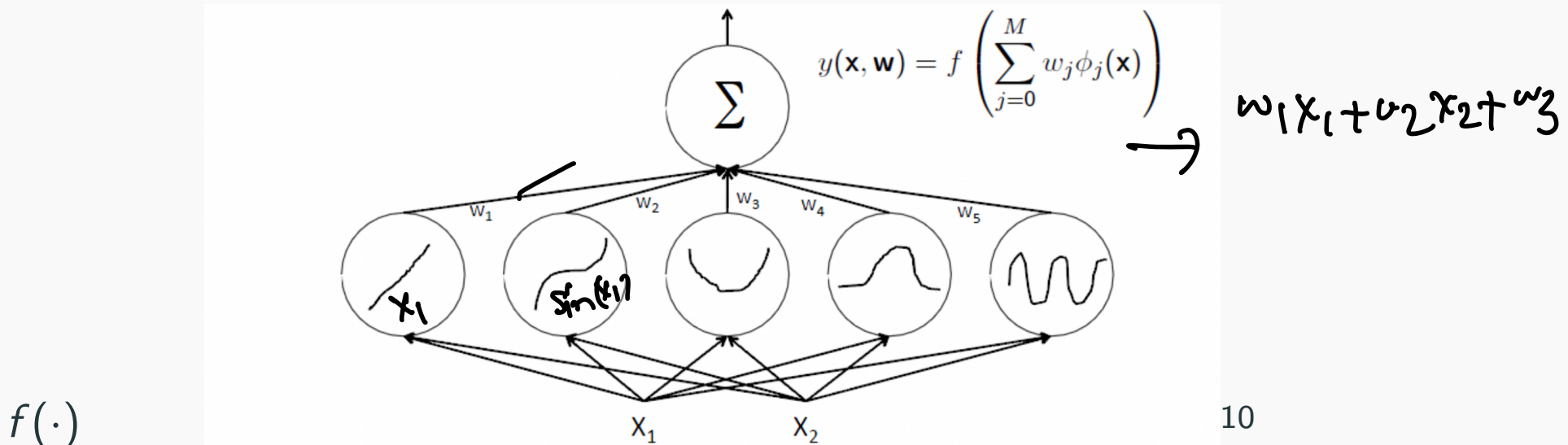
2.7 Building Your Very First Neural Network From Scratch

2.8 Fitting the Neural Network using Tensorflow

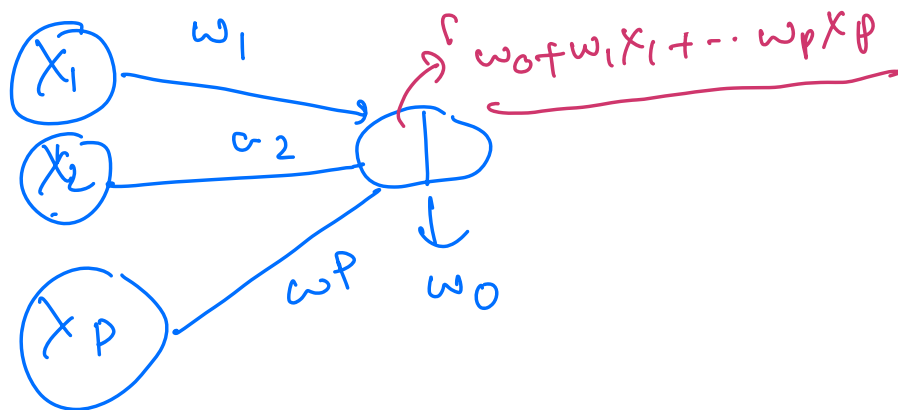
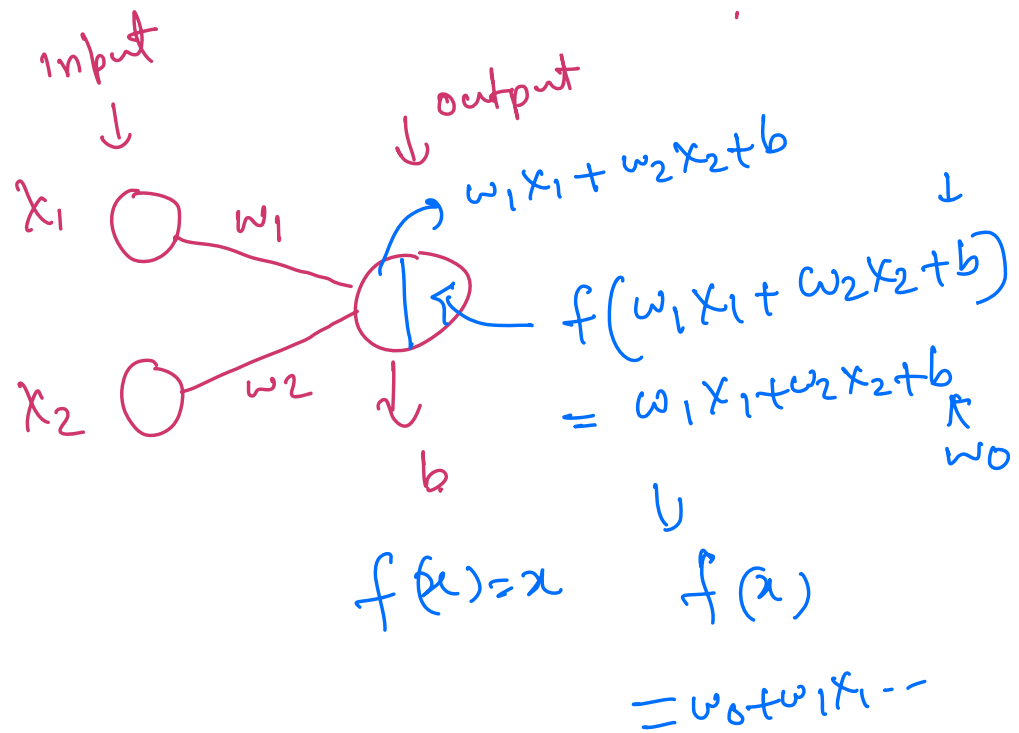
2.9 Some key Takeaway

2.10 Motivation for Deep Neural Network (Deep Learning)

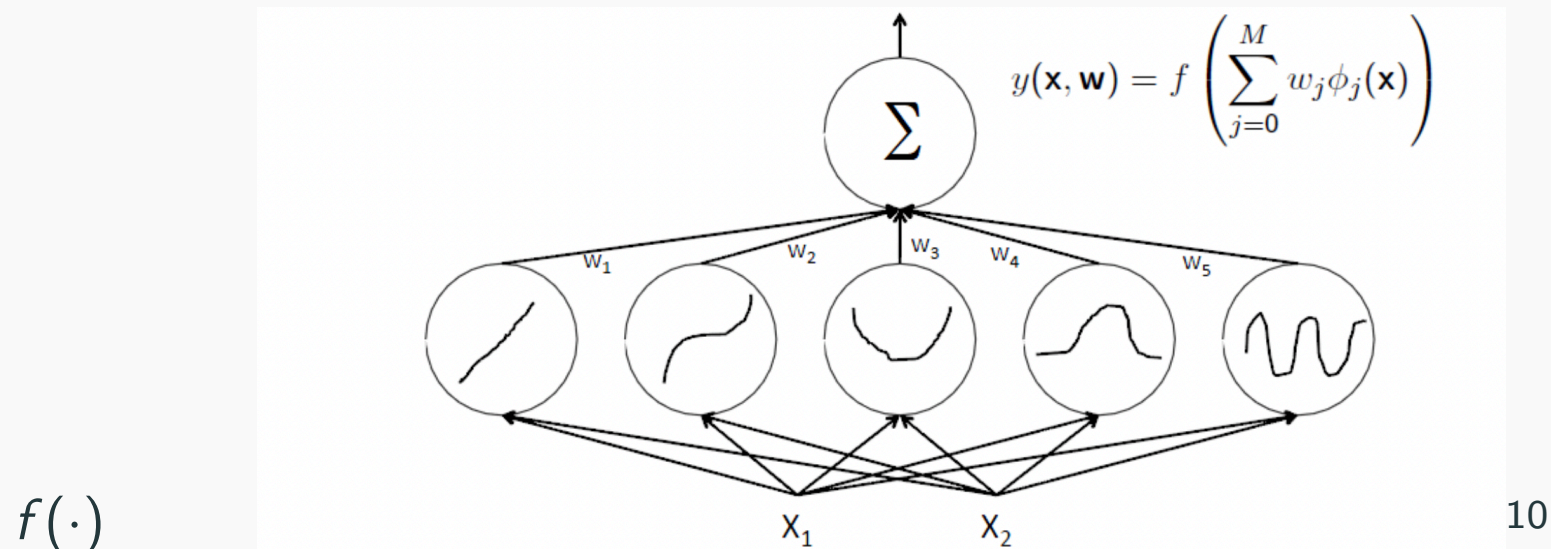
Linear Regression as Neural Networks



- The identity function in the case of regression.
- A nonlinear activation function in the case of classification.



Linear Regression as Neural Networks



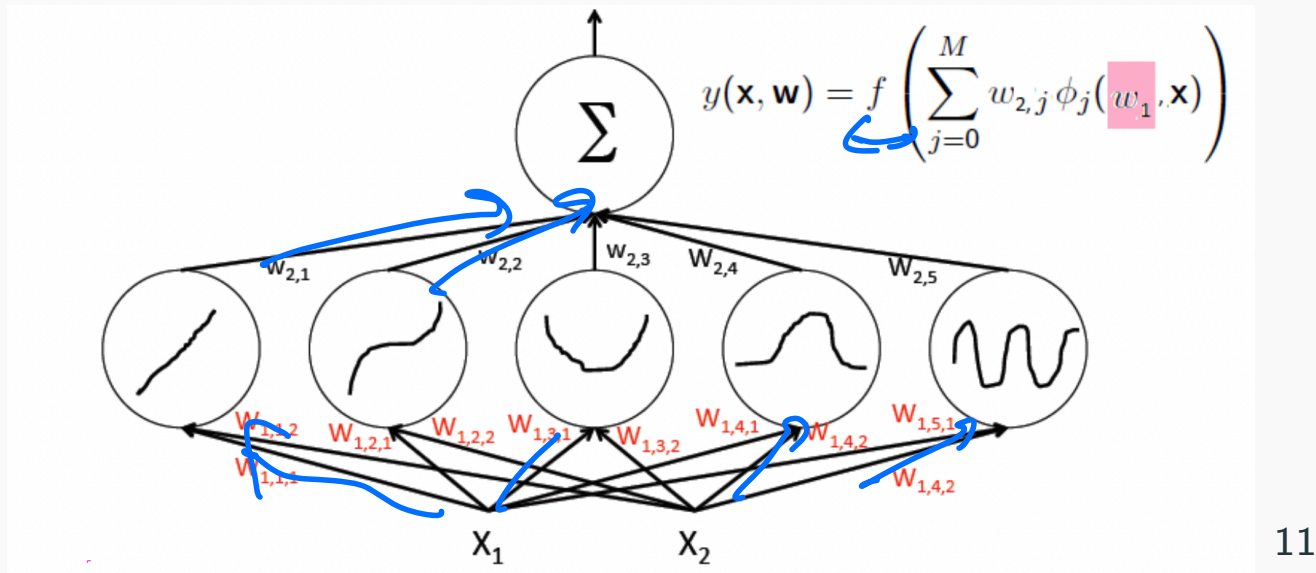
- The identity function in the case of regression.
- A nonlinear activation function in the case of classification.

Learning object:

- Learn $\mathbf{w}$'s so the function fits the data well.
- The basis functions are fixed functions of X .

¹⁰CS 229: Prof. Xiangliang Zhang (KAUST)

Non Linear Regression as Neural Networks



- Learning object: Learn $\mathbf{w}$ so the function fits the data well
- The basis functions are fixed functions of X and $\mathbf{w}_1$

Agenda

2. Neural Networks

2.1 Intro to Neural Networks

2.2 Components of Neural Networks

2.3 Standard Linear and Non-Linear Regressions as Neural Networks

2.4 Unfolding a Two-Layer Neural Network

2.5 Training of NNs (Learning the Weights)

2.6 Some Important Strategies/Considerations When Training a NN

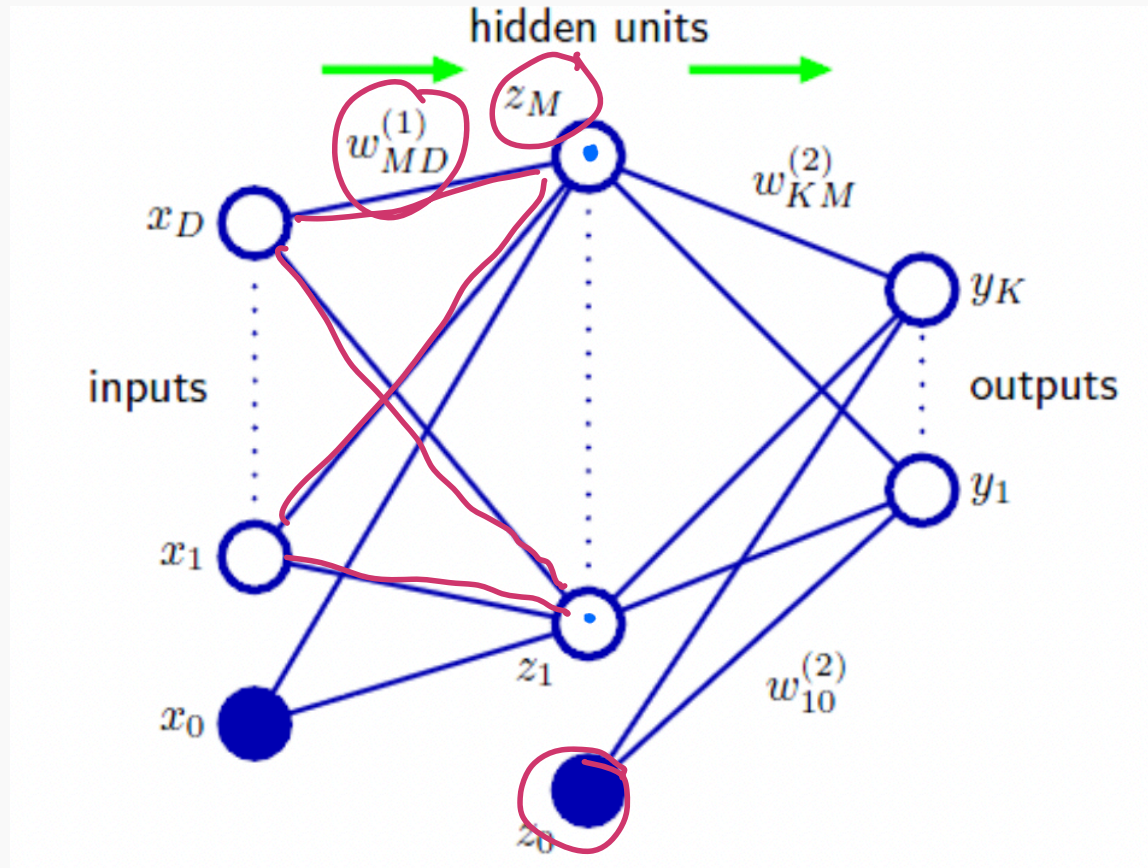
2.7 Building Your Very First Neural Network From Scratch

2.8 Fitting the Neural Network using Tensorflow

2.9 Some key Takeaway

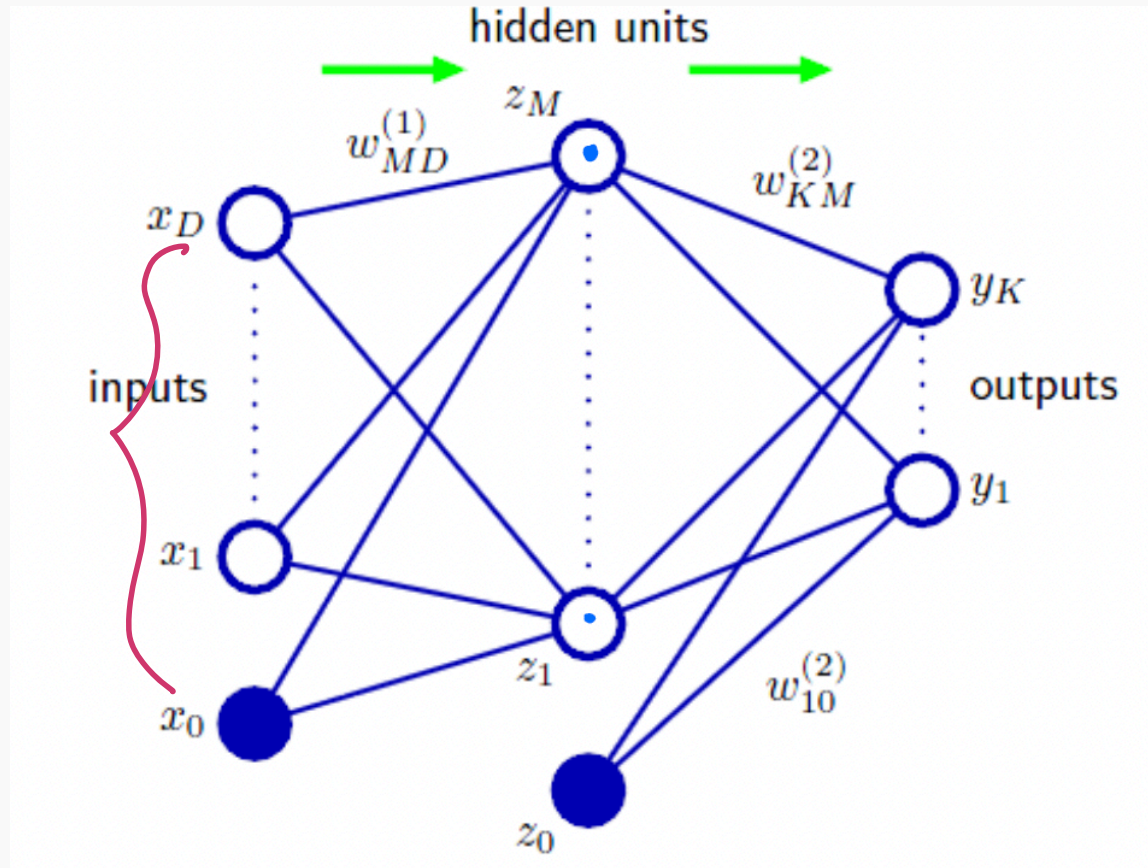
2.10 Motivation for Deep Neural Network (Deep Learning)

A Two-Layer Neural Network (1)



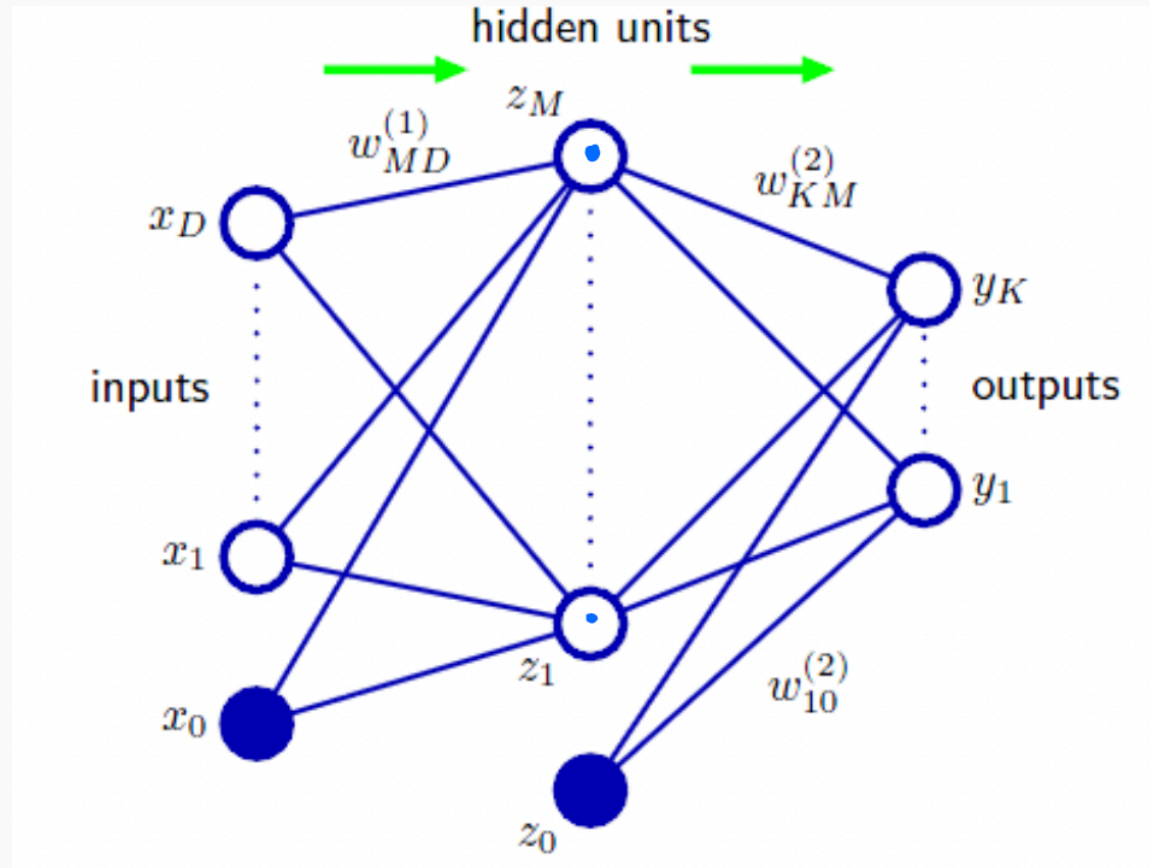
- D input variables

A Two-Layer Neural Network (1)



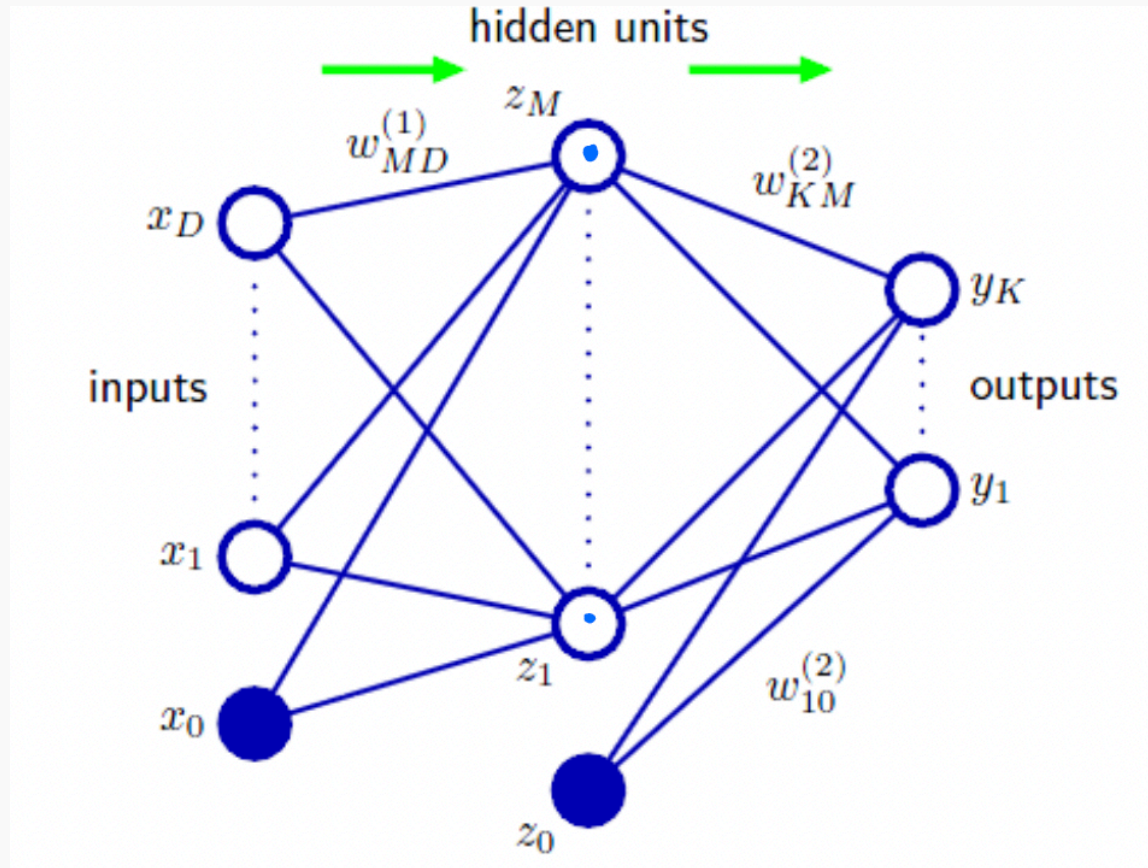
- D input variables
- M hidden neurons

A Two-Layer Neural Network (1)



- D input variables
- M hidden neurons
- K outputs

A Two-Layer Neural Network (1)



- D input variables
- M hidden neurons
- K outputs
- Use the two activation function on weights sum on hidden layers and output layers

A Two-Layer Neural Network (2)

Construct M linear combinations of the inputs $x_1, \dots, x_d$

$$a_j = \sum_{i=0}^D w_{ji}^{(1)} x_i,$$

$x_0 = 1.$

$w_{j0} + \dots$

- a_j are the activations, $j = 1, \dots, M$.

A Two-Layer Neural Network (2)

Construct M linear combinations of the inputs $x_1, \dots, x_d$

$$a_j = \sum_{i=0}^D w_{ji}^{(1)} x_i, \quad x_0 = 1.$$

- a_j are the activations, $j = 1, \dots, M$.
- $w_{ji}^{(1)}$ are the layer one weights, $i = 1, \dots, D$.

A Two-Layer Neural Network (2)

Construct M linear combinations of the inputs $x_1, \dots, x_d$

$$a_j = \sum_{i=0}^D w_{ji}^{(1)} x_i, \quad x_0 = 1.$$

- a_j are the activations, $j = 1, \dots, M$.
- $w_{ji}^{(1)}$ are the layer one weights, $i = 1, \dots, D$.
- $w_{j0}^{(1)}$ are the layer one biases.

A Two-Layer Neural Network (2)

Construct M linear combinations of the inputs $x_1, \dots, x_d$

$$a_j = \sum_{i=0}^D w_{ji}^{(1)} x_i, \quad x_0 = 1.$$

- a_j are the activations, $j = 1, \dots, M$.
- $w_{ji}^{(1)}$ are the layer one weights, $i = 1, \dots, D$.
- $w_{j0}^{(1)}$ are the layer one biases.

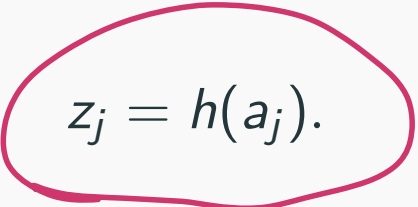
A Two-Layer Neural Network (2)

Construct M linear combinations of the inputs $x_1, \dots, x_d$

$$a_j = \sum_{i=0}^D w_{ji}^{(1)} x_i, \quad x_0 = 1.$$

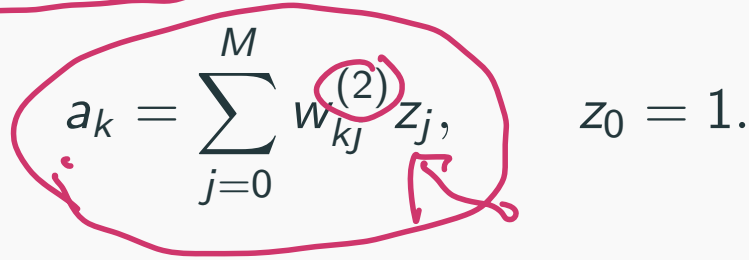
- a_j are the activations, $j = 1, \dots, M$.
- $w_{ji}^{(1)}$ are the layer one weights, $i = 1, \dots, D$.
- $w_{j0}^{(1)}$ are the layer one biases.

Each linear combination a_j is transformed by a (non-linear differentiable) activation function


$$z_j = h(a_j).$$

A Two-Layer Neural Network (3)

The hidden output $z_j = h(a_j)$ are linearly combined in layer two:



The equation $a_k = \sum_{j=0}^M w_{kj}^{(2)} z_j$ is shown with handwritten annotations. A pink oval encircles the entire equation. A pink arrow points from the text 'linearly combined' in the preceding sentence to the summation symbol. Another pink arrow points from the text 'layer two' to the superscript (2) in the weight term $w_{kj}^{(2)}$. The bias term $z_0 = 1$ is written to the right of the equation.

$$a_k = \sum_{j=0}^M w_{kj}^{(2)} z_j, \quad z_0 = 1.$$

- a_k are the output activations, $k = 1, \dots, K$.
- $w_{kj}^{(2)}$ are the layer two weights, $j = 1, \dots, M$.
- $w_{k0}^{(2)}$ are the layer one biases.

A Two-Layer Neural Network (3)

The hidden output $z_j = h(a_j)$ are linearly combined in layer two:

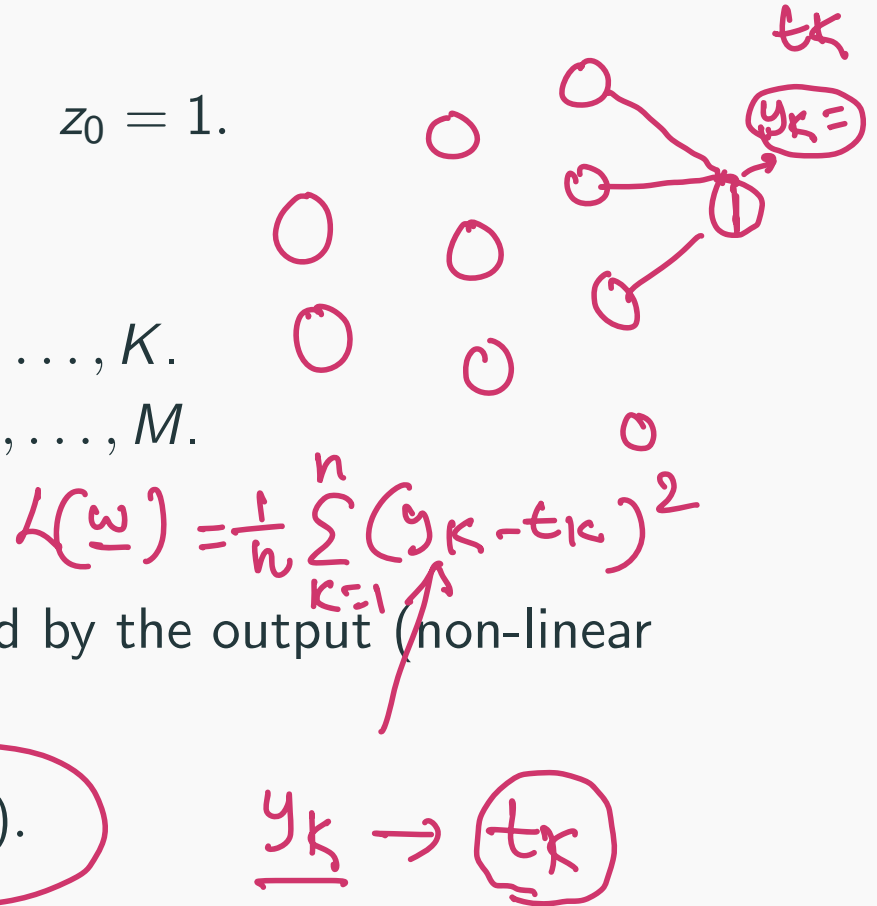
$$a_k = \sum_{j=0}^M w_{kj}^{(2)} z_j \quad z_0 = 1.$$

- a_k are the output activations, $k = 1, \dots, K$.
- $w_{kj}^{(2)}$ are the layer two weights, $j = 1, \dots, M$.
- $w_{k0}^{(2)}$ are the layer one biases.

The output activations a_k are transformed by the output (non-linear differentiable) activation function

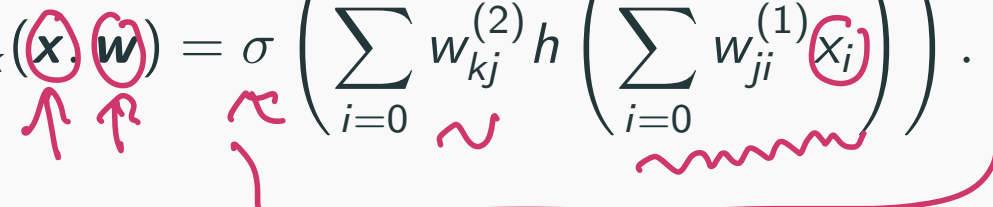
$$y_k = \sigma(a_k).$$

- y_k are the final outputs.
- $\sigma(\cdot)$ is like $h(\cdot)$, but often sigmoid function for classification and linear function for Regressions.



A Two-Layer Neural Network (4)

After substituting $y_k = \sigma(a_k)$ the definitions of a_j and a_k :

$$y_k(\mathbf{x}, \mathbf{w}) = \sigma \left(\sum_{i=0}^M w_{kj}^{(2)} h \left(\sum_{i=0}^D w_{ji}^{(1)} x_i \right) \right).$$


A Two-Layer Neural Network (4)

After substituting $y_k = \sigma(a_k)$ the definitions of a_j and a_k :

$$y_k(\mathbf{x}, \mathbf{w}) = \sigma \left(\sum_{i=0}^M w_{kj}^{(2)} h \left(\sum_{i=0}^D w_{ji}^{(1)} x_i \right) \right)$$

Handwritten notes: $L(\omega) = \frac{1}{2} \sum_{i=1}^n (\underbrace{y_k(x_i, \omega)}_{\sigma(a_k)} - \underbrace{t_i}_{\text{target}})^2$

Diagram illustrating forward propagation: $x_1 \rightarrow y_k(x_1, \omega)$, $x_2 \rightarrow y_k(x_2, \omega)$, ..., $x_n \rightarrow y_k(x_n, \omega)$

Evaluation of this is called **forward propagation**.

- $h(\cdot)$ and $\sigma(\cdot)$ are a sigmoid functions, e.g., the logistics function.

$$s(a) = \frac{1}{1 + \exp(-a)}, \quad s(a) \in [0, 1].$$

- If $\sigma(a)$ is the identity, then a regression model is obtained.

Agenda

2. Neural Networks

- 2.1 Intro to Neural Networks ✓
- 2.2 Components of Neural Networks ✓ ✓
- 2.3 Standard Linear and Non-Linear Regressions as Neural Networks ✓)
- 2.4 Unfolding a Two-Layer Neural Network)
- 2.5 Training of NNs (Learning the Weights) ✓
- 2.6 Some Important Strategies/Considerations When Training a NN
- 2.7 Building Your Very First Neural Network From Scratch
- 2.8 Fitting the Neural Network using Tensorflow
- 2.9 Some key Takeaway
- 2.10 Motivation for Deep Neural Network (Deep Learning)

Recall: Error (Loss) Function

Training a neural network involves estimating the weights and biases by optimizing a loss function.

Recall: Error (Loss) Function

Training a neural network involves estimating the weights and biases by optimizing a loss function.

Let N represent the number of observations of the response $t_1, t_2, \dots, t_N$, and let the neural network predict some output $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_N$ (which could be a vector).

Recall: Error (Loss) Function

Training a neural network involves estimating the weights and biases by optimizing a loss function.

Let N represent the number of observations of the response $t_1, t_2, \dots, t_N$, and let the neural network predict some output $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_N$ (which could be a vector).

- Regression: **Sum of squared**

$$\mathcal{L}(\mathbf{w}) \text{ or } E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(\mathbf{x}_n, \mathbf{w}) - t_n\}^2,$$

where $t_n, n = 1, \dots, N$ is the observed response. N is the total number of training samples (examples).

Recall: Error (Loss) Function

Training a neural network involves estimating the weights and biases by optimizing a loss function.

Let N represent the number of observations of the response $t_1, t_2, \dots, t_N$, and let the neural network predict some output $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_N$ (which could be a vector).

- Regression: Sum of squared

$$\mathcal{L}(\mathbf{w}) \text{ or } E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(\mathbf{x}_n, \mathbf{w}) - t_n\}^2,$$

where $t_n, n = 1, \dots, N$ is the observed response. N is the total number of training samples (examples).

- Binary classification: NNs with only one output node whose activation function is a logistic sigmoid Cross-entropy error function

$$\mathcal{L}(\mathbf{w}) \text{ or } E(\mathbf{w}) = -\frac{1}{2} \sum_{n=1}^N \{t_n \log y(\mathbf{x}_n, \mathbf{w}) + (1 - t_n) \log(1 - y(\mathbf{x}_n, \mathbf{w}))\}.$$

Training a Neural Network: Forward and Backward Pass

Each training step has two key phases:

- **Forward Pass**: Compute output predictions from input through the layers.

Training a Neural Network: Forward and Backward Pass

Each training step has two key phases:

- **Forward Pass**: Compute output predictions from input through the layers.
- **Backward Pass (Backpropagation)**: Compute gradients of loss w.r.t. weights using the chain rule.

Training a Neural Network: Forward and Backward Pass

$$L(\omega) \rightarrow \left(\frac{\partial L}{\partial \omega} \right)$$
$$\omega = \omega - \eta \frac{\partial L}{\partial \omega}$$

Each training step has two key phases:

- **Forward Pass**: Compute output predictions from input through the layers.
- **Backward Pass (Backpropagation)**: Compute gradients of loss w.r.t. weights using the chain rule.
- Update weights using gradient descent (or a variant).

Training a Neural Network: Forward and Backward Pass

Each training step has two key phases:

- **Forward Pass**: Compute output predictions from input through the layers.
- **Backward Pass (Backpropagation)**: Compute gradients of loss w.r.t. weights using the chain rule.
- Update weights using **gradient descent** (or a variant).

Training a Neural Network: Forward and Backward Pass

Each training step has two key phases:

- **Forward Pass**: Compute output predictions from input through the layers.
- **Backward Pass (Backpropagation)**: Compute gradients of loss w.r.t. weights using the chain rule.
- Update weights using **gradient descent** (or a variant).

Goal: Minimize the error/loss function $E(\mathbf{w})$

Forward Propagation: How the Network Makes Predictions

For each layer:

- Compute linear combination (pre-activation):

$$a_j = \sum_i \underbrace{w_{ji} z_i}_{\text{w}} + \underbrace{b_j}_{-}$$

- Apply activation function:

$$z_j = h(a_j)$$

- Repeat this through hidden layers until final output:

$$\hat{y} = f(\mathbf{x}; \mathbf{w}) = z_{\text{output}}$$

$\hat{y} \rightarrow t_y$

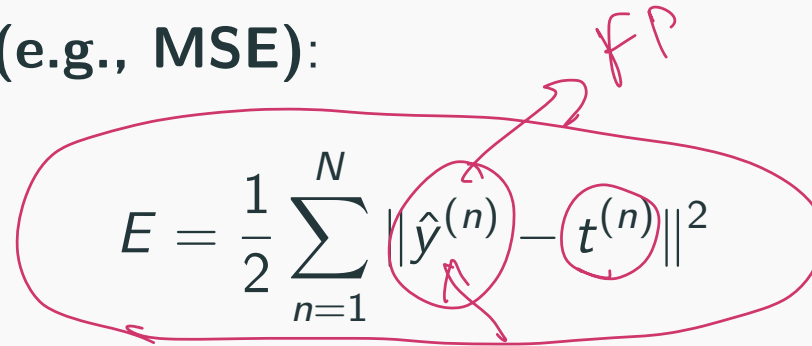
Note:

- z_i : activation from the previous layer (input if it's the first)
- w_{ji} : weight connecting unit i to unit j
- h : activation function (e.g., sigmoid, ReLU, tanh)

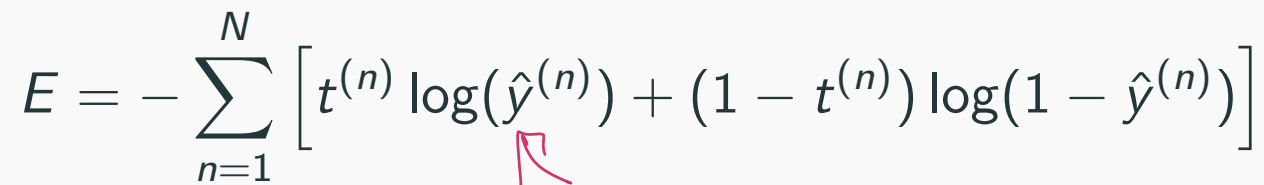
Error/Loss Function

At the end of the forward propagation, calculate the loss E , (same as $E(\mathbf{w})$ or $\mathcal{L}(\mathbf{w})$).

- For Regression (e.g., MSE):

$$E = \frac{1}{2} \sum_{n=1}^N \|\hat{y}^{(n)} - t^{(n)}\|^2$$


- For Binary Classification (Cross-Entropy):

$$E = - \sum_{n=1}^N \left[t^{(n)} \log(\hat{y}^{(n)}) + (1 - t^{(n)}) \log(1 - \hat{y}^{(n)}) \right]$$


- $t^{(n)}$: true target for sample n
 $\hat{y}^{(n)}$: predicted output from forward pass

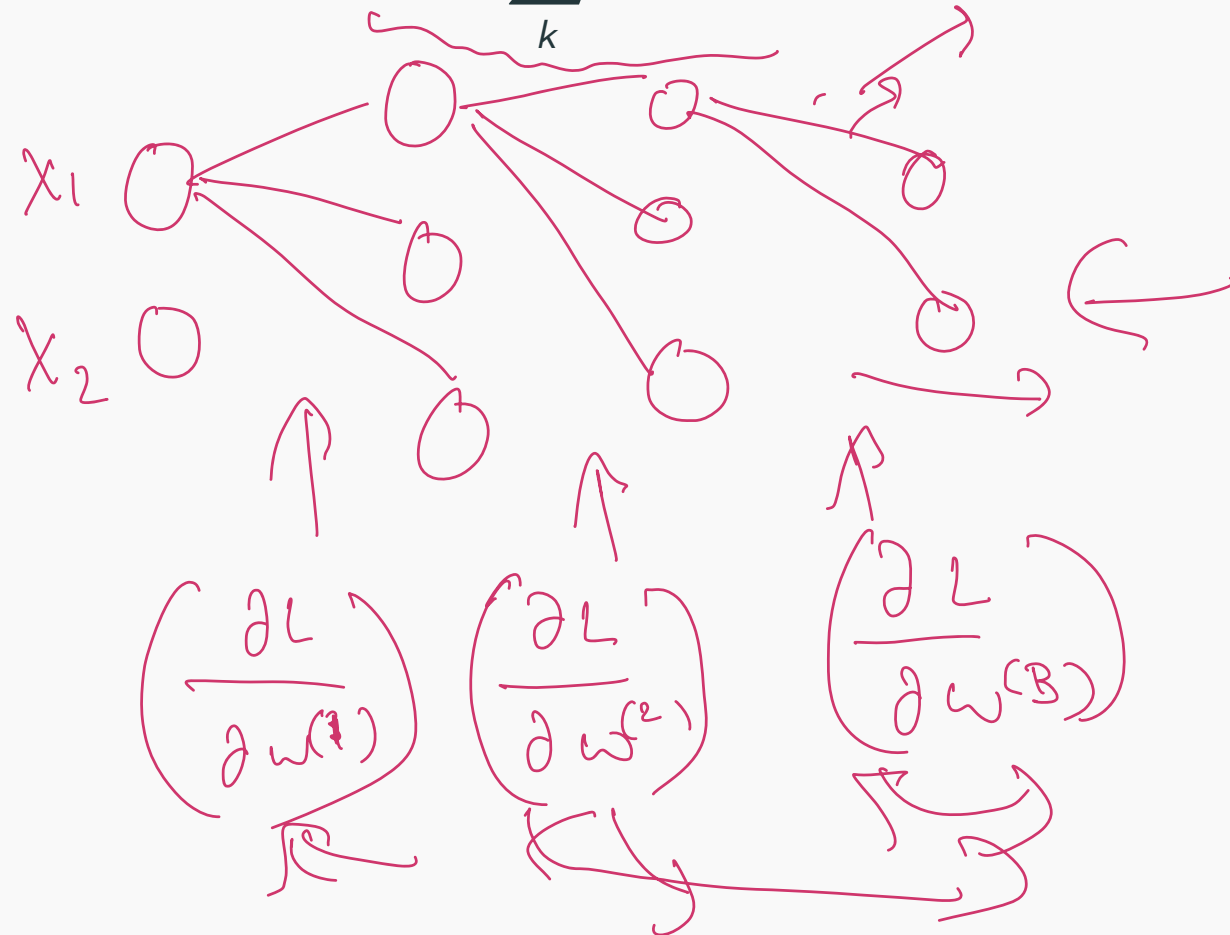
Backpropagation: Computing the Gradients

For output units:

$$\delta_k = \hat{y}_k - t_k$$

For hidden units:

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$



Backpropagation: Computing the Gradients

For output units:

$$\delta_k = \hat{y}_k - t_k$$

For hidden units:

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$

Summary of Backward Pass:

- Forward propagate input to compute hidden and output activations.

Backpropagation: Computing the Gradients

For output units:

$$\delta_k = \hat{y}_k - t_k$$

For hidden units:

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$

Summary of Backward Pass:

- Forward propagate input to compute hidden and output activations.
- Compute output error signal δ_k .

Backpropagation: Computing the Gradients

For output units:

$$\delta_k = \hat{y}_k - t_k$$

For hidden units:

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$

Summary of Backward Pass:

- Forward propagate input to compute hidden and output activations.
- Compute output error signal δ_k .
- Backpropagate errors to hidden layers to get δ_j .

Backpropagation: Computing the Gradients

For output units:

$$\delta_k = \hat{y}_k - t_k$$

For hidden units:

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$

Summary of Backward Pass:

- Forward propagate input to compute hidden and output activations.
- Compute output error signal δ_k .
- Backpropagate errors to hidden layers to get δ_j .
- Compute gradient of error w.r.t. weights:

$$\frac{\partial E}{\partial w_{ij}} = \delta_j z_i$$

Backpropagation: Computing the Gradients

For output units:

$$\delta_k = \hat{y}_k - t_k$$

For hidden units:

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$

Summary of Backward Pass:

- Forward propagate input to compute hidden and output activations.
- Compute output error signal δ_k .
- Backpropagate errors to hidden layers to get δ_j .
- Compute gradient of error w.r.t. weights:

$$\frac{\partial E}{\partial w_{ij}} = \delta_j z_i$$

- Accumulate gradients across samples.

Gradient Descent and Weight Updates

Weight Update Rule:

- Update weights using:

$$w_{ij} \leftarrow w_{ij} - \eta \cdot \frac{\partial E}{\partial w_{ij}^m}$$

$\eta = 1050$

- η : **learning rate**, a small positive number that controls the step size.
- Repeat for multiple **epochs** (complete passes over the dataset) until convergence.

Gradient Descent and Weight Updates

Weight Update Rule:

- Update weights using:

$$\underline{w_{ij}} \leftarrow \underline{w_{ij}} - \eta \cdot \left(\frac{\partial E}{\partial w_{ij}} \right)$$

$$E = \frac{1}{n} \sum ()$$

→ ()

- η : **learning rate**, a small positive number that controls the step size.
- Repeat for multiple **epochs** (complete passes over the dataset) until convergence.

Variants of Gradient Descent:

- Stochastic Gradient Descent (SGD)**: Updates weights using **one example (data) at a time**. Fast but noisy, often helps escape local minima.

$$h=1000 \rightarrow$$
$$b=10$$

$$X = \begin{bmatrix} 1 & x_{11} & \dots & x_{p1} \\ 1 & x_{12} & & 1 \\ \vdots & \vdots & & \vdots \\ 1 & x_{1n} & & x_{pn} \end{bmatrix}$$

→ (n)

Gradient Descent and Weight Updates

Weight Update Rule:

- Update weights using:

$$w_{ij} \leftarrow w_{ij} - \eta \cdot \frac{\partial E}{\partial w_{ij}}$$

- η : **learning rate**, a small positive number that controls the step size.
- Repeat for multiple **epochs** (complete passes over the dataset) until convergence.

Variants of Gradient Descent:

- **Stochastic Gradient Descent (SGD)**: Updates weights using **one example (data) at a time**. Fast but noisy, often helps escape local minima.
- **Mini-batch Gradient Descent**: Uses a **small batch of samples** (e.g., 32 or 64). Balances efficiency and stability.

Gradient Descent and Weight Updates

Weight Update Rule:

- Update weights using:

$$w_{ij} \leftarrow w_{ij} - \overset{h_i's}{\eta} \cdot \frac{\partial E}{\partial w_{ij}}$$

- η : **learning rate**, a small positive number that controls the step size.
- Repeat for multiple **epochs** (complete passes over the dataset) until convergence.

Variants of Gradient Descent:

- Stochastic Gradient Descent (SGD)**: Updates weights using **one example (data) at a time**. Fast but noisy, often helps escape local minima.
- Mini-batch Gradient Descent**: Uses a **small batch of samples** (e.g., 32 or 64). Balances efficiency and stability.
- Adaptive Optimizers (Adam, RMSprop)**: Adjust learning rates **dynamically per parameter**. Often converges faster with minimal tuning.

Agenda

2. Neural Networks

2.1 Intro to Neural Networks

2.2 Components of Neural Networks

2.3 Standard Linear and Non-Linear Regressions as Neural Networks

2.4 Unfolding a Two-Layer Neural Network

2.5 Training of NNs (Learning the Weights) }

2.6 Some Important Strategies/Considerations When Training a NN

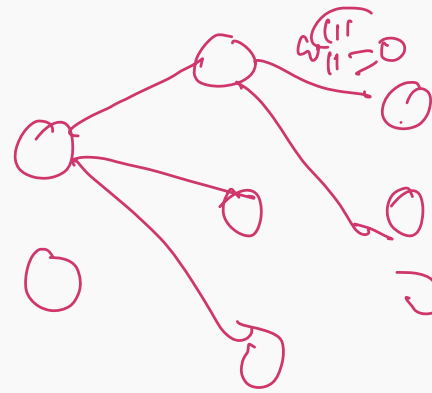
2.7 Building Your Very First Neural Network From Scratch

2.8 Fitting the Neural Network using Tensorflow

2.9 Some key Takeaway

2.10 Motivation for Deep Neural Network (Deep Learning)

Useful Strategies for Training



- **Early Stopping:**
 - Stop training when validation loss stops improving.
- **Dropout Layers:**
 - Randomly deactivate neurons to prevent overfitting.
- **Learning Rate Scheduling:**
 - Decrease learning rate as training progresses.
- **Batch Normalization:**
 - Normalizes activations and accelerates training.

Normalizing Input Data: Why and How?

- What is Normalization?

Normalizing Input Data: Why and How?

- **What is Normalization?**

- Rescaling input features to a standard range, e.g., $[0, 1]$ or $\mathcal{N}(0, 1)$.

$$\begin{aligned} & (-10, 0) + x_1 \odot \\ & \quad x_2 \odot \\ & \quad \rightarrow (10, 200) \end{aligned}$$

Normalizing Input Data: Why and How?

- **What is Normalization?**

- Rescaling input features to a standard range, e.g., $[0, 1]$ or $\mathcal{N}(0, 1)$.

- **Why is it Important for Neural Networks?**

- Ensures faster and more stable training.
- Prevents activation saturation (e.g., in sigmoid or tanh).
- Keeps gradients in a similar scale across layers → reduces risk of exploding/vanishing gradients.

Normalizing Input Data: Why and How?

- **What is Normalization?**

- Rescaling input features to a standard range, e.g., $[0, 1]$ or $\mathcal{N}(0, 1)$.

- **Why is it Important for Neural Networks?**

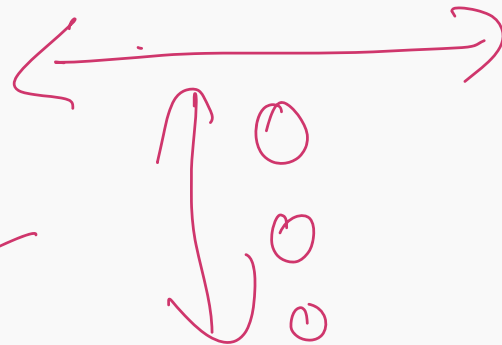
- Ensures faster and more stable training.
- Prevents activation saturation (e.g., in sigmoid or tanh).
- Keeps gradients in a similar scale across layers → reduces risk of exploding/vanishing gradients.

- **How is it Done?**

- Standardization: $x' = \frac{x - \mu}{\sigma}$
- Min-Max Scaling: $x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$

What Are Hyperparameters in NNs?

- **Hyperparameters** are settings used to control the learning process in neural networks.
- Unlike model parameters (e.g., weights and biases), they are not learned from data.
- They are set *before* training begins and can significantly affect performance.
- Common examples include:
 - Learning rate (η)
 - Number of layers & units per layer
 - Batch size
 - Number of training epochs
 - Dropout rate, Weight decay, etc.



Key Hyperparameters and Their Role

- **Learning Rate (η):**

- Controls step size during weight updates.
- Too high $\rightarrow$ **Divergence**, too low $\rightarrow$ **Slow convergence**.

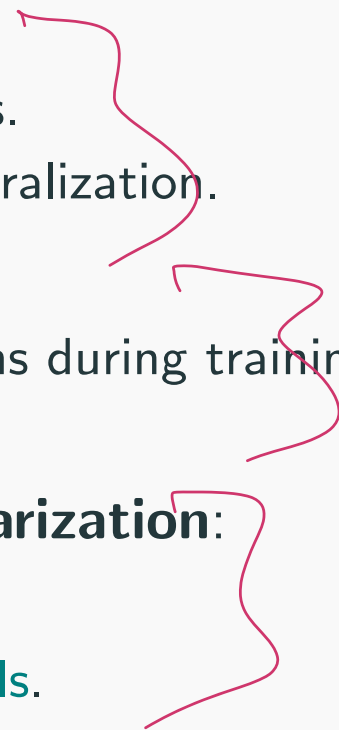
- **Batch Size:**

- Number of samples used in one update.
- Smaller $\rightarrow$ **noisy but faster**; Larger $\rightarrow$ **stable but slower**.

- **Epochs:**

- One pass over the entire dataset.
- **Too many** $\rightarrow$ Overfitting; **Too few** $\rightarrow$ Underfitting.

Other Useful Hyperparameters

- **Network Architecture:**
 - Number and size of layers.
 - Affects capacity and generalization.
 - **Dropout Rate:**
 - Randomly disables neurons during training.
 - Helps prevent **overfitting**.
 - **Weight Decay / L_2 Regularization:**
 - Penalizes large weights.
 - Encourages **simpler models**.
- 

Choosing the Best Model

- Train multiple models with different:
 - Architectures (e.g., number of layers, units)
 - Hyperparameters (learning rate, dropout, etc.)
- Use the validation set to select the best model — the one with:
 - Low validation loss
 - Good generalization (small gap between train and validation loss)
- Test set should only be used once — after selection.

Common Pitfalls to Avoid

- Using test set during training → leads to data leakage.
- Evaluating on only one split → consider using **cross-validation**.
- Over-tuning hyperparameters → may overfit to the validation set.
- Ignoring model complexity → simpler models can generalize better.
- Not tracking experiments → use tools like Weights & Biases, TensorBoard, or spreadsheets.

Agenda

2. Neural Networks

2.1 Intro to Neural Networks

2.2 Components of Neural Networks

2.3 Standard Linear and Non-Linear Regressions as Neural Networks

2.4 Unfolding a Two-Layer Neural Network

2.5 Training of NNs (Learning the Weights)

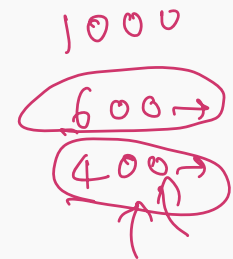
2.6 Some Important Strategies/Considerations When Training a NN

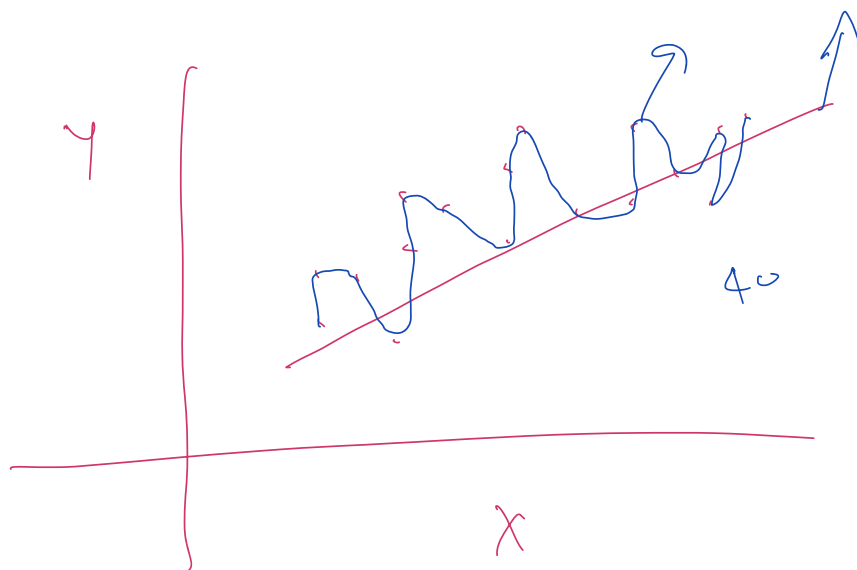
2.7 Building Your Very First Neural Network From Scratch

2.8 Fitting the Neural Network using Tensorflow

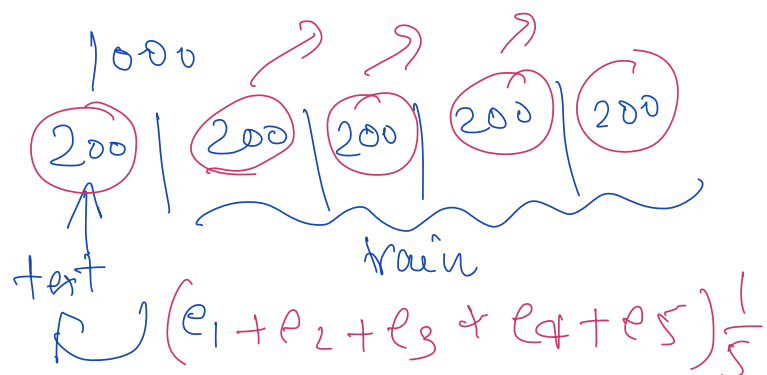
2.9 Some key Takeaway

2.10 Motivation for Deep Neural Network (Deep Learning)

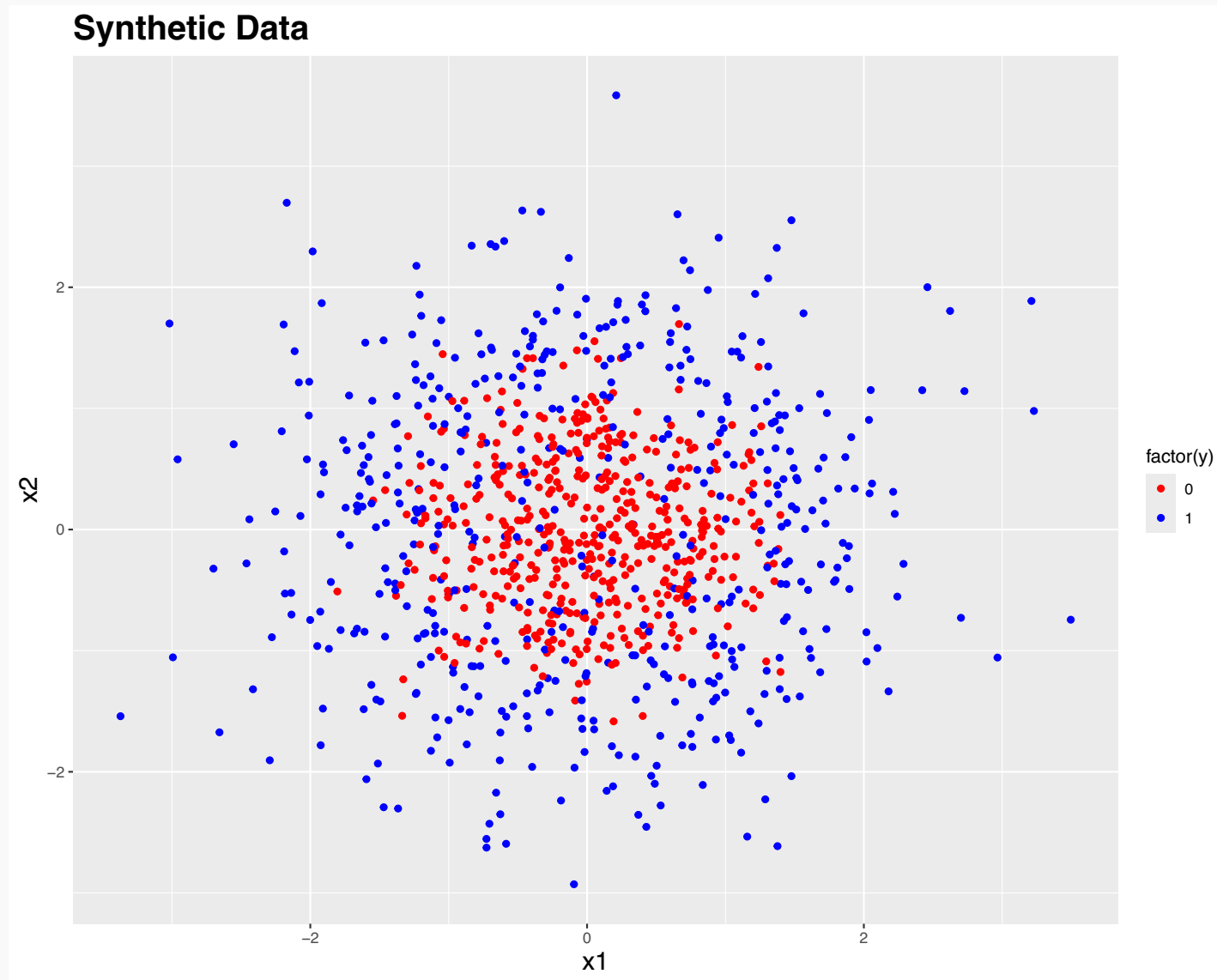




K-fold

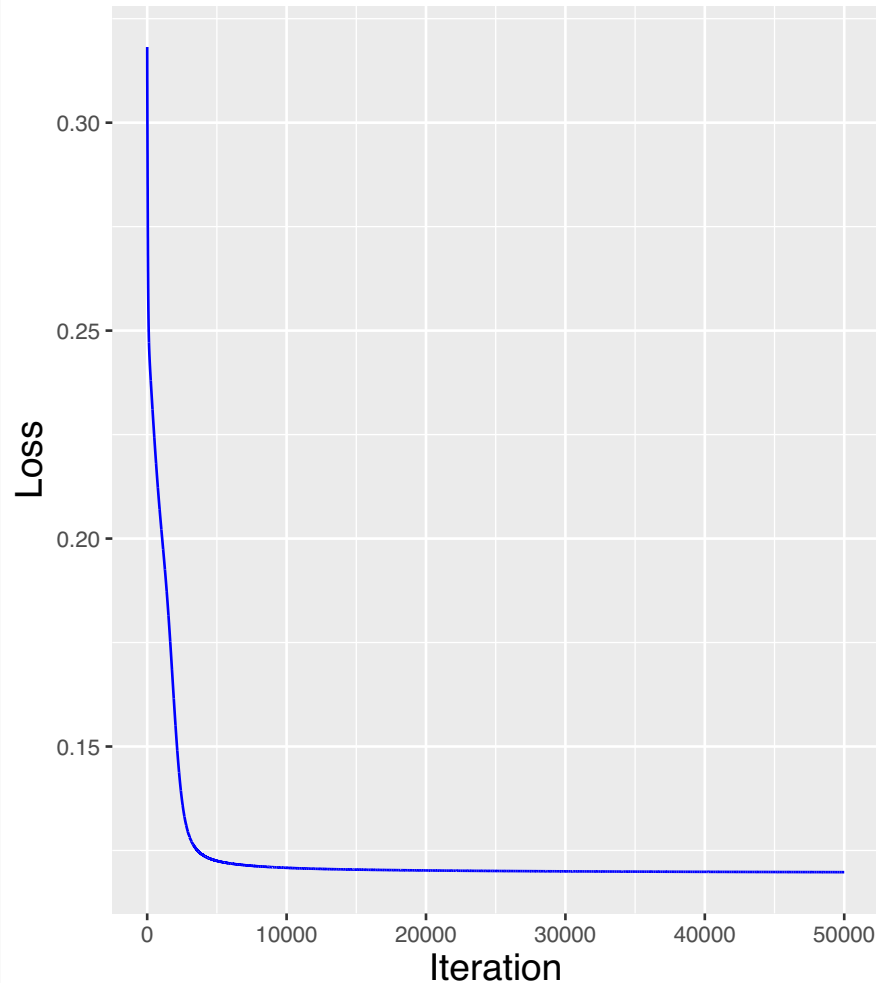


Lab: Classification Using two-Layer Neural Networks (1)

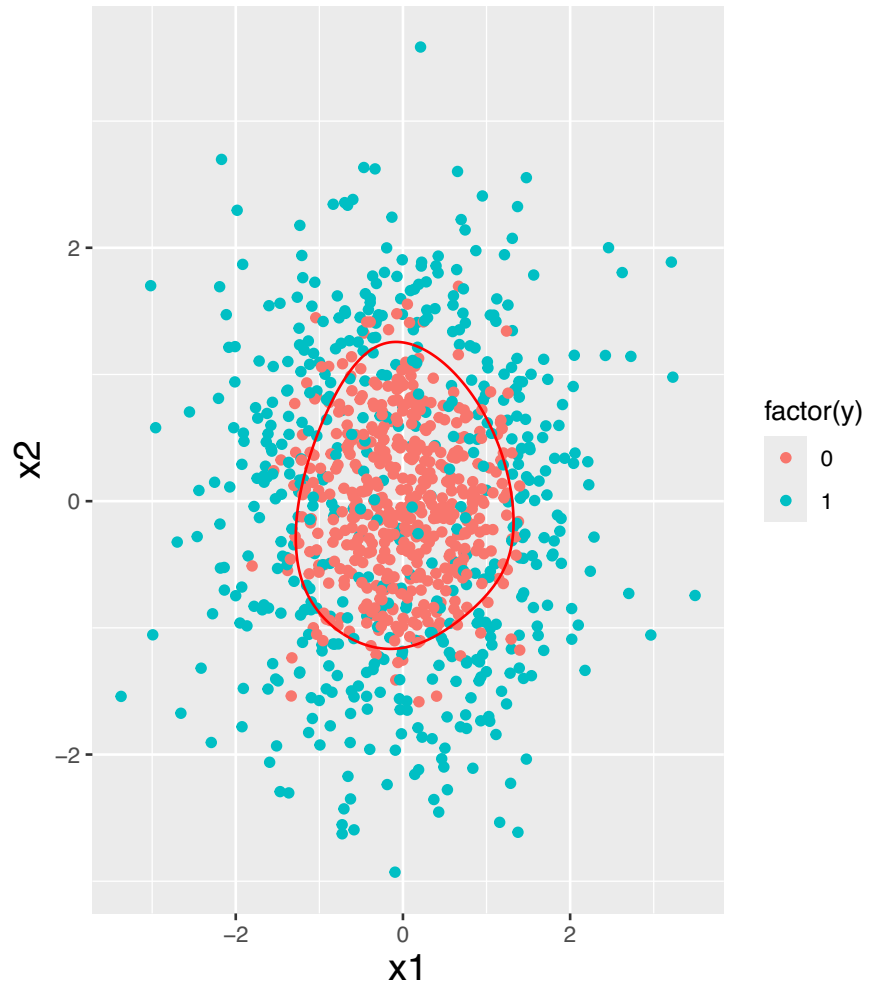


Lab: Classification Using two-Layer Neural Networks (2)

Loss Function Over Iterations



Decision Boundary of the Neural Network



12

¹²See the python code in the GitHub Link:

https://github.com/yadavrishikesh/Masterclass-IITMandi/blob/main/Module3/code/NNs_for_classification_from_scratch.ipynb

Agenda

2. Neural Networks

2.1 Intro to Neural Networks

2.2 Components of Neural Networks

2.3 Standard Linear and Non-Linear Regressions as Neural Networks

2.4 Unfolding a Two-Layer Neural Network

2.5 Training of NNs (Learning the Weights)

2.6 Some Important Strategies/Considerations When Training a NN

2.7 Building Your Very First Neural Network From Scratch

2.8 Fitting the Neural Network using Tensorflow

2.9 Some key Takeaway

2.10 Motivation for Deep Neural Network (Deep Learning)

What is TensorFlow

TensorFlow is an open-source machine learning library developed by Google. It provides a comprehensive, flexible ecosystem of tools, libraries, and community resources that lets researchers push the state-of-the-art in ML, and developers easily build and deploy ML-powered applications.

What is TensorFlow

TensorFlow is an open-source machine learning library developed by Google. It provides a comprehensive, flexible ecosystem of tools, libraries, and community resources that lets researchers push the state-of-the-art in ML, and developers easily build and deploy ML-powered applications.

- Originally developed by the Google Brain Team

What is TensorFlow

TensorFlow is an open-source machine learning library developed by Google. It provides a comprehensive, flexible ecosystem of tools, libraries, and community resources that lets researchers push the state-of-the-art in ML, and developers easily build and deploy ML-powered applications.

- Originally developed by the Google Brain Team
- Supports deep learning, numerical computation, and automatic differentiation

What is TensorFlow

TensorFlow is an open-source machine learning library developed by Google. It provides a comprehensive, flexible ecosystem of tools, libraries, and community resources that lets researchers push the state-of-the-art in ML, and developers easily build and deploy ML-powered applications.

- Originally developed by the Google Brain Team
- Supports deep learning, numerical computation, and automatic differentiation
- Works across multiple platforms: CPUs, GPUs, TPUs

Typical steps in building a model with TensorFlow:

1. Prepare the data

Typical steps in building a model with TensorFlow:

1. Prepare the data
2. Build the model (using `keras.Sequential`)

Typical steps in building a model with TensorFlow:

1. **Prepare** the data
2. **Build** the model (using `keras.Sequential`)
3. **Compile** the model (using `model.compile`) and mention about the loss, optimizer, and metrics

TensorFlow Workflow

Typical steps in building a model with TensorFlow:

1. **Prepare** the data
2. **Build** the model (using `keras.Sequential`)
3. **Compile** the model (using `model.compile`) and mention about the loss, optimizer, and metrics
4. **Train** the model (`model.fit`)

TensorFlow Workflow

Typical steps in building a model with TensorFlow:

1. **Prepare** the data
2. **Build** the model (using `keras.Sequential`)
3. **Compile** the model (using `model.compile`) and mention about the loss, optimizer, and metrics
4. **Train** the model (`model.fit`)
5. **Evaluate** the model (`model.evaluate`)

TensorFlow Workflow

Typical steps in building a model with TensorFlow:

1. **Prepare** the data
2. **Build** the model (using `keras.Sequential`)
3. **Compile** the model (using `model.compile`) and mention about the loss, optimizer, and metrics
4. **Train** the model (`model.fit`)
5. **Evaluate** the model (`model.evaluate`)
6. **Predict** with new data (`model.predict`)

Why Use TensorFlow?

- **Scalability:** Efficient execution across devices and distributed systems
- **Production Ready:** Tools for deploying models to web, mobile, and edge devices
- **Ecosystem:** Strong support for [visualization](#), [data pipelines](#), and [deployment](#)
- **Community:** Backed by Google and widely adopted in academia and industry

[TensorFlow](#) is designed to support research prototyping and production deployment equally well.

Key Steps in Fitting a Neural Network (TensorFlow)

- **Step 1: Load and Prepare Data**
 - Use `tensorflow.keras.datasets` or custom datasets.
 - Normalize/scale input features.
- **Step 2: Define the Model Architecture**
 - Use `keras.Sequential` or subclassing API.
 - Stack layers using `Dense`, `Dropout`, etc.
- **Step 3: Compile the Model**
 - Specify `loss function`, `optimizer`, and `metrics`.

Model Definition Example (TensorFlow)

- A simple feedforward network for binary classification:

```
1 model = keras.Sequential([  
2     keras.layers.input(shape=(num_features,)),  
3     keras.layers.Dense(8, activation='relu'),  
4     keras.layers.Dense(10, activation='relu'),  
5     ...  
6     keras.layers.Dense(1, activation='sigmoid')  
7 ])
```

Model Definition Example (TensorFlow)

- A simple feedforward network for binary classification:

```
1 model = keras.Sequential([  
2     keras.layers.input(shape=(num_features,)),  
3     keras.layers.Dense(8, activation='relu'),  
4     keras.layers.Dense(10, activation='relu'),  
5     ...  
6     keras.layers.Dense(1, activation='sigmoid')  
7 ])
```

- `input_shape`: Number of input features.
- `activation='relu'`: Adds non-linearity.
- `activation='sigmoid'`: For binary classification.

Compiling the Model

```
1 model.compile(  
2     optimizer='adam',  
3     loss='binary_crossentropy',  
4     metrics=['accuracy']  
5 )
```

Compiling the Model

```
1 model.compile(  
2     optimizer='adam',  
3     loss='binary_crossentropy',  
4     metrics=['accuracy']  
5 )
```

- `optimizer`: Controls weight updates (e.g., `adam`, `sgd`).
- `loss`: Chosen based on task:
 - `binary_crossentropy` for binary classification
 - `categorical_crossentropy` for multi-class
 - `mse` for regression
- `metrics`: To monitor model performance during training.

Training and Evaluation

```
1 history = model.fit(  
2     X_train, y_train,  
3     validation_data=(X_val, y_val),  
4     epochs=50,  
5     batch_size=32  
6 )
```

Training and Evaluation

```
1 history = model.fit(  
2     X_train, y_train,  
3     validation_data=(X_val, y_val),  
4     epochs=50,  
5     batch_size=32  
6 )
```

- validation_data: Used to monitor generalization.
- epochs: Number of training passes over data.
- batch_size: Mini-batch size.

```
1 test_loss, test_acc = model.evaluate(X_test, y_test)
```


Lab: Fitting the Neural Network using Tensorflow

See the GitHub Link:

https://colab.research.google.com/github/yadavrishikesh/Masterclass-IITMandi/blob/main/Module3/code/NNs_for_classification_using_tensorflow_pac

Agenda

2. Neural Networks

2.1 Intro to Neural Networks

2.2 Components of Neural Networks

2.3 Standard Linear and Non-Linear Regressions as Neural Networks

2.4 Unfolding a Two-Layer Neural Network

2.5 Training of NNs (Learning the Weights)

2.6 Some Important Strategies/Considerations When Training a NN

2.7 Building Your Very First Neural Network From Scratch

2.8 Fitting the Neural Network using Tensorflow

2.9 Some key Takeaway

2.10 Motivation for Deep Neural Network (Deep Learning)

A Nice Visual of NNs

Recall: and let's have some fun: <https://playground.tensorflow.org/>

Some key Takeaway: Learning Issue (1)

- Vanishing gradient problems due to propagated error coming from the other neurons and layers
 - Use of smart activation functions solve this issue

Some key Takeaway: Learning Issue (1)

- Vanishing gradient problems due to propagated error coming from the other neurons and layers
 - Use of smart activation functions solve this issue
- Overfitting due to a lot number of hidden units the number of hidden units determines the complexity of the learned function (the number of parameters $\mathbf{w}$)
 - We may allow regularizations
 - Drop of techniques
 - Sparse neural networks

Some key Takeaway (2): Practical Rule of Thumb on deciding hidden units

- There is no theoretical results that determines the minimum number of hidden units.
- Practical rule of thumb
 - For binary data $M = 2D$
 - For real valued data $M \gg 2D$
- Multiple hidden layers with fewer nodes maybe trained faster for similar quality in some applications

Some Important Comments About the Architecture of NNs

- Typically, K (Output) $< D$ (Input) $< M$ (Hidden).
 - If $M < D$ or $M < K$, information loss may occur at the hidden units.
- A multilayer network of linear units (where all $h(\cdot)$ are linear) is not interesting and can be reduced to a network without hidden units.
- There may be multiple hidden layers.
- Units need not be fully connected to the next layer.
- Links may skip layers.
- There may be symmetries in the weight space, meaning that different choices of w may define the same mapping from input to output.

Tips When Training Neural Networks

- **Always monitor** training and validation loss.
- Use **early stopping** to avoid overfitting.
- **Normalize or standardize** your input features.
- Use **learning rate schedules** or optimizers like Adam.
- Try small models first and increase complexity if needed.
- Perform **hyperparameter tuning** using grid search, random search, or Bayesian optimization.

Agenda

2. Neural Networks

2.1 Intro to Neural Networks

2.2 Components of Neural Networks

2.3 Standard Linear and Non-Linear Regressions as Neural Networks

2.4 Unfolding a Two-Layer Neural Network

2.5 Training of NNs (Learning the Weights)

2.6 Some Important Strategies/Considerations When Training a NN

2.7 Building Your Very First Neural Network From Scratch

2.8 Fitting the Neural Network using Tensorflow

2.9 Some key Takeaway

2.10 Motivation for Deep Neural Network (Deep Learning)

Why Go Deeper Than Shallow Networks?

Shallow Neural Networks can approximate any function — *but...*

- Require exponentially many hidden units to model complex functions

Why Go Deeper Than Shallow Networks?

Shallow Neural Networks can approximate any function — *but...*

- Require exponentially many hidden units to model complex functions
- Struggle with high-dimensional input (e.g., images, sequences)

Why Go Deeper Than Shallow Networks?

Shallow Neural Networks can approximate any function — *but...*

- Require exponentially many hidden units to model complex functions
- Struggle with high-dimensional input (e.g., images, sequences)
- Lack the ability to learn hierarchical features

Why Go Deeper Than Shallow Networks?

Shallow Neural Networks can approximate any function — *but...*

- Require exponentially many hidden units to model complex functions
- Struggle with high-dimensional input (e.g., images, sequences)
- Lack the ability to learn hierarchical features
- Need heavy feature engineering in many tasks

Why Go Deeper Than Shallow Networks?

Shallow Neural Networks can approximate any function — *but...*

- Require exponentially many hidden units to model complex functions
- Struggle with high-dimensional input (e.g., images, sequences)
- Lack the ability to learn hierarchical features
- Need heavy feature engineering in many tasks

Why Go Deeper Than Shallow Networks?

Shallow Neural Networks can approximate any function — *but...*

- Require exponentially many hidden units to model complex functions
- Struggle with high-dimensional input (e.g., images, sequences)
- Lack the ability to learn hierarchical features
- Need heavy feature engineering in many tasks

Deep Neural Networks (DNNs) solve these by composing multiple non-linear transformations.

Success Stories of Deep Learning

- **Computer Vision:** Object detection, facial recognition, medical imaging

Success Stories of Deep Learning

- **Computer Vision:** Object detection, facial recognition, medical imaging
- **Natural Language Processing:** Translation, chatbots, summarization

Success Stories of Deep Learning

- **Computer Vision:** Object detection, facial recognition, medical imaging
- **Natural Language Processing:** Translation, chatbots, summarization
- **Speech & Audio:** Voice assistants, speech-to-text

Success Stories of Deep Learning

- **Computer Vision:** Object detection, facial recognition, medical imaging
- **Natural Language Processing:** Translation, chatbots, summarization
- **Speech & Audio:** Voice assistants, speech-to-text
- **Games & Robotics:** AlphaGo, autonomous driving, robotic arms

Success Stories of Deep Learning

- **Computer Vision:** Object detection, facial recognition, medical imaging
- **Natural Language Processing:** Translation, chatbots, summarization
- **Speech & Audio:** Voice assistants, speech-to-text
- **Games & Robotics:** AlphaGo, autonomous driving, robotic arms
- **Healthcare:** Protein folding (AlphaFold), drug discovery

Success Stories of Deep Learning

- **Computer Vision:** Object detection, facial recognition, medical imaging
- **Natural Language Processing:** Translation, chatbots, summarization
- **Speech & Audio:** Voice assistants, speech-to-text
- **Games & Robotics:** AlphaGo, autonomous driving, robotic arms
- **Healthcare:** Protein folding (AlphaFold), drug discovery

Success Stories of Deep Learning

- **Computer Vision:** Object detection, facial recognition, medical imaging
- **Natural Language Processing:** Translation, chatbots, summarization
- **Speech & Audio:** Voice assistants, speech-to-text
- **Games & Robotics:** AlphaGo, autonomous driving, robotic arms
- **Healthcare:** Protein folding (AlphaFold), drug discovery

All powered by deep neural networks trained on large datasets and compute.

What's Next: Deep, Adaptive, and Online Learning

Beyond shallow networks:

- **Deep Neural Networks (DNNs)**: Multiple hidden layers; better abstraction

What's Next: Deep, Adaptive, and Online Learning

Beyond shallow networks:

- Deep Neural Networks (DNNs): Multiple hidden layers; better abstraction
- Convolutional Neural Networks (CNNs): For spatial data like images

What's Next: Deep, Adaptive, and Online Learning

Beyond shallow networks:

- **Deep Neural Networks (DNNs)**: Multiple hidden layers; better abstraction
- **Convolutional Neural Networks (CNNs)**: For spatial data like images
- **Recurrent Neural Networks (RNNs)**, **Long Short-Term Memory (LSTMs)** and **Transformers**: For sequences, time series data

What's Next: Deep, Adaptive, and Online Learning

Beyond shallow networks:

- **Deep Neural Networks (DNNs)**: Multiple hidden layers; better abstraction
- **Convolutional Neural Networks (CNNs)**: For spatial data like images
- **Recurrent Neural Networks (RNNs)**, **Long Short-Term Memory (LSTMs)** and **Transformers**: For sequences, time series data
- **Online Learning**: Adapt models as data arrives (e.g., stock prediction)

What's Next: Deep, Adaptive, and Online Learning

Beyond shallow networks:

- **Deep Neural Networks (DNNs)**: Multiple hidden layers; better abstraction
- **Convolutional Neural Networks (CNNs)**: For spatial data like images
- **Recurrent Neural Networks (RNNs)**, **Long Short-Term Memory (LSTMs)** and **Transformers**: For sequences, time series data
- **Online Learning**: Adapt models as data arrives (e.g., stock prediction)
- **Transfer Learning**: Reuse learned models on new tasks (e.g., BERT, ResNet)

What's Next: Deep, Adaptive, and Online Learning

Beyond shallow networks:

- **Deep Neural Networks (DNNs)**: Multiple hidden layers; better abstraction
- **Convolutional Neural Networks (CNNs)**: For spatial data like images
- **Recurrent Neural Networks (RNNs)**, **Long Short-Term Memory (LSTMs)** and **Transformers**: For sequences, time series data
- **Online Learning**: Adapt models as data arrives (e.g., stock prediction)
- **Transfer Learning**: Reuse learned models on new tasks (e.g., BERT, ResNet)
- And Many More....

What's Next: Deep, Adaptive, and Online Learning

Beyond shallow networks:

- **Deep Neural Networks (DNNs)**: Multiple hidden layers; better abstraction
- **Convolutional Neural Networks (CNNs)**: For spatial data like images
- **Recurrent Neural Networks (RNNs)**, **Long Short-Term Memory (LSTMs)** and **Transformers**: For sequences, time series data
- **Online Learning**: Adapt models as data arrives (e.g., stock prediction)
- **Transfer Learning**: Reuse learned models on new tasks (e.g., BERT, ResNet)
- And Many More....

What's Next: Deep, Adaptive, and Online Learning

Beyond shallow networks:

- **Deep Neural Networks (DNNs)**: Multiple hidden layers; better abstraction
- **Convolutional Neural Networks (CNNs)**: For spatial data like images
- **Recurrent Neural Networks (RNNs)**, **Long Short-Term Memory (LSTMs)** and **Transformers**: For sequences, time series data
- **Online Learning**: Adapt models as data arrives (e.g., stock prediction)
- **Transfer Learning**: Reuse learned models on new tasks (e.g., BERT, ResNet)
- And Many More....

Deep learning opens doors to AI that adapts, understands, and generalizes.